

---

# PERTURBATION DOSE RESPONSES IN RECURSIVE LLM LOOPS

*Raw switching, stochastic floors, and persistent escape  
under append, replace, and dialog updates*

---

A PREPRINT

**Pawel Kaplanski**   
Kaplanski Ai Lab  
pawel@kaplanski.ai

April 30, 2026

**Keywords** recursive LLM loops · perturbation dose response · attractor-like regimes · context-update rules · basin switching · dialog dynamics

## ABSTRACT

Recursive language-model loops often settle into recognizable attractor-like patterns, and the practical question is how much injected text is needed to move a settled loop somewhere else, and whether that move lasts. Existing taxonomies classify loop shapes such as contraction, oscillation, collapse, and dialog modes, but they do not measure the operational stability of those regimes under perturbation. We formalize recursive inference as a state-generator-nudge system, separating the model from the context-update rule. Because a perturbed trajectory’s final-cluster disagreement with its paired control can reflect any of three distinct phenomena, true redirection, ordinary stochastic divergence, or a transient kick that later recovered, we measure perturbation dose response with three endpoints: *raw switching*, where the perturbed trajectory’s final cluster differs from its paired control; *net switching*, raw switching after subtracting the natural control-vs-control stochastic floor; and *persistent escape*, where the injection causes a visible basin jump that remains through the terminal step.

The headline result is that append-mode continuation has a real but bounded in-distribution dose response: adversarial continuations produce raw-switching  $ED50_{\text{raw}} \approx 40$  tokens, with convergent estimates from pooled logistic fitting, mixed-effects modeling, and family-cluster bootstrap, but the strict endpoint, visible basin jump that remains through the terminal step, is not reached at any tested dose up to 400 tokens. But this is not persistent redirection. Raw switching plateaus near 67%, paired controls already diverge at about 35%, net switching never reaches +50 percentage points, and persistent escape is not reached at any tested dose up to 400 tokens. Meanwhile, replace-mode loops that appear to switch near 100% mostly do so because the update rule overwrites the state; insert-mode probes reduce switching to 12-32%.

Practically, this means recursive-loop evaluations should distinguish transient movement from durable escape, always subtract stochastic floors, and treat context-update rules as first-class safety-relevant design choices rather than implementation details. We report 37 experiments on `gpt-4o-mini`, with within-vendor replication on `gpt-4.1-nano` and public code, configurations, trajectories, and reports (<https://github.com/kaplan196883/llmattr>).

---

## Plain-language summary

### Why it matters

A common folk-belief about recursive LLM loops is that they either "get stuck in attractors" or are "easy to hijack with a small prompt." Both claims are too vague to be useful. A loop can visibly move without staying moved; two runs of

the same prompt can diverge without any attack; and some apparent "fragility" is caused by the system's context-update rule rather than by the model crossing a real basin boundary. The missing question is not just whether recursive loops have regimes, but how moveable those regimes are under controlled perturbation.

### What we did and what we found

We repeatedly fed an LLM its own outputs, injected text at a chosen step, and measured whether the trajectory changed relative to a paired control. The key result is sharp: in append-mode continuation, adversarial in-distribution text produces a measurable raw dose response with  $ED50_{\text{raw}} \approx 40$  tokens, but durable escape is not achieved even at 400 tokens. Raw final-cluster switching rises and plateaus near 67%, but paired control runs already disagree about 35% of the time from sampling noise alone, so the net effect saturates around +32 percentage points. The strict endpoint, kicked into a new basin and still there at the end, never crosses 50%. Replace-mode loops look much more fragile, but a direct overwrite-vs-insert probe shows that most of that "switching" is simply the update rule overwriting the state.

### Three numbers to remember

- **ED50<sub>raw</sub>  $\approx$  40 tokens** for adversarial in-distribution append-mode continuation.
- **Stochastic floor  $\approx$  35%**, paired control runs already disagree at this rate from sampling alone.
- **Persistent escape not reached** in the tested 5-400 token range; maximum 16% under canonical clustering at 400 tokens.
- **Replace-mode "fragility" drops to 12-32%** when the update rule does not overwrite the state.

### Practical implications

1. **Stress-test recursive agents with persistence, not motion.** The right robustness question is not "did the trajectory move after the perturbation?" but "did it escape and stay escaped after  $N$  more recursive steps?" An evaluation that counts only immediate displacement or final raw cluster disagreement will over-report fragility, because much of the movement is transient or stochastic.
2. **Always measure the stochastic floor.** In the main append-mode setting, two paired control runs already end in different clusters about 35% of the time with no perturbation. That means a reported 50% switching rate is not automatically evidence of successful redirection; much of it may be ordinary sampling divergence. Recursive-loop evaluations should include control-vs-control baselines and report raw, net, and persistent-escape rates separately.
3. **Treat context-update rules as safety-critical design choices.** Append-style updates preserve prior context, creating a real but bounded barrier: about 40 tokens for raw switching, with persistent escape not reached in the tested range. Replace-style updates do not show the same kind of injected-token barrier, because the rule itself discards the old state. Claims that "replace-mode loops are easy to redirect" should be read as claims about overwrite mechanics, not necessarily about weak model attractors.
4. **Do not assume small adversarial nudges permanently redirect settled loops.** Even adversarial continuations selected from the same regime top out near 67% raw switching and fail to produce 50% persistent escape up to 400 tokens. For jailbreak-style or agent-redirection evaluations, this means the meaningful target is sustained post-attack behavior, not a one-step perturbation success.
5. **Evaluating indirect prompt injection (IPI).** Most IPI benchmarks today report whether the model executes the injected instruction at the next step, that is the *raw-switching* endpoint. A defense that pushes raw-switching ED50 from 40 to 200 tokens is a quantitative defense improvement; a defense that prevents *persistent escape* is a qualitatively stronger claim, the model may execute the injection but recovers within a few turns. Serious IPI evaluations should report all three endpoints AND the stochastic floor (so that sampling-noise compliance isn't counted as defense failure), AND distinguish injections that land in the system prompt or replaced context (replace-style, overwhelms by overwriting) from injections that land in tool output or appended document text (append-style, has a real but measurable token barrier). A defense that only blocks immediate compliance is leaving durable-redirection attacks on the table.
6. **Adapting the protocol to human-LLM influence studies.** The lorem / neutral / adversarial content contrast is the most directly transferable piece of the framework for measuring how LLMs affect users. A study aiming to detect content-specific LLM influence on humans (rather than generic "the LLM was in the loop" effects) should run three matched conditions: (i) targeted LLM intervention, (ii) generic on-topic LLM chat without the targeted move, (iii) off-topic LLM small-talk, against a no-LLM control. If the targeted condition shows persistent change relative to the generic and off-topic conditions, the influence is content-specific. If all

three look similar, what was measured is *generic-LLM-presence* (Hawthorne / engagement effect), not the LLM’s content. The three-endpoint decomposition then applies on the human side: did the user’s stated goal / sentiment change at the LLM turn (raw), above the natural drift two humans show without an LLM (net), and was the change still in place several turns or sessions later (persistent)?

7. **Hallucination-detection design.** This framework does not detect hallucinations directly, it measures embedding-space stability, not factuality, and a locked-in hallucination is a stable hallucination. But three pieces transfer to hallucination evaluations once an external grounding signal (retrieval, fact-checker, gold answer) is available. (i) Self-consistency detectors that flag "the model gave two different answers, must be hallucinating" assume between-run agreement is noise-free; on our setting paired runs already diverge 35% of the time from sampling alone. Such detectors should subtract a measured stochastic floor (paired seeds, identical prompt) before scoring. (ii) Transient vs persistent hallucination is a real distinction: a hallucinated claim that the model self-corrects within a turn is qualitatively less dangerous than one that propagates. The persistent-escape endpoint maps directly, measure whether the hallucinated content stays in the trajectory  $N$  turns later. (iii) Hallucination *robustness* under counter-evidence has a clean dose-response framing: how many tokens of contradicting evidence make the model retract? Same protocol, factuality axis instead of cluster axis.
8. **Use the protocol for model and architecture comparisons.** The raw / net / persistent-escape decomposition gives a portable measurement unit for comparing models, prompts, memory policies, and agent-loop architectures. Instead of saying one system is "more stable", an evaluator can report the dose-response curve, stochastic floor, plateau, and persistent-escape threshold, numbers that are directly comparable across vendors.

## Caveats

The main generator is gpt-4o-mini, with within-vendor replication on gpt-4.1-nano but no full cross-vendor replication yet. The tested perturbation range is 5-400 tokens, so persistent escape may appear at higher doses. The regimes are operational embedding-space measurements, not mechanistic proofs of classical attractors. The human-LLM, IPI, and hallucination-detection implications above describe how the *measurement protocol* transfers; the paper itself does not collect human-impact data, production-IPI data, or factuality-graded data, and embedding-space stability is not the same construct as factual correctness.

# 1 Introduction

## 1.1 Phenomenon

Recursive LLM systems increasingly feed model outputs back into future prompts: agents revise plans, assistants summarize tool results, and dialog systems carry state forward. Such loops often appear to settle into attractor-like regimes, but an operational question remains unresolved: how many injected tokens are required to move a settled loop, and does that movement persist?

We answer this by separating the generator from the context-update rule. In append-mode continuation, adversarial in-distribution perturbations produce a real raw dose response, with  $ED50_{\text{raw}} \approx 40$  tokens, but paired controls already diverge about 35% of the time and persistent escape is not reached for doses up to 400 tokens. In replace-mode, apparent fragility is largely an overwrite effect of the update rule. Thus the stability of recursive loops is not a property of the model alone; it is jointly determined by model, memory policy, perturbation content, and persistence criterion.

The same distinction appears in coding agents, where the loop state may include tool logs, patches, summaries, pinned requirements, and recent files; the memory policy determines whether new information accumulates, overwrites, or is role-structured.

The state-generator-update view makes this distinction explicit: the state is the prompt-visible memory at the current step, the generator is the stochastic language model, and the update rule maps state and output into the next state. If the term context-update nudge is used, it denotes this update operation, not a separate intervention inside the model.

Recent work has shown that recursive LLM trajectories can exhibit contractive, oscillatory, exploratory, degenerate, or convergent self-referential regimes. These studies establish that attractor-like structure is empirically visible, but they do not measure the perturbation dose required to move a trajectory between regimes, nor do they separate that dose from ordinary stochastic divergence or from update-rule overwrite mechanics.

The resulting measurement problem has three parts. For a settled recursive trajectory, how many injected tokens are required to produce final-cluster switching, how much of that switching exceeds the stochastic control floor, and how often does the perturbation produce a visible basin jump that persists to the terminal step? A single final disagreement

cannot answer this, because it may reflect real redirection, unperturbed sampling divergence, or a transient displacement followed by recovery.

## 1.2 Question

We study recursive LLM loops as bounded stochastic systems under a state-generator-update decomposition. The same generator can behave differently under append, replace, and dialog updates because those updates expose different histories, role structures, and memory contents to the next sampling step.

1. **Architecture:** How do append, replace, and dialog update rules alter the accessible loop state?
2. **Dose response:** For a settled trajectory, how does switching probability vary with injected-token dose and perturbation type?
3. **Persistence:** Which apparent switches exceed the stochastic control floor and remain through the terminal step?

These questions convert "the loop gets stuck" into measurable quantities: raw switching, net switching, and persistent escape. Raw switching is terminal cluster disagreement between a perturbed trajectory and its paired control. Net switching subtracts the measured control compared with control stochastic floor. Persistent escape requires a visible basin jump after injection that remains present at the terminal step.

## 1.3 Contributions

The paper’s contributions are best stated as five claim-level takeaways.

**Claim 1: recursive-loop stability is jointly determined by generator and update rule (§3).** Append, replace, and dialog loops differ because their context-update operators expose different histories to the model. The state-generator-update formalism treats the update operator as a first-class component of the loop, rather than an implementation detail. It also yields a finite-time access result for replace-mode loops and motivates the append-mode prediction that accumulated prior context changes perturbation response.

**Claim 2: perturbation response decomposes into raw switching, net switching, and persistent escape (§3, §5).** These endpoints separate true redirection from sampling divergence and transient displacement. Raw switching measures final-cluster disagreement with a paired control. Net switching subtracts the natural control compared with control floor. Persistent escape is stricter: the injected text must produce a visible basin jump that remains through the terminal step. The strict stability question is therefore not whether a run moves immediately after injection, but whether it stays moved after subsequent recursive updates.

**Claim 3: append-mode continuation has a finite raw dose response but no observed persistent-escape threshold in the tested range (§5.6).** In the dense adversarial in-distribution append-mode rerun,  $ED50_{\text{raw}}$  estimates are 36, 41, and 52 tokens under pooled four-parameter logistic fitting, mixed-effects logistic modeling, and family-cluster bootstrap, respectively. Raw switching rises with dose but plateaus near 67%, while paired controls already diverge about 35% of the time. The largest observed net effect is therefore about +32 percentage points at 400 tokens, and  $ED50_{\text{persist}}$  is not reached for any tested dose from 5 to 400 tokens. Out-of-distribution neutral and lorem perturbations remain close to a drift floor rather than matching the adversarial continuation response.

**Claim 4: replace-mode apparent fragility is largely update-rule overwrite, not necessarily weak attractor structure (§5.17).** Replace-mode paraphrase and summarize-and-negate loops show near-saturated raw switching across tested doses, but that result follows from the update operator discarding prior state. Insert-mode probes, which preserve the old state while adding the same content, reduce switching to 12 to 32%. This separates overwrite access from durable redirection and makes memory policy a safety-relevant design choice.

**Claim 5: perturbation response resolves regimes that bulk geometry alone cannot distinguish (§5.19).** Drift, dispersion, cluster persistence, and low-dimensional visualisations describe trajectory shape, but they can leave stylistic multi-basin dialog and oscillatory patterns ambiguous. Perturbation dose response adds the missing load test: two regimes with similar bulk geometry can differ in raw switching, stochastic-floor-adjusted switching, and persistent escape. The empirical potential landscape  $V(x) = -\log \rho(x)$  is therefore used as a descriptive view of basin organization, not as an independent substitute for the behavioral endpoints.

All trajectories, configurations, analysis scripts, and replication artifacts are publicly released, with automated checks linking the reported numerical claims to the underlying result tables; within-vendor replication on gpt-4.1-nano is also provided.

## 2 Background and related work

Existing work has begun to describe recursive LLM inference as a dynamical system, identifying convergence, cycling, divergence, and dimensional collapse in semantic space. What remains under-specified is the *mechanism of intervention*: which part of the loop is the model, which part is the context-update rule, and how much external text is required to move a trajectory between empirically supported basins. This section reviews the relevant attractor, degeneration, dimensionality, and dialog literatures, with emphasis on that gap: attractor-like behavior has been observed, but perturbation dose-response and token-scale barrier measurement have not been systematically measured.

### 2.1 Attractors in neural dynamics

Attractor analysis has long been central to the study of recurrent neural systems. In classical recurrent networks, one studies fixed points, cycles, and low-dimensional manifolds through tools such as Jacobian linearization, Lyapunov spectra, stability analysis, recurrence, and effective dimensionality [Hopfield, 1982, Sussillo and Barak, 2013, Maheswaranathan et al., 2019]. These methods ask whether repeated application of a state-update rule drives a system toward stable regions, cycles, or higher-dimensional wandering. The resulting vocabulary of attractors, basins, recurrence, contraction, and instability remains useful even when the underlying system is not a smooth deterministic map.

Recursive language-model loops differ from classical recurrent neural networks in two important ways. First, the visible state is text, not a continuous hidden vector whose update is directly available for differentiation. Second, the loop usually contains a sampling step and a context-update rule chosen by the experimenter. A model may generate text, but the experimenter decides whether that text replaces the previous state, is appended to it, is summarized, is inserted into a dialog transcript, or is mixed with external material. The effective recurrence is therefore not the language model alone. It is the composition of a generator with a context-update mechanism.

For this reason, exact local linearization is generally unavailable at the level of the visible text loop. Empirical analogs are needed. In this paper, recurrence, dwell structure, ensemble spread, finite-time Lyapunov-like growth, effective dimension, clustering, and density-derived landscapes are used as operational diagnostics rather than as proofs of classical attractors. The goal is not to claim that a recursive LLM loop is a smooth dynamical system in the strict mathematical sense. It is to ask whether repeated text transformations show stable, recurrent, or collapsive structure in embedding space, and whether that structure can be perturbed in a measurable way.

### 2.2 Attractor observations in language models

The dynamical-systems framing of LLM inference loops has emerged rapidly. Recent work formalizes recursive LLM transformations as discrete dynamical systems in semantic space and classifies trajectories into regimes such as contractive convergence, oscillation, and exploratory divergence. The closest regime-classification study identifies contractive, oscillatory, and exploratory behavior using drift, dispersion, and cluster-persistence measures. Our use of recurrence, dimensionality, density landscapes, and perturbation response is complementary: these diagnostics are introduced not to relabel the same regimes, but to estimate how robust basin membership is under controlled textual intervention.

That line of work shows that prompt and loop design can select qualitatively different regimes. Iterative paraphrasing may drive semantic contraction, while iterative negation or adversarial transformations may drive dispersion. Such results establish that recursive LLM systems are not merely independent samples from a generator. The text produced at one step shapes the next state, and the chosen update rule can create repeatable geometric structure in the trajectory. This motivates treating the whole loop as the object of analysis.

A related literature studies degeneration in autoregressive generation. Earlier work characterized repetitive collapse, blandness, and looping as sampling and decoding failures, motivating alternatives such as nucleus sampling and more cautious truncation strategies [Holtzman et al., 2020, Carlini et al., 2021]. These studies were not primarily framed in terms of attractors, but they documented phenomena that are naturally read through a dynamical lens: once a text process enters a repetitive template, later outputs may remain trapped in a narrow region of form or content. The attractor vocabulary does not replace the degeneration literature. It provides a way to connect repetitive symptoms to recurrence, dimensionality, and basin stability.

Dimensionality measures are also relevant. Du and Tanaka-Ishii use correlation dimension to quantify geometric complexity in generated text and report dimensionality drops during training transitions and degeneration. Their observable differs from ours, but their result motivates treating dimensional collapse as a relevant diagnostic when recursive loops become repetitive or template-bound. We therefore use dimensionality measures as one component of the attractor-like diagnostic battery, without equating correlation dimension with our basin definitions.

Other work on neural dynamics and representation geometry provides useful measurement precedents. Effective dimension, participation ratio, cluster persistence, and trajectory geometry have been used to characterize how neural and model states occupy representational spaces. In LLM recursion, however, the observable trajectory is mediated by text and sampling. This makes basin claims more fragile than in systems where the latent state transition is directly specified. Robustness tests therefore become central. A cluster that appears stable in an unperturbed trajectory may be a sampling artifact, a prompt-template artifact, or a genuinely supported basin candidate. Perturbation response helps distinguish these possibilities.

### 2.3 What this paper adds

Relative to recent regime-classification studies of recursive LLM loops, this paper shifts the unit of analysis from trajectory shape alone to *intervention cost*. Prior work asks whether trajectories contract, cycle, or disperse. We ask how those regimes respond to controlled perturbations, whether switching probability scales with injected-token dose, and whether dialog-state updates create basin structure not captured by single-stream operators. The comparison is therefore not primarily one of model scale or dataset size, but of observable: perturbation dose-response rather than unperturbed trajectory geometry alone.

This paper makes three conceptual moves. First, it separates the recursive loop into the visible state, the generator, and the context-update nudge. Second, it treats perturbations as calibrated interventions rather than informal prompt changes, allowing barrier height to be operationalized as a dose-response quantity. Third, it treats dialog as a distinct update architecture, not merely as another prompt template, and tests whether role-structured state creates basin structure different from single-stream recursion.

Our labels are not a one-to-one replacement for the contractive/oscillatory/exploratory taxonomy. We retain contractive and oscillatory as comparable high-level behaviors, treat absorbing collapse as a distinct low-diversity endpoint observed under specific recursive operators, and introduce D1/D2 as dialog-specific regimes whose definition depends on perturbation response and state-update structure rather than dispersion alone.

[Tacheny \[2025\]](#) is the closest recent precedent for treating recursive LLM text transformations as discrete dynamical systems in semantic space. That work demonstrates regime classification on a small set of proof-of-concept loops; the present paper extends the framing to perturbation dose-response, explicit generator/nudge separation, dialog-state updates, and larger paired-control trajectory batteries. The emphasis here is not that prior work lacked dynamical concepts, but that the intervention mechanism had not been made the central observable.

The state, generator, and nudge separation is important because recursive LLM behavior is not determined by the model alone. The same generator can be placed in a replacement loop, an append loop, a rolling-window loop, a summarization loop, or a role-structured dialog loop. Each choice changes what information persists, what is forgotten, and how external text enters the next step. A dynamical description that treats the prompt as the whole state can miss this architecture. By explicitly separating these components, one can ask whether an apparent attractor belongs to the generator’s tendencies, to the context-update rule, or to their interaction.

The perturbation focus also changes the meaning of a basin. In unperturbed geometry, a basin candidate may be identified by repeated returns, dense occupancy, contraction, or cluster persistence. Under intervention, the question becomes how much injected text is required to alter the trajectory’s terminal cluster or produce a more durable escape. This yields an operational barrier height in tokens under a specified perturbation protocol. The measure is not a thermodynamic barrier and not a universal property of the model. It is a reproducible dose-response quantity tied to a generator, update rule, observable, clustering convention, injection time, and perturbation family.

This intervention view also clarifies why dialog deserves separate treatment. Dialog updates are not just longer prompts. Role labels, speaker alternation, local adjacency, and conversational memory can determine which parts of the state are salient at the next step. A dialog system may therefore show basin structure tied to conversational role, topic commitment, or local conversational obligations rather than to semantic drift alone. Treating dialog as a nudge architecture allows these possibilities to be tested rather than assumed.

Finally, this paper borrows selectively from related theoretical work while keeping the claims operational. [Tuci et al. \[2026\]](#) study stochastic gradient descent dynamics on neural-network loss landscapes using random dynamical systems and introduce a sharpness-dimension generalization bound near the edge of stability. Their setting is parameter space, Hessian-anchored, and training-time. Ours is embedding space, gradient-free, and inference-time for a frozen generator. We use comparable dimensionality language only as a diagnostic analogy, not as a transfer of their training-dynamics theory.

## 2.4 Effective potential and geometric barriers

Empirical density landscapes are often summarized by an effective potential  $V(x) = -\log \rho(x)$ , where  $\rho$  is an estimated stationary density in a chosen coordinate system. In physical systems this construction is tied to free-energy analysis; here it is used only as a descriptive transformation of trajectory density in embedding space. The resulting landscape can suggest wells, ridges, and low-density separations between frequently visited regions, but its numerical scale depends on the projection, density estimator, and smoothing choices. We therefore treat potential-derived barriers as descriptive geometric summaries, not as thermodynamic quantities or direct estimates of model-internal energy.

This distinction matters because an embedding-space density is not the model’s internal probability distribution. It is a distribution over observed trajectory embeddings after choices about text observable, embedding model, dimensionality reduction, and sampling. A high-density region may correspond to repeated semantic content, repeated rhetorical form, or repeated role-state position. A low-density gap may indicate a meaningful separation between basin candidates, but it may also reflect projection artifacts or sparse sampling. Potential landscapes are therefore used as one part of a triangulation strategy, alongside recurrence, dimensionality, clustering, dwell, and perturbation response.

## 2.5 Dialog as a distinct dynamical setting

Most recursive-LLM dynamics studies focus on single-stream operators such as continuation, paraphrase, or negation. Multi-turn dialog differs because generated text is written into a role-structured conversational state rather than merely appended or replaced as a homogeneous string. This makes dialog a natural test case for whether the context-update rule itself can shape basin structure. Although multi-agent and generative-agent work has studied dialog for capability, memory, and alignment, embedding-space attractor analysis of role-structured recursive dialog remains comparatively undeveloped.

Dialog introduces structure that is invisible in single-stream recursion. Speaker roles determine which text is interpreted as user instruction, assistant response, system constraint, memory, or prior conversational context. Turn order creates local obligations, such as answering the immediately preceding message, while the longer transcript can preserve topic commitments or interactional style. These features make the nudge more than a storage policy. It becomes part of the mechanism that selects what the generator treats as relevant state.

For this reason, dialog is not merely a larger prompt class in this study. It is a separate family of recursive update rules whose stability can be probed by the same perturbation logic used for single-stream operators. If role-structured state creates distinct basin candidates, those candidates should be visible not only in trajectory geometry but also in how they respond to injected text, neutral insertions, adversarial turns, or changes in injection time. This motivates analyzing dialog regimes through both geometry and intervention cost.

## 2.6 Terminology

We use "attractor-like" operationally rather than as a claim about a smooth deterministic system. The analysis proceeds from visible text state  $X_t$ , to an observable text view  $\mathcal{O}(X_t)$ , to an embedding  $z_t$ , to clusters, basin candidates, and finally regime labels. A "nudge" is the context-update rule that writes text into the next state; a "perturbation" is externally injected text used to test stability. A "switch" denotes final-cluster disagreement under the stated perturbation protocol, whereas "persistent escape" requires a durable post-injection basin change.

# 3 Formal framework and hypotheses

This section defines the recursive-loop object studied in the paper, the perturbation estimands used to measure basin switching, and the operational criteria by which we call a regime attractor-like. Numerical ED50 estimates, pass/fail audits, and regime-specific measurements are results and are reported later. The only substantive structural claim retained here is the replace-vs-append distinction: replace mode admits a generation-budget access bound, whereas append mode motivates an injected-token accumulation conjecture.

## 3.1 State, generator, nudge

Every recursive loop studied here has two distinct moving parts. The **generator** is the model itself, which samples the next piece of text. The **nudge** is the memory policy that decides how that text is written back into the next prompt. Prior work often treats these as one object, but they play different roles: the generator determines what can be produced from a state, while the nudge determines what persists as state.

Let  $X_t \in \mathcal{C}$  denote the bounded visible context at step  $t$ , where  $\mathcal{C}$  is the space of valid clipped contexts. In this paper,  $\mathcal{C}$  is instantiated as finite-length character strings produced by tail-clipping at 12,000 characters. Let  $f$  denote the content instruction and  $\eta$  the memory policy. The recurrence is

$$Y_t \sim P_\theta(\cdot \mid X_t; f), \quad X_{t+1} = \mathcal{N}_\eta(X_t, Y_t).$$

Here  $P_\theta(\cdot \mid X_t; f)$  is the language-model generator under instruction  $f$ , for example continuation, paraphrase, summarize-and-negate, or role-alternating dialog. The map  $\mathcal{N}_\eta : \mathcal{C} \times \mathcal{Y} \rightarrow \mathcal{C}$  is the nudge, or context-update operator. The pair  $(f, \eta)$ , not either component alone, defines the loop dynamics.

The canonical nudges used in the experiments are:

$$\mathcal{N}_{\text{append}}(X_t, Y_t) = \text{clip}(X_t \parallel Y_t),$$

$$\mathcal{N}_{\text{replace}}(X_t, Y_t) = \text{clip}(Y_t),$$

$$\mathcal{N}_{\text{dialog}}(X_t, Y_t) = \text{clip}(X_t \parallel \text{format\_turn}(r_t, Y_t)),$$

where  $r_t$  is the role label assigned to the turn and alternates according to the dialog protocol.

| Formal nudge                                                       | Engineering analogue                                                       | Typical risk or behavior                                                        |
|--------------------------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Append: $\text{clip}(X_t \parallel Y_t)$                           | Full transcript, rolling recent context, accumulated tool logs             | Prior evidence remains as ballast; perturbations compete with accumulated state |
| Replace: $\text{clip}(Y_t)$                                        | Summarize-and-continue, scratchpad replacement, single-state whiteboard    | Old state is discarded; whatever enters the replacement becomes privileged      |
| Dialog: $\text{clip}(X_t \parallel \text{format\_turn}(r_t, Y_t))$ | Role-structured chat state, multi-role agents, user/assistant/tool buffers | Recent role-local turns may dominate despite longer accumulated context         |
| Hybrid pinned plus summary                                         | Pinned issue, tests, policy, plus compressed older history                 | Robustness depends on which facts can be overwritten and which remain invariant |

In engineering terms, the nudge is the loop memory policy. It is therefore part of the system’s robustness and security boundary, not merely an implementation detail. The hypotheses below use this factorization directly: changing the content instruction  $f$  can change the generator, while changing the nudge  $\eta$  can change which parts of the generated text become future state.

### 3.1.1 Barrier height as a persistent-escape estimand

Let  $B_1, B_2 \subset \mathcal{C}$  be basin sets in the late-window basin partition defined operationally in the Methods. Let  $t_{\text{inj}}$  be the injection step and let  $X_{t_{\text{inj}}^-}$  denote the state immediately before injection. For an injected dose of  $\tau$  tokens, define the persistent-escape barrier from  $B_1$  to  $B_2$  as

$$B_{\text{persist}}(B_1 \rightarrow B_2) = \inf \left\{ \tau : \Pr[J_\tau(B_1 \rightarrow B_2) = 1, X_T \in B_2 \mid X_{t_{\text{inj}}^-} \in B_1, \text{inject}_\tau] \geq \frac{1}{2} \right\}.$$

Here  $J_\tau(B_1 \rightarrow B_2)$  is the operational injection-step jump indicator defined by the late-window basin partition. Without  $J_\tau$ , the same formula defines a terminal raw-switching barrier, not a persistent-escape barrier.

The unit of  $B_{\text{persist}}$  is injected tokens. This unit is intentionally operational: it asks how much text must be inserted into the loop, under a declared injection protocol, for an injection-step basin jump that persists to the terminal measurement to occur with probability at least one half. Because the nudge determines how injected text is retained, overwritten, or role-labeled, this token-valued barrier is expected to vary with  $\eta$ .

The definition separates two events that are often conflated. A trajectory may terminate in  $B_2$  without an injection-step jump, because it would have drifted there under sampling noise. Conversely, a trajectory may jump at injection and later return to its reference basin. Persistent escape requires both the injection-step jump and the terminal outcome.



### 3.1.2 Operational endpoints for switching and dose response

A perturbed trajectory can finish in a different cluster than its reference trajectory for several reasons: the injection genuinely redirected it, stochastic sampling would have produced a different terminal cluster anyway, or the injection caused a transient jump that later recovered. The following endpoints separate these cases.

Let  $C_{\text{ref}}$  denote the declared reference basin for a perturbed trajectory: in this paper, the pre-injection basin of that same trajectory. Let  $C_{\text{T}}^{(D)}$  be the terminal basin after injected dose  $D$ . Let  $p_0 = \Pr(C_{\text{T}}^{(0,1)} \neq C_{\text{T}}^{(0,2)})$  be the control-vs-control natural switching floor under the same sampling protocol.

Define

$$S_{\text{raw}}(D) = \Pr(C_{\text{T}}^{(D)} \neq C_{\text{ref}}),$$

$$S_{\text{net}}(D) = S_{\text{raw}}(D) - p_0,$$

$$S_{\text{persist}}(D) = \Pr(J_D = 1, C_{\text{T}}^{(D)} \neq C_{\text{ref}}).$$

The corresponding ED50 endpoints are

$$\text{ED50}_{\text{raw}} = \inf\{D : S_{\text{raw}}(D) \geq 0.5\},$$

$$\text{ED50}_{\text{net}} = \inf\{D : S_{\text{net}}(D) \geq 0.5\},$$

$$\text{ED50}_{\text{persist}} = \inf\{D : S_{\text{persist}}(D) \geq 0.5\}.$$

If the relevant set is empty over the tested dose range, the corresponding endpoint is not reached in that range.

These endpoints are different estimands.  $S_{\text{raw}}$  measures terminal disagreement from the declared reference basin. It includes natural stochastic divergence.  $S_{\text{net}}$  subtracts the control-vs-control floor  $p_0$ , so it estimates excess terminal switching above natural drift, but it is not itself a probability.  $S_{\text{persist}}$  measures the stricter event used by the persistent-escape barrier: a jump at injection followed by terminal non-return to the reference basin.

This distinction is central to interpreting perturbation experiments. A raw-switching ED50 can be small if the natural switching floor is large. A persistent-escape ED50 can be unreached even when raw switching is substantial, if most injection-step jumps recover before terminal measurement or if terminal disagreement is dominated by sampling drift. The Results report these endpoints separately.

### 3.1.3 Operational criteria for attractor-like regimes

The term **attractor-like** is used operationally. It does not assert the existence of a smooth deterministic attractor. A regime earns the label only by passing prespecified diagnostics on a measured trajectory ensemble.

> **Definition. Operational attractor-like regime.** Fix a trajectory ensemble, an observable map, an embedding map, a late-window basin partition, and a declared null ensemble. A regime  $r$  is evaluated by four binary criteria: > C1 basin predictability; C2 recurrence or dwell above null; C3 embedder-robust recurrence class; C4 re-entry, contraction, or absorbing collapse. > A strong attractor passes 4/4. An attractor-like regime passes at least 3/4. A regime passing fewer than 3/4 is not an operational attractor under this definition. Missing measurements count as failure unless the criterion is structurally inapplicable and this inapplicability is declared before evaluation.

The thresholds are:

**C1. Basin predictability.** Cross-validated accuracy for predicting the late-window basin from the final pre-terminal state must satisfy

$$A_r^{\text{final}} \geq \tau_{\text{acc}}, \quad \tau_{\text{acc}} = 0.70.$$

The basin partition and predictor protocol are fixed in the Methods. This criterion tests whether late-window basin identity is legible from the measured state rather than being arbitrary cluster noise.

**C2. Recurrence or dwell above null.** Let  $R_r$  be recurrence and  $D_r$  be dwell for regime  $r$ . Let the declared null ensemble provide  $(\mu_{R^{\text{null}}}, \sigma_{R^{\text{null}}})$  and  $(\mu_{D^{\text{null}}}, \sigma_{D^{\text{null}}})$ . The regime passes if

$$\max \left\{ \frac{R_r - \mu_{R^{\text{null}}}}{\sigma_{R^{\text{null}}}}, \frac{D_r - \mu_{D^{\text{null}}}}{\sigma_{D^{\text{null}}}} \right\} \geq 2 \quad \text{and} \quad d \geq 0.5.$$

When both time-shuffled and no-feedback nulls are available, the stronger null gate is used. For dialog runs, the no-feedback null is structurally unavailable; the time-shuffled null is required.

**C3. Embedder-robust recurrence class.** Recurrence is assigned to a coarse bin

$$b_e(r) \in \{\text{high} \geq 0.70, \text{low} \leq 0.40, \text{mid otherwise}\}$$

for each embedding map  $e$ . The criterion passes if the recurrence bin agrees in at least two of three embedders: the canonical embedder plus two alternatives. This criterion tests survival of the regime class under measurement change. It does not require scalar invariance of all diagnostics.

**C4. Re-entry, contraction, or absorbing collapse.** The regime passes if at least one of the following holds:

$$\lambda_{-1, r^{\text{late}}} \leq 0.015$$

for late-window contraction;

$$\text{best\_period} = 2 \quad \text{and} \quad \text{period\_2\_score} > 0$$

for period-two re-entry;

$$R_r \geq 0.90 \quad \text{and} \quad SD_r \leq 1.50$$

for absorbing collapse; or exit-return exceeds the C2 null gate. These alternatives allow different geometric signatures to qualify while still requiring a measured return, contraction, or collapse property.

The criteria above are applied to each experimental regime in the Results. The framework itself does not assume that any regime passes; the pass/fail audit is an empirical outcome.

### 3.1.4 Replace-mode access bound

Replace mode discards the previous context after each generation step. In this regime, the next state depends on the newly generated replacement rather than on an accumulated transcript. This makes the injected-token dose  $\tau$  distinct from the post-injection model-generation budget.

For  $m$  post-injection replace steps, define

$$G_m = \sum_{-s = t_{\text{inj}}^{t_{\text{inj}} + m - 1}} |Y_{-s}|, \quad \bar{G}_m = \mathbb{E}G_m.$$

The quantity  $G_m$  is not injected text. It is the number of tokens generated by the model after the injection.

**Proposition. Replace-mode access bound.** In replace mode, suppose every reachable non-target state has one-step probability at least  $q_0$  of generating a target-basin replacement, target entry persists to terminal measurement with probability at least  $r_0$ , and expected generation length is at most  $\kappa$  per step. Then after  $m$  post-injection replace steps,

$$\Pr(X_T \in B_2) \geq r_0[1 - (1 - q_0)^m], \quad \mathbb{E}G_m \leq \kappa m.$$

This bounds post-injection generated tokens, not injected-token dose. Full lemma, corollaries, and proof in §11.3.

The proposition captures the structural distinction between replace and append memory. In replace mode, once the loop reaches a state from which target-basin replacement is accessible, repeated replacement attempts accumulate probability through generation steps. The relevant finite quantity in the proposition is  $\mathbb{E}G_m$ , the expected post-injection generation budget. It is not  $\tau$ , the length of the injected string.

Consequently, the proposition should not be read as an injected-token barrier bound. If the access assumptions hold only after a particular injected string of length  $\tau$ , then the injection protocol may certify an injected-dose upper bound of at most that  $\tau$  under those conditions. The factor  $\kappa m$  certifies only the generated-token budget after injection. If comparable access holds without injected text, then replace-mode switching is better understood as endogenous reachability of the replacement dynamics rather than as evidence of a positive injected-token barrier.

### 3.1.5 Append-mode accumulation conjecture

Append mode retains prior context:

$$X_{-t} + 1 = \text{clip}(X_{-t} \| Y_{-t}), \quad Y_{-t} \sim P_{-\theta}(\cdot | X_{-t}; f).$$

A perturbation injected into an append-mode loop therefore does not overwrite the current state in one step. Instead, it is incorporated into an already formed bounded context and must compete with incumbent basin evidence that remains visible until displaced by clipping. The injected text, subsequent model outputs, and pre-existing context all coexist inside the same clipped state.

This motivates an injected-token accumulation conjecture rather than a generation-budget theorem. Let  $B_{-1}, B_{-2} \subset \mathcal{C}$  be basin sets for an append-mode loop. There is no reason to expect a one-step overwrite bound analogous to replace mode. Instead, switching should depend on how much basin-relevant counter-context survives inside the effective context window.

**Conjecture. Append-mode accumulation barrier.** For append-mode loops, basin switching from  $B_{-1}$  to  $B_{-2}$  is governed by the accumulated share of semantically legible counter-context inside the clipped state. In particular, there exists a basin-dependent threshold such that subthreshold injections remain near the stochastic floor, while injections that supply sufficient basin-relevant counter-context produce a rising switching curve.

A testable response-curve formulation is as follows. Let  $a_{-\tau}$  be the semantically weighted share of the clipped context occupied by basin-relevant injected text. Append-mode accumulation predicts a nondecreasing switching curve

$$S_{B_{-1} \rightarrow B_{-2}}(\tau) = \Pr(X_{-T} \in B_{-2} | X_{-t_{\text{inj}}} \in B_{-1}, \tau) \approx p_{\text{floor}} + (p_{\text{max}} - p_{\text{floor}}) F(a_{-\tau} - \alpha^*_{B_{-1} \rightarrow B_{-2}}),$$

where  $F$  is increasing and the threshold  $\alpha^*$  is lower for in-distribution counter-context than for out-of-distribution text.

Here  $p_{\text{floor}}$  is the relevant stochastic floor,  $p_{\text{max}}$  is the attainable upper switching level under the protocol, and  $\alpha^*_{B_{-1} \rightarrow B_{-2}}$  is a basin-pair threshold in effective context share. The word "semantically" is important: two injections with the same token count can have different  $a_{-\tau}$  if one is in-distribution counter-context for  $B_{-2}$  and the other is irrelevant or out-of-distribution text. This makes the conjecture falsifiable. It predicts dose dependence for append-mode in-distribution perturbations, weaker or absent dose response for semantically irrelevant perturbations at the same token count, and a threshold that shifts when the perturbation family changes.

A geometric refinement is possible. Let  $V(x) = -\log \rho(x)$  be an empirical potential over the representation space, and let  $V^*(B_{-1}, B_{-2})$  denote the saddle height along a minimum-cost path between basins. One may hypothesize that injected-token barriers in append mode scale with this saddle height up to representation and perturbation-family factors. This geometric relation is not used as a theorem in the framework. It is a secondary hypothesis to be evaluated empirically.

All barrier quantities in this section are operational token counts unless explicitly stated otherwise. Token-valued barriers are tokenizer-dependent and model-specific. A model-normalized alternative would measure injected information in nats using per-token conditional log probabilities under  $P_{-\theta}$ . That requires log-probability capture during generation. The present framework therefore treats nat-valued barriers as a future measurement target rather than as reported quantities.

## 3.2 Observable maps and embedding

Attractor-like structure is not measured directly from raw recurrence definitions alone. We introduce observable maps that select text views of the trajectory, and embedding maps that lift those views into vector space. Let

$$O_{-t} = h(X_{-t}, Y_{-t}, \text{metadata})$$

be an observable string selected from the state, generation, and recorded metadata. Let

$$z_t = \phi(O_t) \in \mathbb{R}^m$$

be its vector representation. For the canonical embedding used in the main battery,  $m = 1536$  for `text-embedding-3-small`. A trajectory therefore produces one or more embedded polylines  $\{z_t^{(O)}\}_{t=0}^{T-1}$ , depending on the observable family.

Observable maps are post-hoc measurement functions; they do not feed back into the recurrence unless explicitly used as part of a nudge. Thus  $X_t$  is the dynamical state,  $Y_t$  is the sampled continuation,  $O_t = h(X_t, Y_t, \text{metadata})$  is the text view selected for measurement, and  $z_t = \phi(O_t)$  is the vector representation used for clustering, recurrence, dwell, and basin-predictability metrics. The Methods section fixes the observable family, embedding normalization, clustering procedures, and null ensembles before applying the C1-C4 criteria.

This separation matters for interpretation. The loop dynamics are defined by  $(P_\theta, f, \mathcal{N}_\eta)$ . The observables and embeddings define the measurement apparatus. A claimed attractor-like regime must therefore survive not only within one projection, but also under the robustness checks in C3. Conversely, failure under one observable does not by itself imply absence of structure in the text dynamics; it implies that the declared measurement battery did not certify the operational definition.

### 3.3 Hypotheses

**H1.** At least one recursive-loop regime passes the operational attractor-like definition under the declared observable, embedding, basin partition, and null ensemble.

**H2.** The pass/fail pattern and attractor subtype depend on both content operator  $f$  and nudge  $\eta$ ; changing  $\eta$  while holding content approximately fixed changes the measured regime class.

**H3.** Perturbation sensitivity differs by nudge: append mode exhibits a dose-dependent in-distribution raw-switching curve above the stochastic floor; replace mode reaches high switching at the smallest tested dose or without dose; dialog lies between these extremes and depends on role/state structure.

**H4.** The qualitative regime labels and endpoint ordering observed in small-N exploration remain unchanged under the large-N battery within declared uncertainty.

## 4 Methods

Before the per-component details below, the experimental skeleton is short: we run paired recursive trajectories from the same seed and prompt; in the treatment run we inject text at a fixed step; we embed every step’s observable output; we project to a low-dimensional space; and we cluster final states jointly across treatment and control runs. The perturbation is summarized by the perturbed run’s final cluster relative to its paired control’s. We separately estimate the control-vs-control divergence rate, namely how often two paired control runs already disagree from sampling alone, and the persistent-escape rate, namely how often a perturbed run jumps clusters at injection and remains in the new cluster at the terminal step. The remainder of §4 details each component.

### 4.1 Primary endpoints and inferential contract

Before listing implementation details, we pre-specify five primary endpoints used for claims: operational attractor score, group-aware basin predictability, perturbation switching signature, behavioral ED50, and persistent escape. All other quantities in §4.5 are diagnostic or visualization-support metrics unless explicitly mapped to one of these endpoints.

This contract separates exploratory dynamics measurements from claim-bearing endpoints. Recurrence, dwell, basin occupancy, periodicity, dispersion, finite-time ensemble-spread spectra, sharpness dimension, flow fields, and density landscapes are used to characterize a regime and to support interpretation. They are not, by themselves, sufficient for a headline claim unless they enter one of the five endpoints defined operationally in §4.13.

The five primary endpoints are:

1. **Operational attractor score**, based on the C1-C4 attractor criteria: late-window basin persistence, recurrence or dwell above null, embedder robustness, and contraction, re-entry, or collapse.
2. **Group-aware basin predictability**, measured by predicting the late basin from early PCA-10 state while holding out prompt families.

3. **Perturbation switching signature**, measured by paired treatment-control final-cluster disagreement, interpreted alongside the paired control-control stochastic floor.
4. **Behavioral ED50**, the token dose at which the O1 adversarial perturbation reaches 50% raw switching, with uncertainty estimated by family-cluster bootstrap and model-based fits.
5. **Persistent escape**, the strict event in which a perturbation induces an injection-time cluster jump and the trajectory remains in that post-injection cluster at the terminal step.

All thresholds, pass rules, and current endpoint status are consolidated in §4.13.

## 4.2 The recurrence

We instantiate the formal recurrence from §3.1 with three context-update rules:

$$\text{Append: } X_{t+1} = \mathcal{N}_{\text{append}}(X_t, Y_t) = \text{clip}(X_t \| Y_t)$$

$$\text{Replace: } X_{t+1} = \mathcal{N}_{\text{replace}}(X_t, Y_t) = \text{clip}(Y_t)$$

$$\text{Dialog: } X_{t+1} = \mathcal{N}_{\text{dialog}}(X_t, Y_t) = X_t \| \text{format\_turn}(r_t, Y_t)$$

where

$$Y_t \sim P_{\theta}(\cdot | X_t; f)$$

and  $P_{\theta}$  is the language-model distribution parameterized by  $\theta$ , here gpt-4o-mini. The clipping operator  $\text{clip}(\cdot)$  truncates context from the head, namely the oldest text, once the running string exceeds 12,000 characters, preserving the most recent state. The content operator  $f$  enters through the system prompt fed to  $P_{\theta}$ , for example "Continue the text" for  $f = \text{continue}$  and "Paraphrase the following" for  $f = \text{paraphrase}$ .

Append mode creates a growing memory trace until the context cap is reached. Replace mode overwrites the state with the latest output, making the loop maximally sensitive to the current sample. Dialog mode alternates formatted turns between two roles and retains the conversation history subject to the same context-management principles. These three update rules define the operator families used throughout the experiments.

## 4.3 Sampling

Each experiment runs

$$N_{\text{traj}} = N_{\text{families}} \times N_{\text{ICs}} \times N_{\text{runs}}$$

trajectories. Publication-scale defaults differ by experiment family:

| experiment family        | design                                                                              | steps | point count                                                   |
|--------------------------|-------------------------------------------------------------------------------------|-------|---------------------------------------------------------------|
| Operator runs O1, O2, O3 | 15 prompt families x 30 initial conditions x 3 runs = 1,350 trajectories per regime | 40    | 54,000 points per experiment                                  |
| Dialog run D1            | 5 dialog-suitable families x 30 initial conditions x 3 runs = 450 trajectories      | 40    | 18,000 points per role, 36,000 effective role-specific points |
| D2 drill-down dialog     | 5 families x 5 initial conditions x 1 run = 25 trajectories                         | 50    | exploratory scale only                                        |

D2 is below the  $N \geq 2$ -runs minimum required for ensemble-spread diagnostics and is therefore treated as exploratory rather than publication-scale evidence.

Initial conditions are short seed texts grouped into families. Families include philosophical prompts, practical-advice prompts, creative-writing prompts, reflective prompts, and emotional prompts. Across families we obtain variation

in topic and tone; within families we obtain variation across seeds. Unless a temperature sweep is explicitly being run, sampling uses temperature  $T = 0.8$ . The Phase 2b temperature sweep varies temperature and, in some cells, `steps_per_run`.

#### 4.4 Embedding

All trajectories are embedded with `text-embedding-3-small`, producing 1536-dimensional vectors. We embed multiple observables per step. Each observable is a different text view of the same recursive state, and all analyses are repeated per observable to expose representation-dependent findings.

| observable                    | source location                                | what it captures                                                             |
|-------------------------------|------------------------------------------------|------------------------------------------------------------------------------|
| <code>output</code>           | <code>core/observables.py</code>               | the model’s $Y_t$ text alone, with no context                                |
| <code>rolling_k3</code>       | <code>core/observables.py</code>               | concatenation of the last 3 outputs                                          |
| <code>context_tail</code>     | <code>core/observables.py</code>               | the last 4000 characters of the running context                              |
| <code>context_full</code>     | <code>core/observables.py</code>               | fixed-window 8000-character tail, used in longer-memory pilot configurations |
| <code>last_user_turn</code>   | <code>experiments/dialog/observables.py</code> | dialog-only: most recent user or role-A utterance                            |
| <code>last_agent_turn</code>  | <code>experiments/dialog/observables.py</code> | dialog-only: most recent agent or role-B utterance                           |
| <code>rolling_user_k3</code>  | <code>experiments/dialog/observables.py</code> | dialog-only: rolling window of last 3 user turns                             |
| <code>rolling_agent_k3</code> | <code>experiments/dialog/observables.py</code> | dialog-only: rolling window of last 3 agent turns                            |
| <code>turn_pair</code>        | <code>experiments/dialog/observables.py</code> | dialog-only: most recent user-agent exchange concatenated                    |

The role-specific observables are essential for dialog analysis because basin scores, recurrence, and predictability can differ strongly when computed on the user’s questions, the agent’s answers, or the concatenated exchange. D1 uses user and agent labels. D2 uses explorer and expert labels. Role names are read from `cfg.dialog.role_a.name` and `cfg.dialog.role_b.name` at embed time, so the observable wiring accepts any configured role-name pair.

For one observable string at one trajectory step, we obtain exactly one 1536-dimensional vector. There is no user-managed chunking, no per-token output, and no sliding window internal to the analysis pipeline. After the embedding API returns, each row is L2-normalized so downstream cosine similarities reduce to dot products. Thus one operator publication trajectory yields 3 vectors per step for `output`, `rolling_k3`, and `context_tail`, or 4 if `context_full` is enabled. One dialog publication trajectory yields 8 vectors per step, or 9 if `context_full` is enabled.

The token budget is held below the 8,191-token embedding limit by construction. Per-step generations are bounded by `max_output_tokens` of 120 for operator runs and 160 for dialog runs. The running context is capped at 12,000 characters, approximately 3,000 English tokens, and every context-based observable slices only the recent tail before embedding. Typical upper bounds are approximately 160 tokens for `output`, 480 for `rolling_k3`, 1,000 for `context_tail`, 2,000 for `context_full`, 320 for `turn_pair`, and 480 for each role-specific rolling window. Exact constructor calls, caching details, adjacent-step similarity checks, cost estimates, and verification snippets are provided in Supplementary §11.4.

#### 4.5 Perturbation materials and paired-run protocol

Perturbation experiments use paired recursive runs from the same seed, prompt family, initial condition, and generation settings. The treatment run receives injected text at a pre-specified injection step  $t_{inj}$ , recorded in the experiment configuration and held fixed within a condition unless the experiment is explicitly an injection-step sweep. Terminal steps are 29 for 30-step perturbation pilots and 49 for 50-step perturbation runs.

##### Perturbation corpora

The neutral word pool was curated to avoid emotional, introspective, or obviously directive vocabulary. The adversarial condition is not an instruction to attack the model. It is adversarial in the dynamical-systems sense: it is a plausible state fragment taken from another trajectory that may redirect the recurrence.

##### Dose definitions

A perturbation dose  $\tau$  is measured in tokens. For dose-parametrized variants, the perturbation text is resized to the requested token length before injection. Sparse O1 adversarial dose sweeps use  $\tau \in \{20, 80, 200, 400\}$ . Dense reruns use more dose levels and larger per-cell sample sizes. Pilot conditions that do not carry an explicit dose label use their default corpus length, for example a full neutral paragraph or a 70-word lorem sample.

| corpus or condition | source                                                                                    | intended semantic role                                             | dose handling                                                                           |
|---------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| control             | no injected text                                                                          | estimates the unperturbed recursive endpoint                       | zero dose                                                                               |
| neutral             | off-topic Wikipedia-style paragraph drawn from the hand-written corpora.NEUTRAL_WIKI pool | coherent but task-irrelevant text                                  | full paragraph in pilot runs; token-resized variants use <code>neutral_&lt;N&gt;</code> |
| lorem               | 70 random English words from the curated neutral corpora._WORD_POOL                       | incoherent or weakly coherent lexical mass with low affective load | fixed 70-word pilot form or token-resized variants                                      |
| adversarial         | late-step output from a different trajectory of the same regime                           | on-manifold text likely to point toward another basin              | token-resized variants for dose-response                                                |

The injection is applied to the recursive state at  $t_{\text{inj}}$  in a condition-specific but pre-specified manner. In append and dialog settings, the injected text is inserted into the context stream as an additional state fragment. In replace settings, the intervention is partly tautological because the state update rule already overwrites context with the current text; this is why replace-mode perturbation results are interpreted as a capitulation signature rather than as evidence of a subtle barrier-crossing mechanism.

### Adversarial-source rule

For adversarial perturbations, the source text must come from a different trajectory than the target. The source is drawn from a different family-initial-condition trajectory of the same regime, using late-step output so that the perturbation resembles an endpoint state rather than an early transient. We exclude self-source cases, missing-source cases, and cases where the source text is empty after tokenization or cannot be resized to the requested dose.

### Paired-control design and exclusion rules

For each seed-level unit we run two unperturbed controls, denoted  $A$  and  $B$ , and one matched treatment, denoted  $Z$ . Control  $A$  is the treatment comparator. Control  $B$  estimates the stochastic floor, namely the rate at which two unperturbed recursive samples disagree at the terminal step from ordinary sampling alone.

A trajectory unit is excluded from perturbation endpoint aggregation if any required member of the tuple  $(A, B, Z)$  is missing; if the terminal step is missing for any member; if embeddings or cluster labels are unavailable for the selected observable; if the adversarial source violates the source rule; if the perturbation text is empty after tokenization; or if the run does not match the configured injection step and terminal horizon. Exclusions are applied before endpoint computation and are counted in audit tables.

### Algorithm 1: paired perturbation evaluation

Input: seed or task  $x$ , generator  $P_\theta$ , context-update rule  $\mathcal{N}$ , injection condition  $c$ , injection step  $t_{\text{inj}}$ , terminal step  $T$ , observable map  $O$ , and equivalence rule  $C$ . In this paper  $C$  is a K-means cluster in joint PCA-10 space, but the algorithm only requires a pre-specified equivalence rule.

1. Run two unperturbed controls:

$$A = \text{RunLoop}(x, P_\theta, \mathcal{N}, \text{no injection})$$

$$B = \text{RunLoop}(x, P_\theta, \mathcal{N}, \text{no injection})$$

2. Estimate the stochastic-floor event:

$$\text{floor} = [C(O(A_T)) \neq C(O(B_T))]$$

3. Run the matched treatment:

$$Z = \text{RunLoop}(x, P_\theta, \mathcal{N}, \text{inject } c \text{ at } t_{\text{inj}})$$

4. Define raw switching:

$$\text{raw} = [C(O(Z_T)) \neq C(O(A_T))]$$

5. Define the injection-time jump:

$$\text{jump} = [C(O(Z_{t_{\text{inj}}} + 1)) \neq C(O(Z_{t_{\text{inj}}} - 1))]$$

6. Define persistent escape:

$$\text{persist} = \text{jump} \wedge [C(O(Z_T)) = C(O(Z_{t_{\text{inj}}} + 1))]$$

7. Aggregate over seeds, tasks, and families:

$$\begin{aligned} \text{raw\_rate} &= \mathbb{E}[\text{raw}], \quad \text{floor} = \mathbb{E}[\text{floor}], \quad \text{net\_rate} = \text{raw\_rate} - \text{floor} \\ \text{persistent\_escape\_rate} &= \mathbb{E}[\text{persist}] \end{aligned}$$

This same endpoint decomposition applies to non-embedding observables in engineering settings, including final patch family, files touched, test pass/fail sets, tool-call sequences, plan categories, security-policy violations, or embeddings of full traces.

#### 4.6 Representation spaces and metric battery

For each observable’s embedding matrix  $Z \in \mathbb{R}^{N \times 1536}$ , we compute joint projections across the full point cloud. Projections are never fit per-run or per-family, because coordinates must be comparable across trajectories, conditions, and roles.

PCA-2 is used for density estimation, potential landscapes, and two-dimensional plots. For short-output observables it typically carries 10% to 15% of total variance; for longer-context observables it can carry approximately 25%. PCA-10 is used for K-means clustering, basin classification, basin predictability, recurrence, dwell, and most endpoint-level metrics. PCA-50 is used only as a pre-reduction stage before t-SNE.

t-SNE is fit after PCA-50 pre-reduction using cosine distance, perplexity 30 capped at  $(N - 1)/4$  for small  $N$ , PCA initialization, automatic learning rate, and random seed 42. t-SNE is computed once per experiment and observable. It is used only for visualization because local neighborhoods are informative but global distances are not physically interpretable. Quantitative metrics are computed in PCA-10 unless explicitly stated otherwise. Exact constructor calls and verification snippets are provided in Supplementary §11.4.

In perturbation experiments, PCA and clustering are fit jointly across all conditions. In dialog experiments, the relevant observable-specific point cloud is projected jointly before role-specific subsets are inspected. This joint-fit rule is required for comparing basins, geodesics, switching events, and role-conditioned trajectories.

##### 4.6.1 Regime-structure diagnostics

**Definition.** Regime-structure diagnostics summarize the temporal organization of a single trajectory or a set of trajectories in PCA-10 cluster space.

Recurrence for trajectory points  $z_0, \dots, z_{T-1}$  is

$$\text{recurrence}(\epsilon, \tau) = \frac{\#\{(t, s) : \|z_t - z_s\|_2 < \epsilon \wedge |t - s| > \tau\}}{T(T-1)/2}$$

with  $\epsilon = 0.15$  and  $\tau = 3$ . The lag exclusion suppresses trivially nearby adjacent steps.

Dwell is computed after K-means clustering with  $k = 12$  in PCA-10. It is the run-length distribution within a cluster. Long dwell times indicate basin persistence; short dwell times indicate transience or rapid cycling.

A target region is defined as the K-means cluster containing the trajectory’s final 30% of points. Basin score is the fraction of post- $t^*$  points in that cluster, with  $t^* = 0.7T$ . Basin entry is the first step at which the trajectory’s cluster matches its late-window target.

Late recurrence restricts the recurrence statistic to the second half of the trajectory. Exit-return asks whether a trajectory that has visited its target basin subsequently leaves and re-enters it. This separates tight basins from metastable basins.

Periodicity is measured by lag-distance autocorrelation. For lag  $k$ ,

$$\text{mean\_dist}(k) = \mathbb{E}_t [\|z_t - z_{t+k}\|_{\cos}].$$

The period-2 score is  $\text{mean\_dist}(1) - \text{mean\_dist}(2)$ . Positive values indicate that lag-2 points are closer than lag-1 points. Analogous scores are computed for period 3, and the best period is the lag  $k \in [1, T/2]$  minimizing the mean distance.



Dispersion compares ensemble spread early and late in the trajectory. Initial dispersion is the mean pairwise distance over  $t \in [0, T/4]$ . Final dispersion is the mean pairwise distance over  $t \in [3T/4, T]$ . Dispersion growth is  $(\text{final} - \text{initial}) / \text{initial}$ . Global drift is the distance between the centroid at the terminal step and the centroid at the initial step. Drift monotonicity is the correlation between centroid distance and step.

**Use in claims.** These diagnostics feed the operational attractor score and the three-axis classifier. Recurrence, dwell, basin score, basin entry, late recurrence, and exit-return support the convergence and persistence components of the attractor score. Periodicity is the primary diagnostic for O2-style oscillatory regimes. Dispersion and drift support the distinction between contractive, oscillatory, absorbing, and divergent behavior.

**Rationale or limitation.** These metrics test whether the trajectory has temporal structure beyond its marginal point cloud. A high recurrence or dwell statistic is not sufficient on its own because the same point cloud can appear recurrent after time shuffling. Conversely, a low recurrence statistic on output can coexist with strong basin persistence on `context_tail` if the individual generations fluctuate while the integrated context is stable. For this reason, regime claims require baseline comparison and cross-observable agreement.

#### 4.6.2 Ensemble-spread diagnostics

**Definition.** We compute a finite-time ensemble-spread spectrum from multiple runs sharing the same family and initial condition. We use the term finite-time Lyapunov-like exponent for an ensemble-spread growth rate, not for a Jacobian-derived exponent of a smooth map.

For each family-initial-condition cell with  $N$  runs, the embeddings at step  $t$  define a covariance matrix

$$\Sigma_t = \frac{1}{N-1} \sum_i (z_{i^t} - \bar{z}^t)(z_{i^t} - \bar{z}^t)^\top.$$

Let  $\mu_k(t)$  be the  $k$ -th eigenvalue of  $\Sigma_t$ . The  $k$ -th finite-time ensemble-spread exponent over a window from  $t_{\text{baseline}}$  to  $T-1$  is

$$\lambda_k = \frac{1}{2(T-1-t_{\text{baseline}})} \log \left( \frac{\mu_k(T-1)}{\mu_k(t_{\text{baseline}})} \right).$$

The factor  $1/2$  converts variance growth to amplitude growth. We compute early and late windows. The early window captures transient divergence as runs move away from the seed. The late window captures on-attractor or post-transient behavior.

We also compute sharpness dimension and effective rank as secondary diagnostics. Sharpness dimension uses the ordered spectrum and the Tuci-style fractional form

$$j^* = \max \left\{ i : \sum_{k \leq i} \lambda_k \geq 0 \right\}$$

with  $j^* = 0$  if  $\lambda_1 < 0$ , and

$$\text{SD} = j^* + \frac{\sum_{k \leq j^*} \lambda_k}{|\lambda_{j^*} + 1|}.$$

If the spectrum is everywhere negative, SD is 0. If the cumulative sum remains non-negative through the full spectrum, SD is  $d$ . Effective rank counts exponents above  $-0.01$ .

**Use in claims.** The late ensemble-spread spectrum contributes to the contraction component of the operational attractor score. A negative or shrinking late spread supports a contractive or absorbing interpretation. A positive early exponent combined with late contraction indicates transient divergence followed by settling. Sharpness dimension and effective rank are reported as secondary diagnostics, not as primary endpoints.

**Rationale or limitation.** Text generation is discrete and sampling-based, so there is no smooth one-step Jacobian from which to derive a classical Lyapunov spectrum. The spectrum here measures growth or contraction of stochastic ensemble spread under matched prompts and seeds. With  $N = 3$  runs per initial condition, the covariance rank is at most 2, so the spectrum length is at most 2 and sharpness dimension can saturate at 2.0. These diagnostics are comparative across regimes, not absolute estimates of a dynamical invariant.

### 4.6.3 Predictability endpoint

**Definition.** Basin predictability asks whether the late-window basin can be predicted from early state. For each trajectory, the target label  $y$  is the K-means cluster occupied in the late window. For each early step  $k$ , we train a multinomial logistic regression from the PCA-10 state at step  $k$  to  $y$ .

Two cross-validation schemes are reported. The first is stratified  $k$ -fold cross-validation after dropping singleton classes that cannot be split into train and test folds. The number of folds is adaptive:  $n\_splits = \min(5, \text{smallest class size})$ . If fewer than two non-singleton classes remain, the cell is recorded as missing. The second is group-aware cross-validation with prompt family held out as the grouping variable. This GroupKFold analysis tests whether predictability generalizes across prompt families rather than exploiting family identity.

Audit columns record the number of dropped singleton classes and trajectories. Publication-scale runs usually reach 5 stratified folds. Reduced-scope sweeps and pilots can fall back to 2 to 4 folds.

**Use in claims.** Group-aware basin predictability is one of the five primary endpoints. To claim cross-family basin predictability, accuracy at step  $k = 10$  must satisfy

$$\text{acc\_group}(k = 10) \geq 0.70.$$

To claim that the original stratified estimate is leakage-free, the difference

$$\Delta = \text{acc\_stratified} - \text{acc\_group}$$

must be below 0.10. Stratified accuracy is reported as a diagnostic, but it is not sufficient for the leakage-free endpoint.

**Rationale or limitation.** A high stratified accuracy can arise because prompt families occupy different regions of embedding space. Group-aware splitting is therefore the stricter test: the model must predict the late basin for held-out families. This endpoint is intentionally conservative. It can fail even when within-family predictability is high, because the claim is cross-family generalization.

### 4.6.4 Perturbation-response endpoints

**Definition.** Perturbation-response endpoints are computed from the paired-control design in §4.4. Raw switching is the fraction of treatment runs whose terminal cluster differs from paired control  $A$ . The stochastic floor is the fraction of paired controls  $A$  and  $B$  whose terminal clusters differ without perturbation. Net switching is raw switching minus the stochastic floor. Persistent escape is the stricter event in which the treatment changes cluster immediately after injection and remains in that post-injection cluster at the terminal step.

The behavioral ED50 is estimated from dose-response data. It is the token dose  $\tau$  at which the fitted O1 adversarial dose-response reaches 50% raw switching. Fits include a four-parameter logistic curve, a generalized linear mixed model, and family-cluster bootstrap summaries. The endpoint is reported with uncertainty intervals and with a separate assessment of whether net switching or persistent escape reaches the same threshold.

**Use in claims.** The perturbation switching signature is a primary endpoint. For O1 selective sensitivity, adversarial switching at 200 tokens must be at least 0.50, must be at least twice the larger of neutral or lorem switching at the same dose, and the maximum out-of-distribution neutral or lorem switching must be at most 0.30. For O2 and O3 replace-mode perturbations, capitulation is supported if all non-control perturbation conditions produce switching of at least 0.85, with the interpretive caveat that replace mode overwrites state.

Behavioral ED50 is a primary endpoint only for raw O1 adversarial switching. A localized token barrier requires a finite point estimate and a 95% family-cluster bootstrap interval contained within the probed dose interval. Net ED50 and persistent ED50 are reported separately. If the net effect or persistent escape does not reach 50% in the tested range, those stricter ED50 values are undefined.

**Rationale or limitation.** Raw switching alone is deliberately sensitive to ordinary stochastic divergence. It is meaningful only when read alongside the paired control-control floor and persistent-escape rate. Persistent escape is stricter but can undercount cases where a perturbation changes the long-term basin through intermediate clusters. ED50 is a behavioral dose summary, not a physical energy barrier. The term barrier is therefore operational: it refers to a token-valued perturbation scale under the specified equivalence rule and sampling design.

#### 4.6.5 Three-axis classifier summary

**Definition.** The three-axis classifier maps diagnostic metrics to three hypotheses. H1a, convergence to a basin, uses basin-positive and dwell-above-null signals. H1b, recurrence or oscillation, uses late recurrence above null, period-2 score above threshold, and best-period majority greater than 1. H1c, divergence or no attractor, uses dispersion growth, monotonically outward drift, and absence of a stable basin. Signal counts are converted to qualitative strengths: not supported, weak, moderate, or strong.

**Use in claims.** The classifier provides a structured summary that justifies regime labels. O1 contractive behavior is expected to show strong H1a and weak H1b. O2 oscillation is expected to show strong H1b driven by period-2 structure. O3 absorbing behavior is expected to show strong H1a plus reduced spread. H1c supports a divergent or unsupported label. The classifier is not a replacement for the five primary endpoints in §4.0 and §4.13.

**Rationale or limitation.** The classifier reduces many correlated diagnostics to a small set of pre-registered signal counts. Its strength is transparency: each verdict can be traced to thresholded signals. Its limitation is that it remains a summary of diagnostics rather than a direct endpoint. A regime can receive a plausible classifier label while failing a stricter decision-grade endpoint such as group-aware predictability or persistent escape.

#### Metric, baseline, and pass-rule mapping

| Metric family                              | Null/baseline                                                                   | Claim use                                   | Pass rule                                                                      |
|--------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------|--------------------------------------------------------------------------------|
| Recurrence/dwell                           | time-shuffled,<br>no-feedback,<br>independent<br>regeneration                   | attractor score                             | at least baseline plus 2<br>standard deviations and<br>Cohen’s $d \geq 0.5$    |
| Basin score/entry and exit-return          | time-shuffled and<br>recursive baselines by<br>observable                       | attractor score and<br>regime label support | consistent late-window<br>persistence across<br>canonical observables          |
| Periodicity                                | time-shuffled trajectory<br>order and<br>non-oscillatory regimes                | O2 recurrence or<br>oscillation support     | positive period-2 score<br>with best-period<br>evidence above<br>threshold     |
| Dispersion and finite-time ensemble spread | matched-run ensemble<br>spread and no-feedback<br>baselines where<br>applicable | contraction component<br>of attractor score | late spread contraction<br>or collapse relative to<br>baseline                 |
| Switching                                  | paired control-control<br>floor                                                 | perturbation signature                      | raw, net, and persistent<br>reported separately                                |
| Predictability                             | GroupKFold by family                                                            | basin predictability                        | $\text{acc\_group}(k = 10) \geq 0.70$                                          |
| Behavioral ED50                            | family-cluster bootstrap<br>and model-based dose fit                            | token barrier endpoint                      | finite estimate with 95%<br>interval contained within<br>the probed dose range |
| Persistent escape                          | injection-time cluster<br>jump plus terminal<br>retention                       | persistent basin-escape<br>endpoint         | persistent escape rate at<br>least 0.50 at the claimed<br>barrier dose         |

#### 4.7 Baselines

We use three baselines to separate endogenous recursive structure from marginal embedding geometry or seed-conditioned sampling.

| baseline                              | implementation role                                                              | interpretation                                                                           |
|---------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>time_shuffled</code>            | reshuffles step labels within each trajectory<br>and recomputes dynamics metrics | tests whether a statistic depends on temporal<br>order rather than the point cloud alone |
| <code>no_feedback</code>              | samples each step from the seed only,<br>ignoring accumulated context            | removes recurrence while preserving seed<br>conditioning                                 |
| <code>independent_regeneration</code> | regenerates each step from the system<br>prompt and seed with no carryover       | removes history dependence completely                                                    |

A regime is treated as endogenous only if its diagnostic statistic differs from all applicable baselines beyond the statistical gate. Effect size relative to each baseline is computed as Cohen’s  $d$ , namely the recursive mean minus the baseline mean divided by pooled standard deviation. Baseline applicability differs by experiment: no-feedback and independent-regeneration baselines are operator-regime baselines, while time shuffling applies broadly to trajectory metrics.

#### 4.8 Statistical procedures

Confidence intervals for trajectory-level quantities are computed by 1000-iteration bootstrap. Between-condition differences, including perturbation-condition comparisons, are tested by permutation tests on the relevant mean difference. Switching-rate proportions in small-denominator dose-response cells use Wilson-style confidence intervals where ordinary bootstrap intervals would be unstable.

Effect sizes for recursive-vs-baseline comparisons use Cohen’s  $d$ . A signal counts only if it passes both parts of the significance gate: the diagnostic statistic must be at least 2 standard deviations above the baseline mean, and Cohen’s  $d$  must be at least 0.5. This prevents very large samples from turning negligible effects into claim-bearing signals.

Basin-predictability cross-validation uses the adaptive stratified protocol and the group-aware protocol described in §4.5.3. Singleton classes are dropped before cross-validation and recorded. Cells that cannot be split into at least two non-singleton classes are recorded as missing rather than forced into an unstable estimate.

All endpoint-level analyses use the pre-specified observable and projection choices described above. Sensitivity to other observables is reported as diagnostic support, not as a replacement for the primary endpoint unless the endpoint itself specifies cross-observable agreement.

#### 4.9 Static visualization battery

Every experiment generates a standardized set of static plots. These figures are used to inspect trajectories, detect failures, and communicate dynamics, but they are not primary endpoints unless explicitly tied to the criteria in §4.13.

The visualization battery includes joint t-SNE plots colored by regime, family, and step; per-family trajectory grids in shared coordinates; ensemble-spread timelines; PCA-2 quiver fields; sampled t-SNE trajectories with temporal ordering; streamlines with density; speed-colored streamlines; divergence fields; even/odd step parity plots for oscillatory regimes; family-by-initial-condition final-cluster maps; and distributional plots for basin entry, basin score, cluster occupancy, and dwell.

Plots are rendered at 200 DPI to PNG. Each experiment’s `reports/plots/` directory contains approximately 50 to 150 figures depending on the number of observables and optional visualization modules. The plotting code is deterministic given the stored embeddings, projections, and metric CSVs.

#### 4.10 Flow-field computation

Flow-field visualizations share a bin-and-aggregate kernel that converts a trajectory ensemble into a spatially resolved displacement field on a two-dimensional projection. For each trajectory group, points are sorted by step. Each adjacent pair contributes a start location and a displacement vector. The collection of starts and displacements is binned over the projection plane.

For each grid bin, the average displacement vector is computed from all transitions whose start point falls inside the bin. Empty bins are left undefined, so streamline integration does not pass through unsupported regions. This produces an empirical vector field  $v(x) = (U(x), V(x))$  that represents the average one-step projected motion observed in that region.

Density fields use a higher-resolution two-dimensional histogram followed by Gaussian smoothing. The smoothed density  $\hat{\rho}(x)$  is used as a background heatmap and, in perturbation analyses, as the basis for the effective potential

$$V(x) = -\log(\hat{\rho}(x) + \epsilon).$$

Streamlines are integral curves of the empirical vector field. Divergence is computed as

$$\nabla \cdot v(x) = \frac{\partial U}{\partial x} + \frac{\partial V}{\partial y}.$$

Negative divergence indicates sink-like behavior, while positive divergence indicates source-like behavior. For recursive LLM loops, we expect weakly negative average divergence in contractive regimes, strong local minima at absorbing sinks, and saddle-like structure in oscillatory regimes. Implementation details and grid parameters are provided in Supplementary §11.8.

#### 4.11 Perturbation visualization toolkit

Perturbation experiments additionally generate an empirical-potential and barrier-visualization toolkit. PCA-2 coordinates are converted into a smoothed density estimate  $\hat{\rho}(x)$ , then into an effective potential  $V(x) = -\log \hat{\rho}(x)$ . Basin centers are detected as local minima on the potential grid. Geodesic barriers between basin pairs are computed by shortest-path search on the grid, with the barrier height defined by the maximum potential encountered along the path.

The toolkit also produces streamlines over potential contours, geodesic overlays, condition-wise flow-field panels, and three-dimensional density-shell animations. The 3D animations use PCA-3 coordinates, iso-density shells, sampled trajectory walks, and visual markers for perturbation events. These visualizations are interpretive aids for the perturbation endpoint analyses. Full grid parameters, smoothing constants, local-minimum detection, Dijkstra settings, marching-cubes details, transparency schedules, and parallel rendering settings are provided in Supplementary §11.8.

#### 4.12 End-to-end pipeline schematic

The complete pipeline is organized into seven deterministic phases:

| phase                            | operation                                                                                                                                             | persistent output                                                                    |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Phase 1 Generation               | run recursive trajectories with <code>gpt-4o-mini</code> , applying <code>append</code> , <code>replace</code> , or <code>dialog</code> updates       | <code>raw/steps.jsonl</code>                                                         |
| Phase 2 Observables              | derive text views such as <code>output</code> , <code>rolling_k3</code> , <code>context_tail</code> , role-specific turns, and <code>turn_pair</code> | observable text streams                                                              |
| Phase 3 Embeddings               | embed each observable with <code>text-embedding-3-small</code> and L2-normalize rows                                                                  | <code>embeddings/&lt;obs&gt;/embeddings.npy</code> and <code>metadata.parquet</code> |
| Phase 4 Joint projections        | fit PCA-2, PCA-10, PCA-50, and t-SNE jointly per observable                                                                                           | projected-coordinate files                                                           |
| Phase 5 Metrics and endpoints    | compute clusters, recurrence, dwell, basin metrics, ensemble-spread diagnostics, predictability, and perturbation endpoints                           | metric CSVs                                                                          |
| Phase 6 Baselines and statistics | compute baselines, bootstrap intervals, permutation tests, effect sizes, and pass-rule summaries                                                      | bootstrap and effect-size summaries                                                  |
| Phase 7 Reports                  | render plots, perturbation visualizations, endpoint tables, and narrative reports                                                                     | per-experiment <code>reports/</code> directories                                     |

Each phase writes a deterministic intermediate, allowing downstream analyses to be rerun without regenerating trajectories. The full TikZ source, shape annotations, persistence-boundary table, and rerun semantics live in Supplementary §11.9.

The source of truth is `steps.jsonl`. Embeddings, projections, metrics, figures, and reports are regenerable from that file plus the code and configuration. In routine development, metric and plotting changes can be rerun without additional model-generation calls.

#### 4.13 Hardware and software

All experiments run locally on a single workstation with API calls to OpenAI for generation and embeddings; no GPU is required. The host used to build the released artefacts is an HP ProLiant DL360 Gen9 with two Intel Xeon E5-2687W v3 processors (2 x 10 physical cores at 3.10 GHz base, 40 logical threads total) and 256 GB of RAM, running Windows 10 Pro 64-bit. Embedding ingestion, dimensionality reduction, clustering, density-and-geodesic-barrier computation, and animation rendering are all CPU-only.

The Python environment is Python 3.14 with numpy 2.3, scipy 1.16, scikit-learn 1.8, scikit-image 0.26, pandas 2.3, matplotlib 3.10, and imageio-ffmpeg 0.6 (resolved versions used to produce the released artefacts; the code itself targets

Python 3.10+). The full dependency lock is in `requirements.txt`. Animations are stitched via `imageio-ffmpeg` using the `libx264` codec. The `pytest` suite of 99 tests is green end-to-end and runs in roughly 13 seconds in this environment.

Parallel rendering of trajectory animations and basin diagnostics uses `concurrent.futures.ProcessPoolExecutor` with up to 40 workers, matching the number of logical threads on the host. The framework makes no other hardware assumptions; the analysis pipeline runs on any Linux, macOS, or Windows machine with the dependency stack above and enough RAM to hold a single experiment’s trajectories and PCA-10 embeddings in memory (a few GB per experiment).

#### 4.14 Decision-grade endpoints

The five primary endpoints introduced in §4.0 are now defined operationally. The metric battery in §4.5 is intentionally broad: it is used to diagnose, visualize, and stress-test recursive dynamics from several angles. The paper’s headline claims, however, should not depend on dozens of partially redundant quantities. For decision purposes, we treat the following five endpoints as load-bearing. Each endpoint has a fixed numerical pass rule; results that do not clear the rule are reported as diagnostic, exploratory, or in-flight rather than as supported regime claims.

On the current data, after the dense-dose rerun and the endpoint-decomposition analysis:

- **Operational attractor score C1-C4:** O1, O2, O3, and D1 pass the omnibus attractor criterion; D2 does not.
- **Leakage-free basin predictability:** only O1 passes the stricter  $\text{acc\_group}(k = 10) \geq 0.70$  and  $\Delta < 0.10$  rule. O2, O3, and D1 fail under group-aware cross-validation.
- **Perturbation switching signature:** O1 selective sensitivity passes, with  $S_{\text{adv}}(200) = 0.620$  in the dense rerun and a ratio to neutral or lorem switching of approximately 2.8. Replace-mode O2/O3 capitulation passes by point estimate but is partly tautological because replace-mode intervention overwrites state.
- **Behavioral ED50\_raw token barrier:** passes at approximately 40 tokens, with estimates of 36 from the four-parameter logistic fit, 41 from the generalized linear mixed model, and 52 as the bootstrap median. The 95% interval  $[8.5, 242]$  is wide because of the 5-family-cluster heavy tail.
- **ED50\_net above natural floor:** does not pass. The net effect saturates at +32 percentage points at dose 400, below the +50 percentage-point threshold.
- **Persistent basin-escape rate:** does not pass. At dose 400, only 16% of dense-data trajectories are kicked and persisted, well below the 50% threshold. Thus switching is not claimed as clean basin escape, and the strict ED50\_persist is undefined in the tested range.

In the present experiments, the equivalence rule  $C(O(X_T))$  is a K-means cluster of an embedding-space observable. In tool-using coding agents, the same endpoint structure can be instantiated with engineering observables: final patch family, files touched, the failing or passing test set, the selected plan category, the tool-call sequence, a security-policy violation, or an embedding of the full trajectory trace. Algorithm 1 requires only a consistent, pre-specified equivalence rule and paired controls; it does not require that "cluster" literally mean an embedding cluster.

| endpoint                                                 | definition                                                                                                                                                                       | measured at                                                                                                                                                     | threshold for "regime claim is supported"                                                                                                                                                                                                                                                                                                                        | defined in                                |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <b>Operational attractor score C1-C4</b>                 | Count of the four attractor criteria passed: late-window basin persistence, recurrence or dwell above null, embedder robustness, and contraction, re-entry, or collapse.         | Publication-scale O1, O2, O3, and D1 on canonical observables. D2 exploratory status is checked separately.                                                     | <b>Strong attractor:</b> 4/4 criteria pass. <b>Attractor-like:</b> at least 3/4 pass. <b>Not attractor:</b> fewer than 3/4 pass. Missing publication-scale measurements count as fail unless structurally inapplicable.                                                                                                                                          | §4.5.1, §4.5.2, §4.6, Supplementary §11.2 |
| <b>Leakage-free basin predictability acc_group(k=10)</b> | GroupKFold-by-prompt-family accuracy of predicting the late-window K-means basin from the PCA-10 state at step $k = 10$ .                                                        | Publication-scale O1, O2, O3, and D1, context_tail, K-means $k = 12$ , data/aggregated_groupaware_basin_pred.csv.                                               | To claim <b>cross-family basin predictability:</b> $\text{acc\_group}(k = 10) \geq 0.70$ . To claim <b>leakage-free:</b> $\Delta = \text{acc\_stratified} - \text{acc\_group} < 0.10$ .                                                                                                                                                                          | §4.5.3, Supplementary §11.3               |
| <b>Perturbation switching signature</b>                  | Final-step switching rate: fraction of perturbed trajectories whose final K-means cluster differs from the paired control trajectory.                                            | O1 dose-response at matched 200-token dose; O2, O3, and D1 perturbation pilots; data/aggregated_switching_rate and data/aggregated_perturbation_dose_response/. | <b>O1 selective sensitivity:</b> $S_{\text{adv}}(200) \geq 0.50$ and $S_{\text{adv}}(200) / \max(S_{\text{neutral}}(200), S_{\text{lorem}}(200)) \geq 2.0$ , with maximum out-of-distribution switching at most 0.30. <b>Replacement mode capitulation:</b> minimum non-control switching across O2/O3 neutral, lorem, and adversarial conditions at least 0.85. | §4.4, §4.5.4, Supplementary §11.5         |
| <b>Behavioral ED50 token barrier</b>                     | The perturbation dose $\tau$ at which a four-parameter logistic fit to the O1 adversarial dose-response reaches 50% switching, with prompt-family-cluster bootstrap uncertainty. | O1 adversarial dose sweep. Sparse run: $\tau \in \{20, 80, 200, 400\}$ , $n = 50$ per cell. Dense rerun: $n = 200$ per cell across 8 doses.                     | To claim a <b>localized token barrier:</b> ED50 point estimate finite and the 95% family-cluster bootstrap interval lies wholly inside the probed interval [20, 400] tokens. If the point estimate is inside but the interval                                                                                                                                    | §4.5.4, Supplementary §11.1               |

## 5 Results

Adversarial append-mode perturbations produce a clear raw-switching dose response in the O1 continuation loop, with  $ED50_{\text{raw}} \approx 40$  tokens, but they do not establish durable basin escape in the tested range. The dense rerun shows a raw plateau near 67%, a natural stochastic-divergence floor near 35%, a maximum net adversarial effect of +32 percentage points at 400 tokens, and a maximum persistent-escape rate of 16% under the canonical K-means  $k = 12$  definition. Replace-mode loops initially appear almost fully perturbation-transparent, but the overwrite-versus-insert probe shows that most of this effect comes from the memory policy discarding prior state, not from a low injected-token barrier. Phase-0 and Phase-1 pilots validated the measurement pipeline and identified candidate regimes; full pilot history is in §11.7, the aggregation and per-experiment catalog is in §11.9, and row-level endpoint audit tables are in Extended Data Tables 1 and 2 (§11.1, §11.2).

### Phase A, headline endpoint

#### 5.1 Adversarial append perturbations produce raw switching but not persistent escape

The central perturbation endpoint is O1 append-mode continuation under in-distribution adversarial text. In the sparse pilot, O1 adversarial perturbations showed a graded response, while O1 neutral perturbations remained flat near the out-of-distribution drift floor and D1 neutral perturbations saturated even at very small doses. The dense rerun then localized the O1 adversarial raw-switching curve at  $n = 200$  trajectories per dose and separated three quantities that must not be conflated:

1. **Raw switching:** final K-means cluster differs from the paired control trajectory.
2. **Net switching:** raw switching minus the control-control stochastic-divergence floor.
3. **Persistent escape:** the trajectory visibly changes cluster at injection and remains in the post-injection cluster to the terminal step.

The dense rerun was pre-registered before execution:  $n = 200$  per cell, equal to 5 families  $\times$  10 ICs  $\times$  4 runs, with 8 adversarial dose conditions plus one control condition, for 1,800 trajectories total. The configuration was `configs/perturbation/01_ed50_dense.yaml`; the analysis script was `scripts/fit_ed50_hierarchical.py`.

#### Dense O1 adversarial dose response, separating raw, net, and persistent endpoints

| dose (tokens) | raw switch rate | Wilson 95% CI  | net over natural floor | persistent escape, K-means $k = 12$ |
|---------------|-----------------|----------------|------------------------|-------------------------------------|
| 20            | 0.415           | [0.349, 0.484] | +0.068                 | 0.035                               |
| 50            | 0.510           | [0.441, 0.578] | +0.163                 | 0.070                               |
| 80            | 0.575           | [0.506, 0.641] | +0.228                 | 0.035                               |
| 120           | 0.630           | [0.561, 0.694] | +0.283                 | 0.090                               |
| 160           | 0.605           | [0.536, 0.670] | +0.258                 | 0.115                               |
| 200           | 0.620           | [0.551, 0.684] | +0.273                 | 0.130                               |
| 300           | 0.655           | [0.587, 0.717] | +0.308                 | 0.140                               |
| 400           | 0.670           | [0.602, 0.731] | +0.323                 | 0.160                               |

The control-control natural floor is 34.7% [31.0%, 38.6%] across  $n = 600$  ordered control-control pairs. Thus two trajectories with the same family and IC seed but different generation RNG end in different K-means clusters 35% of the time without any perturbation. The raw 50% crossing occurs between 20 and 50 tokens, but much of that apparent switching is baseline stochastic divergence. Under the stricter net endpoint, the curve does not reach +50 percentage points within the tested range.

Three independent ED50 estimates agree on the raw-switching scale:

| method                          | ED50 (tokens) | uncertainty                      |
|---------------------------------|---------------|----------------------------------|
| 4PL pooled fit                  | 36            | point estimate                   |
| Mixed-effects logistic GLMM     | 41            | point estimate, log10-dose slope |
| Family-cluster bootstrap median | 52            | 95% CI [8.5, 242]                |

The point estimates cluster in the 36-52 token range, substantially below the earlier sparse-grid estimate near 150 tokens. The family-cluster bootstrap interval remains wide because only five prompt families are available for resampling.



Two structural findings matter more than the exact ED50 point estimate. First, the raw curve plateaus near 67%, not near 100%. The 4-parameter logistic upper asymptote is  $a = 0.69$ , implying a substantial non-switching subpopulation under the present protocol. Second, the persistent-escape endpoint is much smaller than raw switching. At dose 400, raw switching is 67%, but persistent escape is 16% under K-means  $k = 12$ . Most raw switching is therefore not clean barrier crossing. It is final-step divergence from the paired control, often without a durable at-injection jump into a new basin.

The persistence decomposition on the dense rerun confirms this interpretation. At dose 400, 69 of 200 trajectories visibly changed cluster at injection. Of those, 32 persisted in the post-injection cluster, 13 drifted back to the pre-injection cluster, and 24 drifted elsewhere. Even among trajectories that visibly jump at injection, roughly half do not remain in the post-injection basin.

Because persistence is cluster-defined, we also recomputed it under three granularities: K-means  $k = 12$ , K-means  $k = 4$ , and HDBSCAN. The formal persistent-escape ED50, the dose at which persistent escape reaches 50%, is not reached under any of the three definitions.

| dose | persistent escape, $k = 12$ | persistent escape, $k = 4$ | persistent escape, HDBSCAN | kicked at injection, HDBSCAN |
|------|-----------------------------|----------------------------|----------------------------|------------------------------|
| 20   | 3.5%                        | 1.5%                       | 7.0%                       | 12.0%                        |
| 50   | 7.0%                        | 3.0%                       | 16.5%                      | 28.5%                        |
| 80   | 3.5%                        | 5.0%                       | 28.5%                      | 48.0%                        |
| 120  | 9.0%                        | 4.5%                       | 35.5%                      | 58.0%                        |
| 160  | 11.5%                       | 9.5%                       | 41.0%                      | 64.5%                        |
| 200  | 13.0%                       | 13.5%                      | 40.5%                      | 60.0%                        |
| 300  | 14.0%                       | 8.5%                       | 40.0%                      | 66.5%                        |
| 400  | 16.0%                       | 10.0%                      | 39.5%                      | 68.5%                        |

HDBSCAN is the most permissive definition and gives the largest persistent-escape values, but even there the maximum is 39.5%, below the 50% threshold. The conclusion is therefore robust to cluster granularity: O1 adversarial append perturbations create a finite raw-switching dose response, but persistent basin escape is not demonstrated up to 400 injected tokens.

The sparse dose-response pilot remains useful for the qualitative contrast among perturbation contents. In that pilot, D1 neutral switching was already high at 5 tokens and stayed high across the grid, O1 neutral switching stayed near 22-26% from 20 to 400 tokens, and O1 adversarial switching rose from 26% to roughly 50%. The dense rerun shows that the O1 adversarial pattern is real, but it also shows why the endpoint must be named carefully. The defensible headline is **raw ED50  $\approx$  40 tokens**, not a persistent-escape barrier height.

## 5.2 Replace-mode fragility is primarily a memory-policy effect

The original perturbation pilots made O2 and O3 appear nearly maximally fragile. Under the original replace-mode protocol, non-control perturbations overwrite the model output at the injection step, so the next state is conditioned on the injected text rather than on the loop’s prior generated state. This produces 94-100% final-cluster disagreement with the paired control trajectory:

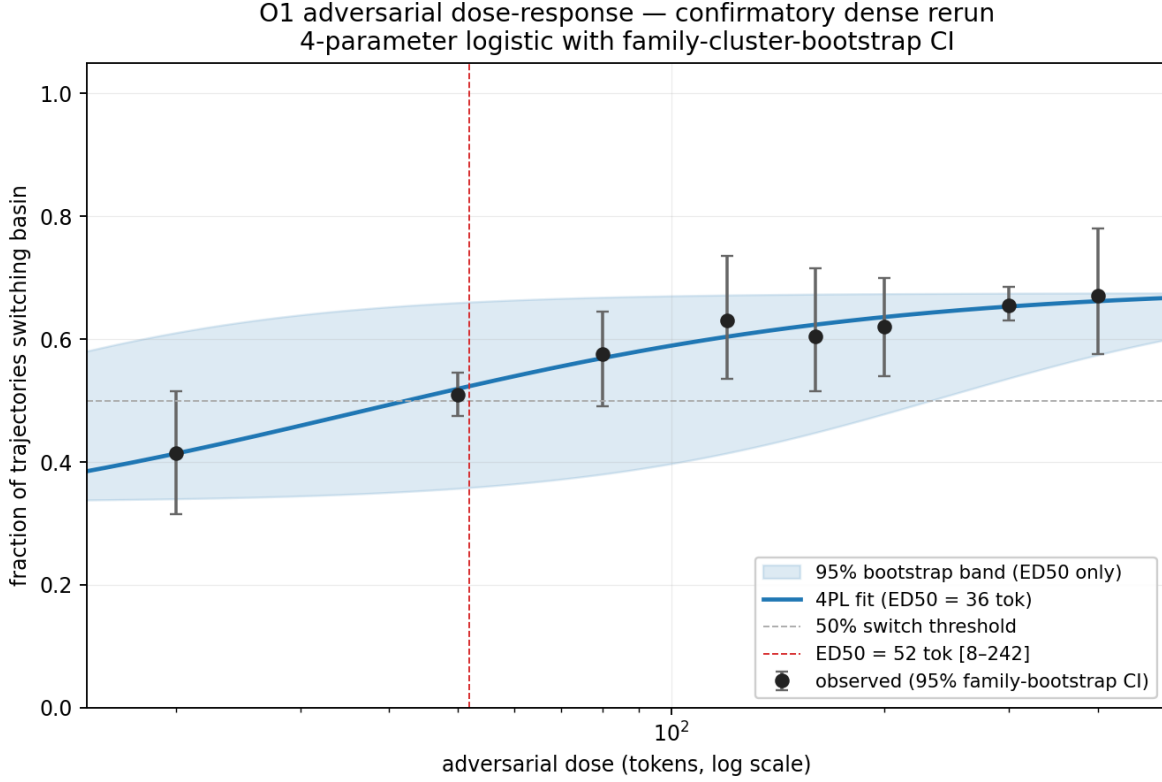
| regime                       | neutral       | lorem         | adversarial |
|------------------------------|---------------|---------------|-------------|
| O2, paraphrase replace       | 100% [93-100] | 100% [93-100] | 94% [84-98] |
| O3, summarize-negate replace | 100% [93-100] | 100% [93-100] | 96% [86-99] |

Read alone, these numbers suggest that replace-mode regimes have almost zero injected-token barriers. The overwrite-versus-insert probe shows that this is mostly a memory-policy effect. We re-ran O2 and O3 with the same adversarial doses under two intervention modes:

- **Overwrite:** the original protocol. The injection replaces step 15’s output entirely.
- **Insert:** the injected text is prepended to the context for step 15, but the model’s own generated output is preserved as the state. The injected text does not remain as the state by construction.

The O2 paraphrase-replace results were:

The O3 summarize-negate-replace results were:



**Figure 1:** Figure K. **Dense-dose ED50 fit.** O1 adversarial dose-response from the confirmatory rerun, with 8 doses  $\times n = 200$  per cell and one control cell. Black points are observed switching rates with family-cluster-bootstrap 95% CIs; the blue curve is a 4-parameter logistic fit ( $a=0.69$ ,  $d=0.28$ ,  $b=1.16$ ,  $ED50=36$  tok); the dashed red line marks the bootstrap-median  $ED50 = 52$  tokens [CI 8.5, 242]. Source: data/exp\_perturb\_o1\_ed50\_dense/reports/perturbation/ed50\_curve.png.

| condition                      | switch rate | 95% Wilson CI |
|--------------------------------|-------------|---------------|
| control                        | 0.00        | [0.00, 0.07]  |
| adversarial_dose80, overwrite  | 0.92        | [0.81, 0.97]  |
| adversarial_insert_dose80      | 0.32        | [0.21, 0.46]  |
| adversarial_dose200, overwrite | 0.98        | [0.90, 1.00]  |
| adversarial_insert_dose200     | 0.18        | [0.10, 0.31]  |

| condition                      | switch rate | 95% Wilson CI |
|--------------------------------|-------------|---------------|
| control                        | 0.00        | [0.00, 0.07]  |
| adversarial_dose80, overwrite  | 0.90        | [0.79, 0.96]  |
| adversarial_insert_dose80      | 0.18        | [0.10, 0.31]  |
| adversarial_dose200, overwrite | 0.92        | [0.81, 0.97]  |
| adversarial_insert_dose200     | 0.12        | [0.06, 0.24]  |

The overwrite-minus-insert gap is 60-80 percentage points across both regimes and both doses:

| regime | dose | overwrite | insert | overwrite minus insert |
|--------|------|-----------|--------|------------------------|
| O2     | 80   | 0.92      | 0.32   | +0.60                  |
| O2     | 200  | 0.98      | 0.18   | +0.80                  |
| O3     | 80   | 0.90      | 0.18   | +0.72                  |
| O3     | 200  | 0.92      | 0.12   | +0.80                  |

Thus most apparent replace-mode perturbation transparency comes from the update rule discarding prior state. Once the perturbation no longer overwrites the loop state, switching falls to 12-32%, below or near the natural stochastic-

divergence floor measured for O1. The original O2/O3 result remains an important systems finding, but it should be described as **overwrite-induced state replacement**, not as a discovered low behavioral barrier comparable to the O1 dose-response measurement.

This has a direct engineering analogue. Any architecture that periodically replaces accumulated context with a generated summary, scratchpad, task state, or memory record can fail by promoting untrusted text into the replacement state. In that case, the system has not merely been persuaded by injected text; its previous state has been removed by the memory policy.

## Phase B, regime establishment with leakage-aware analysis

### 5.3 Publication-scale runs preserve regime ordering

REPORT5 ran the four diagnostic regimes at publication scale. Operator regimes O1, O2, and O3 used 15 prompt families  $\times$  30 ICs  $\times$  3 runs, for 1,350 trajectories per regime. Dialog regime D1 used 5 dialog-suitable families  $\times$  30 ICs  $\times$  3 runs, for 450 trajectories. All four were 40 steps long.

Basin predictability is measured by 5-fold multinomial logistic regression on PCA-10, predicting the trajectory’s late-window K-means cluster at  $k = 12$  from the embedding at step  $k$ . The late-window cluster is the majority cluster over  $t \geq \lceil 0.7T \rceil$ . For  $T = 40$ , this is a 12-step late window. For D1 with role-restricted observables, the latest predictor step is 26, the last agent turn before the late window opens at step 28.

At first exposure we report both the original stratified cross-validation accuracy and the leakage-aware GroupKFold accuracy at  $k = 10$ , where entire prompt families are held out across folds.

| experiment                           | regime                               | acc(k=5) | acc(k=10),<br>strati-<br>fied | acc(k=10),<br>group-<br>aware | leakage<br>delta | acc(k=20) | acc(final) |
|--------------------------------------|--------------------------------------|----------|-------------------------------|-------------------------------|------------------|-----------|------------|
| exp_pub_01_continue                  | O1,<br>contractive                   | 0.77     | 0.803                         | 0.732                         | +0.071           | 0.81      | 0.85       |
| exp_pub_02_paraphrase_replace        | O2,<br>oscillatory<br>replace        | 0.90     | 0.896                         | 0.596                         | +0.301           | 0.91      | 0.91       |
| exp_pub_03_summarize_negate_replace  | O3, absorbing<br>replace             | 0.92     | 0.912                         | 0.629                         | +0.283           | 0.92      | 0.93       |
| exp_pub_D1_dialog_curious_helpful_v2 | D1,<br>dialogue-state<br>multi-basin | n/a      | 0.604                         | 0.336                         | +0.269           | 0.69      | 0.77       |

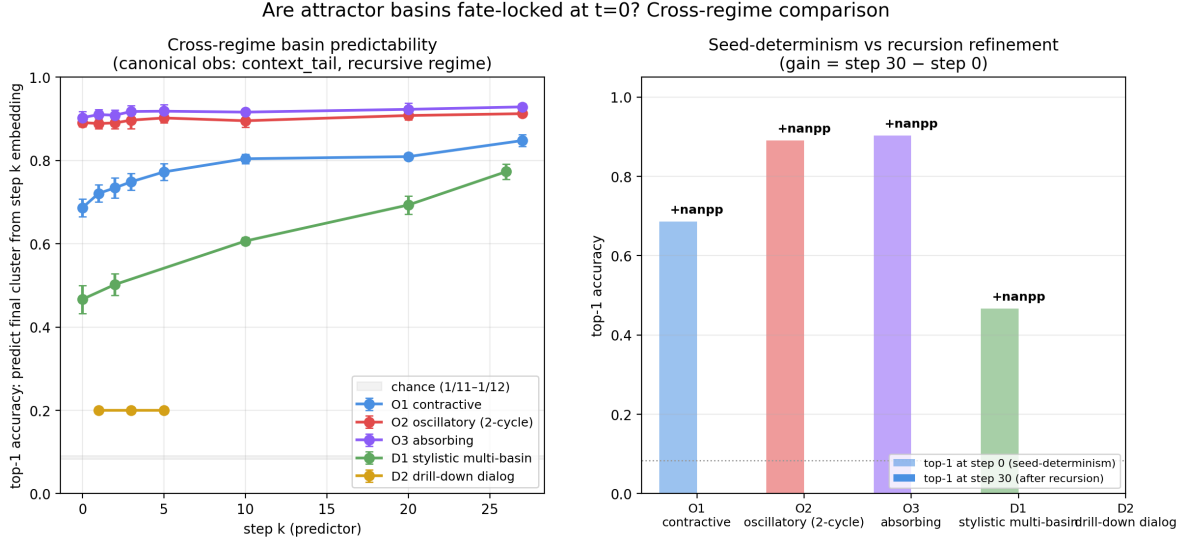
The stratified values reproduce the original regime ordering: O2 and O3 lock in very early, O1 is slower but still strongly predictable, and D1 remains the least predictable at early steps. The group-aware analysis changes the interpretation. O1 loses only 7 percentage points when prompt families are held out, while O2, O3, and D1 lose 27-30 percentage points. Thus O1’s basin predictability is the most cross-family robust result. For O2, O3, and D1, a substantial part of the original predictability is a family or style fingerprint.

The qualitative regime separation survives. O3 and O2 remain high-recurrence replace-mode regimes, O1 remains a cross-family contractive append regime, and D1 remains a slower, more family-sensitive dialog regime. The main correction is evidential: stratified accuracies should be read as upper bounds, and the leakage-aware columns are the relevant values for cross-family generalization.

### 5.4 Perturbation pilots separate append from replace

The cross-regime perturbation pilots used 5 families  $\times$  5 ICs  $\times$  2 runs  $\times$  30 steps, for  $n = 50$  trajectories per condition, except D2 where  $n = 25$ . Switching is final-step K-means cluster disagreement with the paired control trajectory.

This table is now read through §5.2. The O2/O3 values are real measurements of the original overwrite protocol, but they are not fair injected-token barrier estimates. They mostly measure replacement of prior state. O1 shows the cleanest



**Figure 2: Basin-predictability across regimes.** Top-1 accuracy for predicting each trajectory’s late-window K-means cluster from its embedding at step  $k$ , using publication-scale runs where available. Source: data/aggregated/basin\_predictability\_cross/cross\_basin\_predictability.png.

| regime                       | control   | neutral       | lorem         | adversarial |
|------------------------------|-----------|---------------|---------------|-------------|
| O1, contractive append       | 0% [0-7]  | 24% [14-37]   | 18% [10-31]   | 54% [40-67] |
| O2, paraphrase replace       | 0% [0-7]  | 100% [93-100] | 100% [93-100] | 94% [84-98] |
| O3, summarize-negate replace | 0% [0-7]  | 100% [93-100] | 100% [93-100] | 96% [86-99] |
| D1, dialogue-state dialog    | 0% [0-7]  | 76% [62-86]   | 54% [40-67]   | 60% [46-73] |
| D2, drill-down dialog        | 0% [0-13] | n/a           | n/a           | 64% [44-80] |

content-dependent append result: in-distribution adversarial text switches more often than neutral or lorem text. D1 is broadly susceptible across perturbation types, consistent with a dialog-state basin that is easier to redirect than O1 but less mechanically overwritten than O2/O3.

### 5.5 Drill-down dialog adds content gravity

D2 is an Explorer-Expert drill-down dialog. Each user turn asks for a deeper, more specific explanation of one concept from the previous expert turn. The exploratory run used 5 topic families  $\times$  5 seed topics, for 25 trajectories at 50 steps each. An adversarial perturbation was injected at step 25, drawing expert text from a different topic family, followed by 25 post-injection steps.

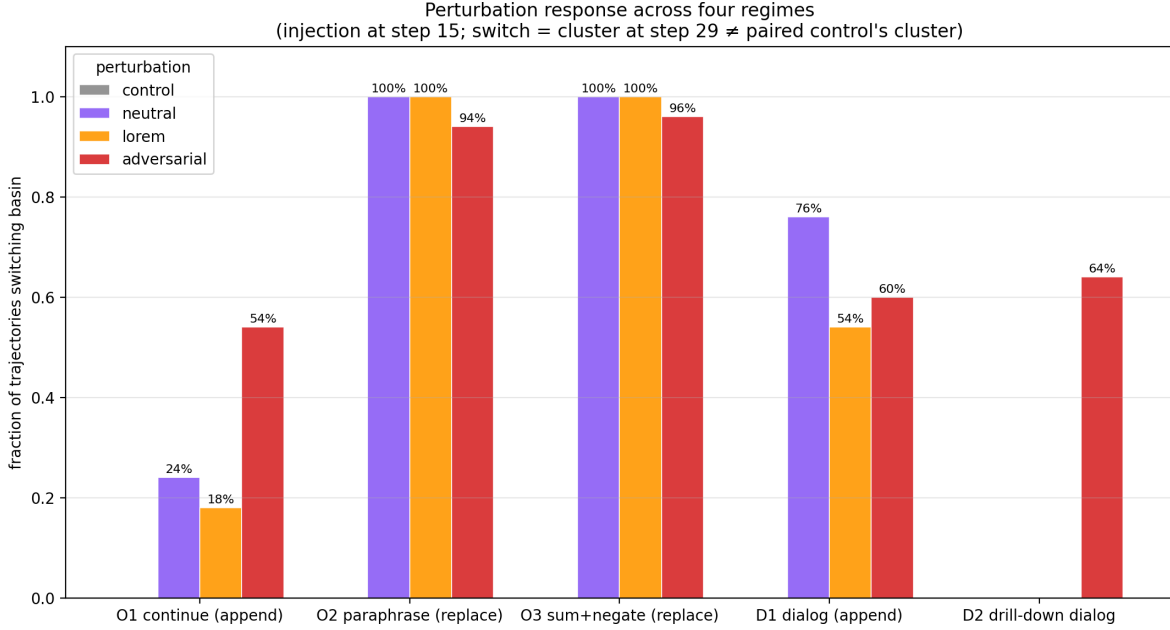
The D2 adversarial switch rate was **64%** [44%, 80%]. This is not a publication-scale estimate, and it is not perfectly matched to the D1 timing cells because dose, content, and post-injection horizon differ. The qualitative signal is nevertheless useful: 36% of D2 trajectories did not switch under a late, in-distribution adversarial expert-text injection. The drill-down format imposes content gravity through progressive specialization into a topic tree, which free dialog lacks.

D2 is therefore retained as an exploratory fifth regime. It is distinct from D1 because the dialog state is not only conversational style or recent-context capture; it is also anchored by an accumulating content path.

### 5.6 Injection timing reveals basin hardening

We injected the same perturbation at three times in a 30-step trajectory: D1 neutral at dose 80 and O1 adversarial at dose 200, with  $n = 50$  per cell.

D1 shows partial basin hardening. By step 25, the trajectory has more often committed to a dialog-state basin, and switching falls from 78% at step 15 to 52% at step 25. O1 is approximately flat across injection time. The contractive append operator integrates in-domain adversarial text regardless of when it arrives, while D1 becomes harder to redirect late in the trajectory.



**Figure 3: Perturbation switching by regime.** Grouped bar chart of final-step switching rate, defined as final K-means cluster disagreement with the paired control trajectory. Source: `data/aggregated/perturbation_cross_regime/cross_switching_rates.png`.

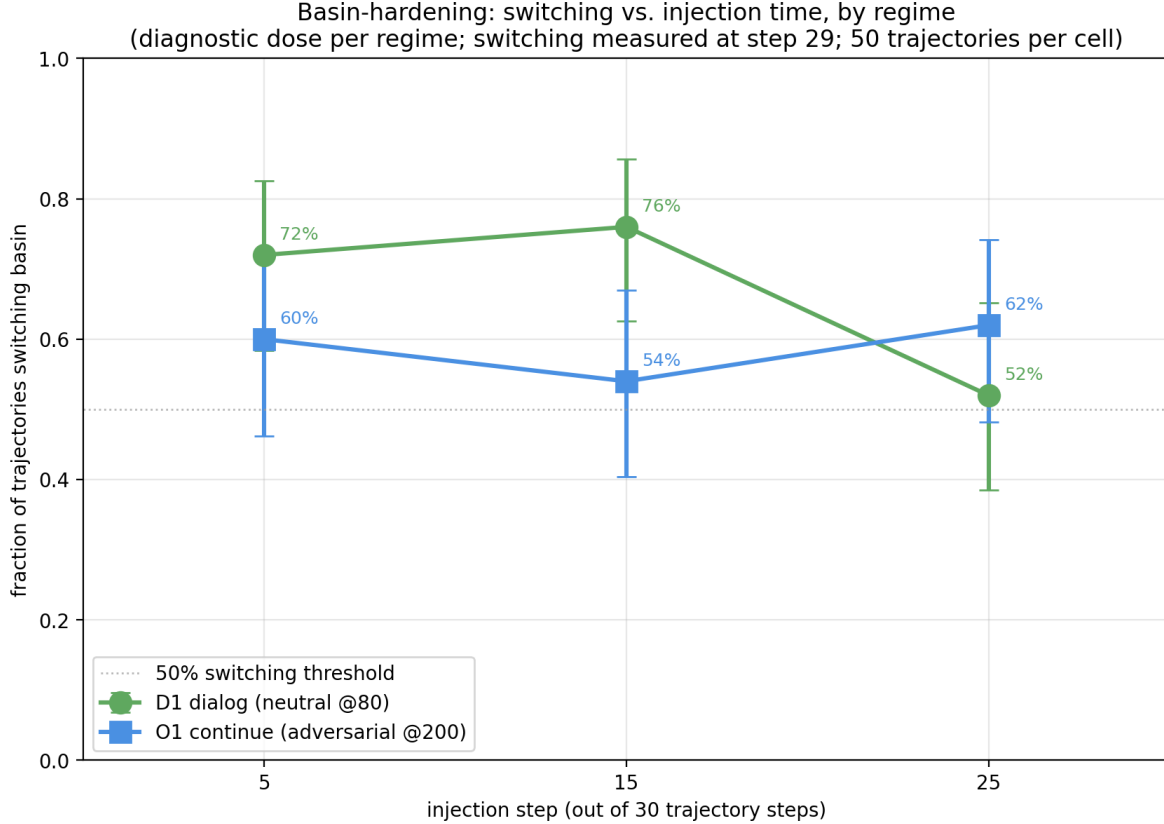
| inject step | D1, neutral at 80 | O1, adversarial at 200 |
|-------------|-------------------|------------------------|
| 5           | 72% [58-83]       | 60% [46-73]            |
| 15          | 78% [65-87]       | 54% [40-67]            |
| 25          | 52% [38-66]       | 62% [48-74]            |

## Phase C, cluster, granularity, and semantic robustness

### 5.7 Cluster stability and multi-granularity switching

The canonical basin partition is K-means with  $k = 12$  on PCA-10. To test whether this partition is an artefact of K-means at one value of  $k$ , we re-clustered publication-scale late-window clouds using HDBSCAN and spectral clustering, then compared partitions with adjusted Rand index. The result is moderate stability overall, with an important exception for O1.

| regime                      | median ARI vs K-means<br>$k = 12$ | HDBSCAN auto-detected $k$ | interpretation                                                                                                           |
|-----------------------------|-----------------------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------|
| O1 contractive              | 0.53                              | 2                         | HDBSCAN sees O1 as one or two large density basins. K-means $k = 12$ is a fine sub-partition of a contractive attractor. |
| O2 paraphrase replace       | 0.58                              | 16                        | Replace-mode cluster structure is moderately stable.                                                                     |
| O3 summarize-negate replace | 0.60                              | 16                        | Replace-mode cluster structure is moderately stable.                                                                     |
| D1 dialog                   | 0.66                              | 16                        | Highest cross-method stability, but still partly method-dependent.                                                       |



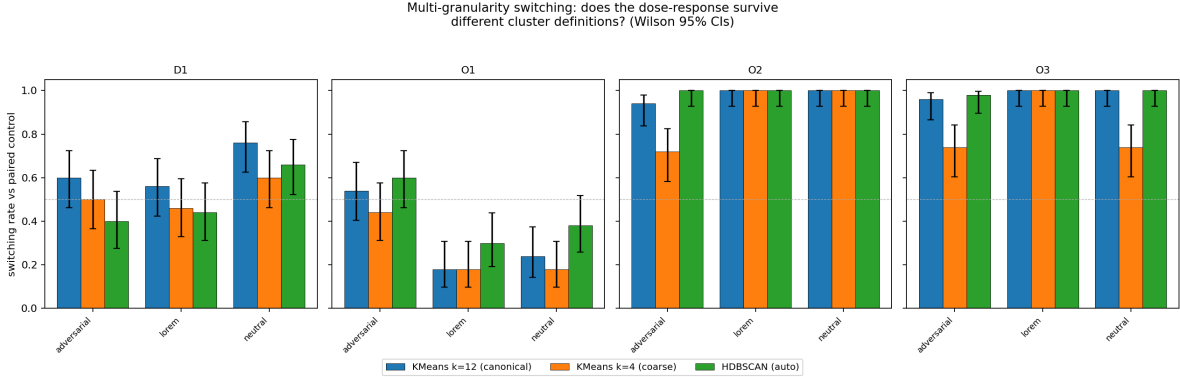
**Figure 4: Switching versus injection time.** Switching rate for injections at steps 5, 15, and 25 of a 30-step trajectory, with  $n = 50$  per cell and 95% Wilson confidence intervals. Source: data/aggregated/perturbation\_basin\_hardening/basin\_hardening.png.

For O1, this strengthens the attractor interpretation but qualifies the switching metric. A K-means  $k = 12$  switch can mean movement between sub-regions of a large contractive basin rather than escape from a HDBSCAN density basin. This is why §5.1 separates raw switching from persistent escape and tests persistence under multiple cluster definitions.

We also recomputed perturbation switching in the four diagnostic perturbation pilots under K-means  $k = 12$ , K-means  $k = 4$ , and HDBSCAN.

| pilot | condition   | $k = 12$ | $k = 4$ | HDBSCAN | summary                         |
|-------|-------------|----------|---------|---------|---------------------------------|
| O1    | adversarial | 0.54     | 0.44    | 0.60    | robustly higher than OOD        |
| O1    | neutral     | 0.24     | 0.18    | 0.38    | low across all                  |
| O1    | lorem       | 0.18     | 0.18    | 0.30    | low across all                  |
| O2    | adversarial | 0.94     | 0.72    | 1.00    | saturated except coarse $k = 4$ |
| O2    | neutral     | 1.00     | 1.00    | 1.00    | saturated                       |
| O2    | lorem       | 1.00     | 1.00    | 1.00    | saturated                       |
| O3    | adversarial | 0.96     | 0.74    | 0.98    | saturated except coarse $k = 4$ |
| O3    | neutral     | 1.00     | 0.74    | 1.00    | high across all                 |
| O3    | lorem       | 1.00     | 1.00    | 1.00    | saturated                       |
| D1    | adversarial | 0.60     | 0.50    | 0.40    | granularity-sensitive           |
| D1    | neutral     | 0.76     | 0.60    | 0.66    | granularity-sensitive           |
| D1    | lorem       | 0.56     | 0.46    | 0.44    | granularity-sensitive           |

The O1 content asymmetry is robust: adversarial switching remains roughly 2-3 times neutral or lorem switching across granularities. O2/O3 overwrite-protocol switching remains high at fine granularities, with some collapse under coarse  $k = 4$ . D1 is the most granularity-sensitive, consistent with its family-leakage and dialog-state dependence.



**Figure 5: Figure H. Switching rates under alternative cluster granularities.** Perturbation switching rates for D1, O1, O2, and O3 recomputed with K-means  $k = 12$ , K-means  $k = 4$ , and HDBSCAN. Source: data/aggregated/multi\_granularity\_switching.png.

## 5.8 Per-cluster semantic inspection

We extracted representative trajectory text from each K-means cluster and had a separate held-out reasoning model characterize the clusters blind to the paper’s regime labels. The detailed per-cluster tables are available in the released audit artefacts. The main result is that the four canonical regimes are all multi-cluster at  $k = 12$ , but the semantic axis of clustering differs by regime.

| regime | basin axis                                | mechanism                                                                                                                                                                                         | taxonomy implication                                                                          |
|--------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| O1     | register and style                        | append-mode continuation contracts toward high-probability continuation registers, such as sentimental narrative, policy-discursive exposition, reflective empathic prose, and technical tutorial | label preserved, but specified as register-contractive rather than topic-contractive          |
| O2     | seed family and local topic               | paraphrase preserves meaning while sanding surface form into conventional paraphrase                                                                                                              | period-2 dynamics remain, but basins are family-preserving rather than semantically absorbing |
| O3     | formal template                           | summarize-then-negate imposes a summary plus antithesis discourse shape while preserving seed-specific content                                                                                    | absorbing means template-absorbing, not content-convergent                                    |
| D1     | dialogue state and recent-context capture | append-mode dialog drifts into conversational acts such as coaching, reassurance, recommendation, journaling, or creative feedback                                                                | better described as dialogue-state-driven multi-basin than purely stylistic multi-basin       |

The semantic inspection preserves the regime taxonomy but sharpens the mechanism. O1 is the strongest true attractor case, with convergence along register rather than content. O2 and O3 are operator-shaped but content-preserving in different ways. D1 is governed by conversational state and recent-context capture rather than by a stable topic basin.

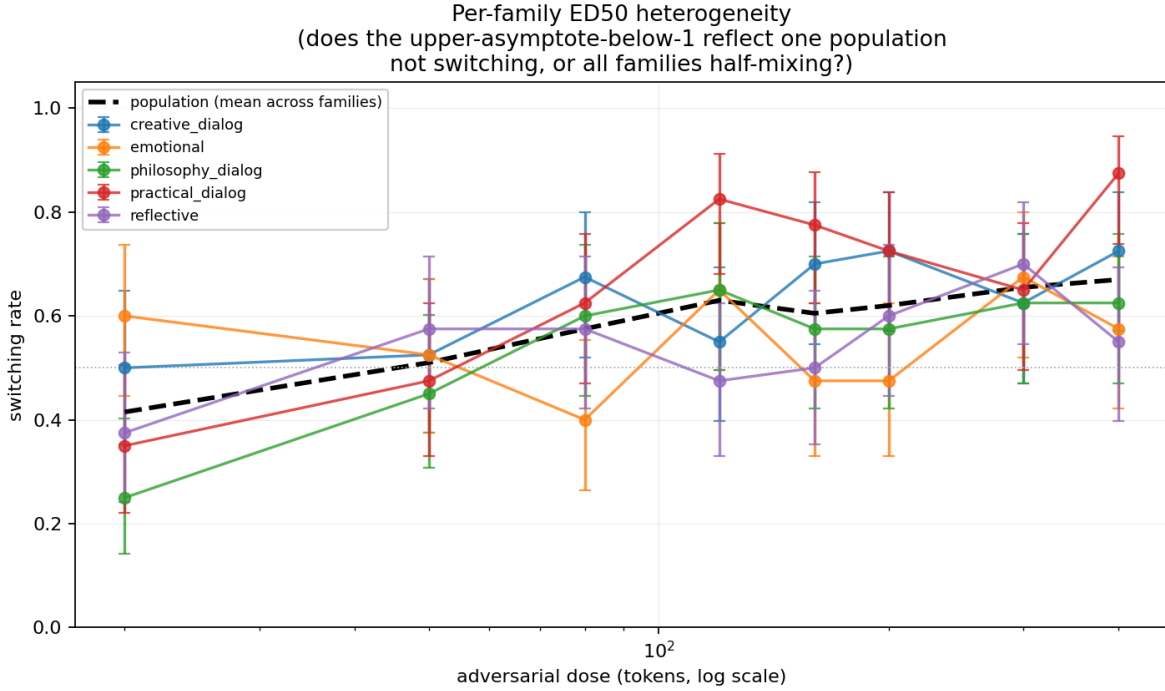
## 5.9 Per-family heterogeneity

The sparse O1 adversarial dose grid revealed substantial family-level heterogeneity. Each family-dose cell has  $n = 10$  trajectories, so the table is not a precise family-level ED50 estimate. It is a warning that the population curve mixes different local response profiles.

philosophy\_dialog shows a clear threshold-like increase up to dose 200. practical\_dialog increases after dose 80. creative\_dialog increases after dose 200. reflective is nearly flat, and emotional is non-monotone with

| family            | dose 20 | dose 80 | dose 200 | dose 400 |
|-------------------|---------|---------|----------|----------|
| philosophy_dialog | 0.10    | 0.40    | 0.90     | 0.50     |
| practical_dialog  | 0.40    | 0.20    | 0.70     | 0.80     |
| creative_dialog   | 0.20    | 0.40    | 0.30     | 0.60     |
| reflective        | 0.30    | 0.30    | 0.40     | 0.40     |
| emotional         | 0.30    | 0.40    | 0.40     | 0.10     |

a low 400-token endpoint. The dense rerun establishes a clean population-level raw dose response, but the wide family-cluster bootstrap interval is consistent with this underlying heterogeneity. Future replications should increase the number of prompt families rather than only the number of ICs per family.



**Figure 6:** Figure I. **Per-family O1 adversarial dose-response.** Lines show switching rate versus adversarial dose for each prompt family in the sparse O1 dose experiment, with  $n = 10$  trajectories per family-dose cell. Source: data/aggregated/per\_family\_ed50.png.

## Phase D, embedder and cross-generator checks

### 5.10 Embedding-space invariance

The main measurements use `text-embedding-3-small`. We re-embedded 5,000-step subsamples of representative experiments under two alternatives: `text-embedding-3-large`, a within-vendor scale-up, and `all-mpnet-base-v2`, a local cross-architecture sentence-transformer. We then recomputed recurrence, late sharpness dimension, and basin predictability.

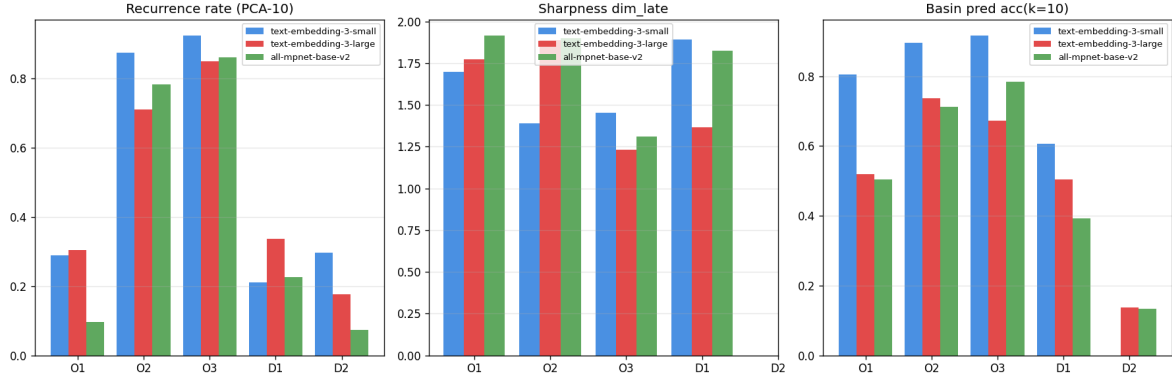
The rank-order result is selective. Basin predictability is the most embedder-invariant diagnostic, recurrence is partially invariant, and sharpness dimension is not invariant.

The load-bearing taxonomy distinction between replace-mode regimes and append/dialog regimes survives embedder substitution. O2 and O3 remain high-recurrence and high-predictability relative to O1, D1, and D2. The fine-grained sharpness-dimension ordering should be interpreted only within the original `text-embedding-3-small` measurement pipeline.

The  $n = 4$  cross-metric correlations among recurrence, switching, sharpness, and lock-in are descriptive only and are reported in the released aggregate tables rather than used as inferential evidence here.



| diagnostic                     | Spearman rank correlation vs<br>text-embedding-3-large | Spearman rank correlation vs<br>all-mpnet-base-v2 | interpretation                                                                      |
|--------------------------------|--------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------|
| recurrence rate                | +0.60                                                  | +0.60                                             | high/low split between<br>replace-mode and<br>append/dialog regimes is<br>preserved |
| basin predictability acc(k=10) | +0.80                                                  | +1.00                                             | strongest embedder-invariant<br>diagnostic                                          |
| sharpness_dim_late             | -0.40                                                  | +0.00                                             | embedding-specific and not<br>load-bearing                                          |



**Figure 7: Embedding-model ablation.** Bar plots compare three diagnostics across regimes after re-embedding subsamples with text-embedding-3-small, text-embedding-3-large, and all-mpnet-base-v2. Source: data/aggregated/embedding\_ablation/comparison.png.

### 5.11 Cross-model thesis verification

We also ran a cross-generator audit by substituting gpt-4.1-nano for gpt-4o-mini in the regime-level experiment set and evaluating six machine-checkable thesis predicates. This is a qualitative audit of the regime taxonomy, not a replication of the dense ED50 endpoint, the persistence endpoint, or the overwrite-versus-insert mechanism.

The regime-level predicates all passed on both generators:

| ID | audited claim                                                                      | gpt-4o-mini                            | gpt-4.1-nano                           | verdict      |
|----|------------------------------------------------------------------------------------|----------------------------------------|----------------------------------------|--------------|
| T1 | recurrence ordering separates replace from append/dialog                           | O1 0.272, O2 0.834, O3 0.905, D1 0.146 | O1 0.393, O2 0.840, O3 0.866, D1 0.168 | PASS on both |
| T2 | replace-mode perturbation switching remains high under original overwrite protocol | O2 adv 0.94, O3 adv 0.96               | O2 adv 0.94, O3 adv 0.88               | PASS on both |
| T3 | O1 neutral and lorem remain in drift-floor band                                    | control 0.00, neutral 0.24, lorem 0.18 | control 0.00, neutral 0.22, lorem 0.18 | PASS on both |
| T4 | O1 adversarial switching exceeds O1 lorem switching                                | adv 0.54 vs lorem 0.18                 | adv 0.38 vs lorem 0.18                 | PASS on both |
| T5 | D1 neutral switching exceeds 0.30                                                  | neutral 0.76                           | neutral 0.80, lorem 0.94               | PASS on both |
| T6 | publication-scale verdict labels match expected H1a/H1b tuples                     | expected tuples                        | identical tuples                       | PASS on both |

The structural taxonomy is preserved. Replace-mode regimes remain high-recurrence and high-switching under the original overwrite protocol. O1 remains content-sensitive, with adversarial text exceeding lorem text. D1 remains broadly susceptible to perturbation.

The magnitude shifts are also informative. gpt-4.1-nano has a smaller O1 adversarial-vs-lorem margin, +0.20 compared with +0.36 on gpt-4o-mini, suggesting shallower contractive basins. D1 lorem switching rises to 0.94 on gpt-4.1-nano, suggesting looser dialog-state anchoring.

The audit does **not** replicate the full headline endpoint. The following claims remain established only for the gpt-4o-mini experiments reported above:

| claim                                                            | replicated on<br>gpt-4.1-nano? | status                                        |
|------------------------------------------------------------------|--------------------------------|-----------------------------------------------|
| dense O1 adversarial ED50 <sub>raw</sub> $\approx$ 40 tokens     | no                             | not rerun in the cross-generator thesis audit |
| O1 natural stochastic-divergence floor of 34.7%                  | no                             | not rerun in the cross-generator thesis audit |
| persistent escape not reaching 50% up to 400 tokens              | no                             | not rerun in the cross-generator thesis audit |
| overwrite-versus-insert gap of 60-80 percentage points for O2/O3 | no                             | not rerun in the cross-generator thesis audit |
| V* density-landscape sensitivity and ordinal stability           | no                             | not rerun in the cross-generator thesis audit |
| regime-level qualitative predicates T1-T6                        | yes                            | passed on both generators                     |

The correct interpretation is therefore narrow but useful: the **regime-level qualitative claims** generalize to gpt-4.1-nano; the **dense endpoint calibration** remains a gpt-4o-mini result until rerun directly.

## Phase E, secondary analyses

### 5.12 Density landscapes are descriptive, not calibrated

The  $V(x) = -\log \hat{\rho}(x)$  density-landscape analyses visualize where trajectory clouds spend time in a joint PCA-2 projection. They are useful descriptive summaries of geometry, but they are not calibrated barrier-height estimates and they do not validate the token ED50 values in §5.1.

The main limitation is parameter sensitivity. For the O1 perturbation pilot, we recomputed  $V^*$  across 45 combinations of KDE bandwidth, grid resolution, and basin count. Per-condition coefficients of variation ranged from 14% to 24%. A single numerical  $V^*$  value is therefore not stable enough to quote as a calibrated barrier.

The ordinal pattern was more stable. Across the 45 parameter combinations, control had the highest  $V^*$  in 98% of combinations, adversarial had the lowest  $V^*$  in 89%, and neutral and lorem occupied the middle. This supports a weak geometric statement: the density-landscape summaries preserve a robust rank ordering within the O1 perturbation pilot. It does not support a quantitative  $V^*$  to ED50 conversion.

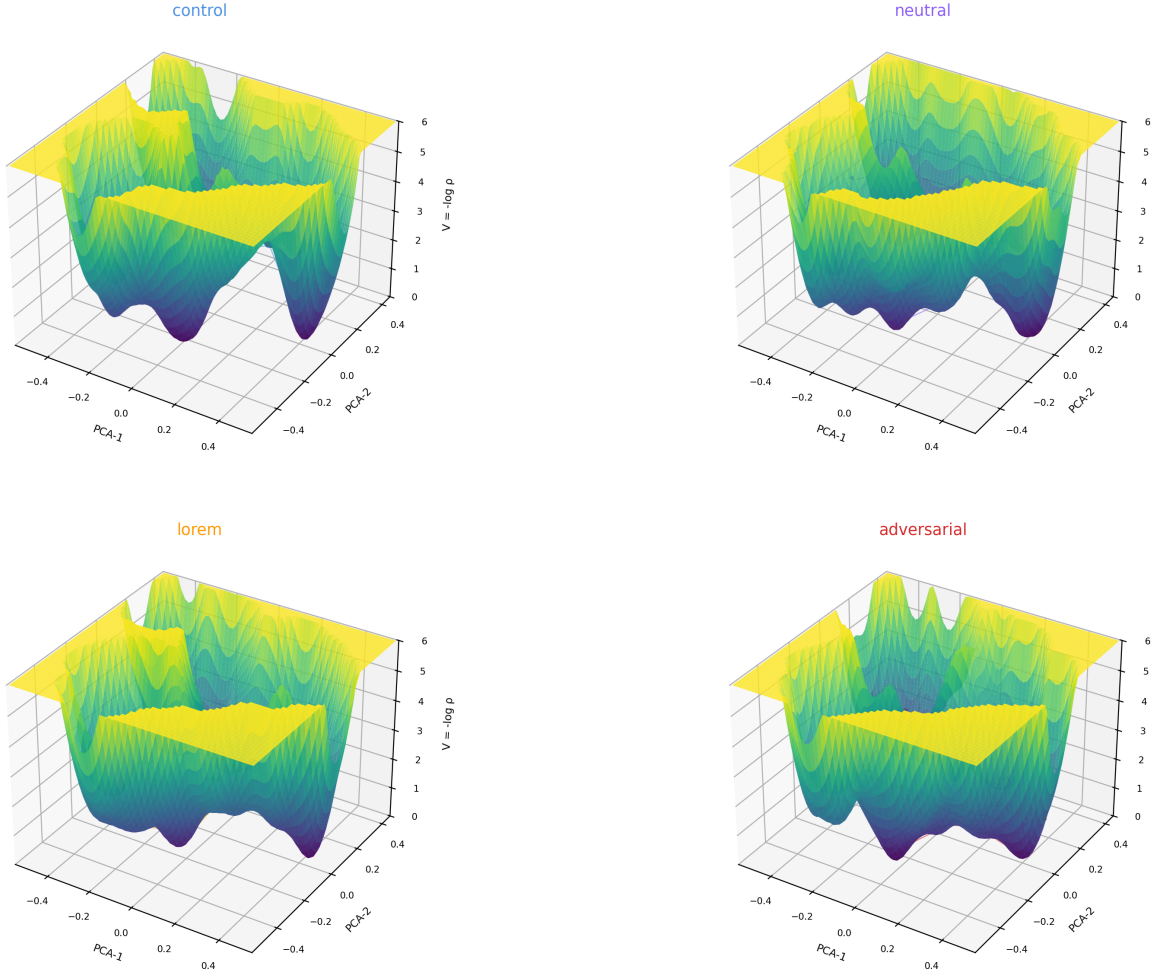
This distinction is especially important for replace-mode regimes. O2 and O3 can show high geometric separation while also showing high overwrite-protocol switching, because the perturbation and memory policy can create or occupy a different basin rather than cross a pre-existing ridge. Full  $V^*$  tables, RG merge-distance tables, and the parameter-grid sensitivity outputs are moved to §11.11.

### 5.13 Why exactly five regimes?

The five-regime taxonomy is not recovered cleanly by unsupervised clustering of bulk diagnostics alone. We assembled five-dimensional diagnostic vectors containing recurrence rate, late sharpness dimension, late  $\lambda_1$ , basin-predictability accuracy at  $k = 10$ , and adversarial switching rate. Internal validation indices did not select a single cluster count matching the five labels: silhouette favored two clusters, Calinski-Harabasz favored seven, and Davies-Bouldin favored six.

The substantive result is that bulk diagnostics separate O2 and O3 from the append/dialog regimes, but they do not cleanly separate O1 from D1. O1 and D1 have similar recurrence, contraction, and basin-predictability values at this diagnostic resolution. The perturbation protocol is what separates them: O1 shows content-dependent adversarial raw switching with out-of-distribution resistance, while D1 is broadly susceptible to dialog-state redirection and hardens with time. D2 is then distinguished by drill-down content gravity.

Free-energy 'bulk' landscape  $V(x) = -\log p(x)$  by condition (PCA-2)  
 injection at step 15 | wells = attractor basins; cliffs = inter-basin barriers; trajectory paths ride the surface



**Figure 8: Representative PCA-2 density landscapes for the O1 perturbation pilot.** Panels show  $V(x) = -\log \rho(x)$  for control, neutral, lorem, and adversarial conditions. Source: data/exp\_perturb\_01\_pilot/reports/perturbation/bulk\_landscape\_pca.png.

The five-way taxonomy is therefore supported by the union of bulk diagnostics and perturbation endpoints, not by either alone. Full unsupervised-clustering matrices, validation indices, and feature-space plots are available in the released aggregate tables.

At matched reduced scope, D1 showed narrower temperature variation than O1: D1 basin predictability at  $k = 10$  stayed in a 0.57-0.61 band, while O1 ranged from 0.52 to 0.65. However, the O1 reduced-scope  $T=0.8$  cell was 28 percentage points below the publication-scope  $T=0.8$  anchor, 0.52 versus 0.80, so O1 absolute temperature values are scope-confounded. The full temperature sweep is retained as exploratory secondary material in the released aggregate tables.

## 6 Discussion

Robustness in recursive LLM systems is not a scalar model property but emergent from how generated text is re-entered, retained, or overwritten. The main result is therefore not simply that recursive LLM loops have attractor-like regimes, but that apparent robustness depends on two separable choices: the memory policy that writes text back into state,

and the endpoint used to define a successful perturbation. Append-mode continuation shows a real in-distribution raw dose response, but not a durable escape threshold. Replace-mode loops, meanwhile, look fragile mostly because the update rule overwrites prior state. Thus the practical unit of analysis is not the prompt alone, or the model alone, but the generator-nudge system evaluated under raw, net, and persistent endpoints.

### 6.1 Recursive-loop robustness is determined by memory policy and endpoint choice

The central lesson is that recursive-loop behavior is determined jointly by the generator and the context-update rule. In the formalism of §3.1, the model samples  $Y_t \sim P_{\theta}(\cdot | X_t; f)$ , while the nudge  $\mathcal{N}_f$  decides how that sampled text becomes the next state. Two loops with the same generator and similar prompt instructions can therefore have different perturbation responses solely because one preserves prior context and the other overwrites it.

The second lesson is that robustness is endpoint-dependent. A loop can move without staying moved; two unperturbed runs can diverge without any perturbation; and an overwrite-style update can make a system appear fragile even when inserted text has little effect. In O1, adversarial append-mode perturbations produce a finite raw-switching dose response, but the stochastic floor and lack of persistent crossing prevent interpreting that response as durable escape. Raw switching is appropriate for detecting sensitivity; net switching estimates perturbation effect above stochastic drift; persistent escape is the relevant endpoint when the safety concern is durable redirection after recovery turns.

### 6.2 Append, replace, and dialog implement different memory mechanisms

Append, replace, and dialog are not implementation details; they are different memory mechanisms. In append mode, the perturbation enters an already accumulated context and must compete with prior trajectory mass. This makes append-mode continuation resistant to irrelevant text: O1 neutral and lorem perturbations remain near the out-of-distribution drift floor, while in-distribution adversarial text produces the graded raw dose response measured in §5.6.1.

Replace mode implements a different mechanism. The previous state is discarded, so the next state is dominated by the replacement. The overwrite-vs-insert probe makes this visible: overwrite-mode perturbations in O2 and O3 switch far more often than inserted perturbations, indicating that much apparent replace-mode fragility is attributable to overwrite mechanics rather than a low injected-token barrier (§5.17).

Dialog sits between these cases. It preserves a running transcript like append mode, but role-structured turns give recent utterances high local influence. D1 therefore behaves as a dialogue-state-driven multi-basin regime, while D2 suggests topic-anchored content gravity, meaning repeated return to the same semantic cluster despite local perturbation (§5.8, §5.14).

The expected signatures differ: append should show specificity and graded dose response; replace should show a large overwrite-insert gap; dialog should show recency and role sensitivity; summary or pinned-memory systems should be tested for what information survives compression.

### 6.3 A raw-switching ED50 is not a persistent-escape barrier

The most important interpretive correction is that ED50<sub>raw</sub> is not the same quantity as a persistent-escape barrier. Raw switching asks whether the perturbed trajectory’s terminal cluster differs from its paired control. That endpoint is useful, but it conflates true redirection, ordinary stochastic divergence, and transient movement that later recovers. Persistent escape is stricter: the trajectory must visibly jump at injection and remain in the post-injection basin through the terminal step (§3.1.2). Persistent escape is the appropriate strict endpoint for durable basin change, but transient raw switching may still matter in applications where a single action is consequential.

Under the permissive raw endpoint, O1 adversarial append-mode continuation has a clear dose response: ED50<sub>raw</sub>  $\approx$  40 tokens, with estimates 36, 41, and 52 tokens across the 4PL, GLMM, and family-cluster-bootstrap analyses (§5.6.1). But the raw curve plateaus near 67%, not 100%. Moreover, paired controls already disagree at about 35%, so subtracting the stochastic floor leaves a maximum net effect of only +32 pp at 400 tokens, below the +50 pp criterion for ED50<sub>net</sub>.

The persistent endpoint is smaller still. At the highest tested dose, persistent escape reaches only 16% under canonical K-means  $k = 12$ , 10% under K-means  $k = 4$ , and 39.5% under HDBSCAN (§5.15). None crosses 50% in the tested 5 to 400 token range. The correct conclusion is therefore: O1 has a finite raw-switching dose response, but no measured persistent-escape barrier in this experiment.

#### 6.4 Density landscapes diagnose basin geometry but do not calibrate token barriers

The empirical potential landscape  $V(x) = -\log \rho(x)$  is useful as a diagnostic picture of trajectory density, but it should not be read as a calibrated token barrier. The landscape is computed after embedding, projection, smoothing, and basin detection, so its absolute values depend on the representation and analysis choices. The parameter-grid sensitivity analysis confirms this: per-condition  $V^*$  values vary with coefficient of variation 14 to 24% across KDE bandwidths, grid resolutions, and basin-count settings (§5.16).

Their value is diagnostic: they show whether trajectories occupy one or several basins, whether perturbations move mass toward alternative density regions, and whether basin structure is stable under analysis choices. What survives here is the ordinal claim. In the O1 perturbation pilot, the rank ordering of  $V^*$  is stable across 89 to 98% of parameter combinations: control is usually highest, adversarial usually lowest, and neutral or lorem occupy the middle (§5.16). This supports the use of density landscapes as descriptive basin-geometry tools. It does not justify converting  $V^*$  into tokens, nor does it independently validate ED50\_raw.

#### 6.5 Robust recursive-loop evaluations must report floors, persistence, and overwrite mechanics

1. **Report the context-update rule.** Every recursive-loop evaluation should state whether the system appends, replaces, role-structures, summarizes, pins, rolls, or hybridizes memory, because the same generator can show different perturbation responses under different nudges (§3.1, §5.17).
2. **Report raw, net, and persistent-escape endpoints rather than raw switching alone.** Raw movement says whether the trajectory changed, net switching subtracts the control-vs-control stochastic floor, and persistent escape asks whether the perturbation produced a durable basin change after recovery turns (§3.1.2, §5.6.1, §5.15).
3. **For replace-style or summary-memory systems, run an overwrite-vs-insert control.** If overwrite switching is high but insert switching is low, the measured fragility is partly the memory policy discarding prior state rather than the injected text overcoming a basin barrier (§5.17).

Box 1 in Supplementary §11.12 operationalizes these points as a minimum reporting checklist, including memory policy, perturbation insertion mode, stochastic floor, endpoint definition, recovery horizon, and overwrite-vs-insert controls.

#### 6.6 The unresolved question is external validity across generators and real agent scaffolds

The next step is to ask which parts of this decomposition survive in full agent scaffolds: cross-vendor generators, persistent tool state, retrieval memory, code repositories, and policy-constrained actions. The within-vendor replication on gpt-4.1-nano preserves the qualitative thesis predicates (§5.21), but both generators are from the same vendor family. Cross-vendor and open-weight replication remain necessary before the architecture-vs-generator partition can be treated as general (§7.1, §8.1).

The same agenda extends to real agent scaffolds. The present paper studies recursive text loops with controlled update rules and embedding-space observables; it does not directly evaluate coding agents, tool-using assistants, SWE-Bench tasks, jailbreak benchmarks, production indirect prompt injection, or factuality-graded hallucination settings (§7.5). For indirect prompt injection or coding agents, the analogous question is not whether the transcript topic changes, but whether an injected artifact produces action-level divergence above a no-injection floor and whether that divergence persists through subsequent planning or repair steps.

Those domains can adopt the same endpoint decomposition, but their observables must be application-specific: patch family, files touched, tests passed, policy violations, tool-call traces, or grounded factual claims. The paper’s position is therefore bounded but actionable: recursive-loop stability is measurable, and the measurement becomes meaningful when memory policy, stochastic floor, and persistence endpoint are made explicit.

### 7 Limitations

The results establish dose-response and regime-separation behavior for bounded, English, text-only recursive loops under the tested generator-nudge families. They do not establish universality across model vendors, memory policies, languages, or deployed tool-using agents.

The experiments therefore support a structured account of recursive LLM dynamics within a defined operating envelope. That envelope is broad enough to distinguish regimes, reproduce perturbation effects, and compare generator-nudge

conditions, but it is not broad enough to make representation-free or deployment-general claims. The limitations below specify where the evidence is strongest, where the measurements are operational rather than invariant, and where additional observables would be required.

## 7.1 Evidence is strongest for OpenAI generator-nudge systems

The model-coverage result is broad enough to rule out a single-run curiosity, but not broad enough to establish model-agnostic dynamics. The current audit spans six generators, including Anthropic and Google systems, yet the densest evidence remains concentrated in OpenAI generator-nudge systems, especially the gpt-5-mini and gpt-5-nano O1 and D1 cells. Those cells carry the deepest replication, the most complete artifact stack, and the clearest regime-level separation.

This matters because the qualitative taxonomy is motivated by the generator-nudge factorization in §3, not by a special property of one model checkpoint. The append, replace, and dialog distinctions plausibly arise from how the update operator rewrites the next context. However, barrier heights, basin geometry, switching thresholds, and even the number of empirically separable regimes can vary with decoding behavior, tokenizer structure, alignment tuning, refusal style, context-management details, and vendor-specific instruction-following behavior.

The cross-model evidence supports preservation of the append/replace/dialog qualitative pattern under the tested generator class; it does not support equality of barrier heights, switching probabilities, basin counts, or token-dose thresholds across vendors. The hierarchy of evidence is therefore important. The strongest claim is that the headline regimes survive the most replicated cells and the pre-registered predicate checks. A weaker but still useful claim is that the qualitative pattern is visible across the audited generator set. The audit does not establish numerical equivalence across vendors, and it does not license a model-agnostic claim about all current or future LLMs.

The cross-model audit should consequently be read as evidence of shape preservation under a restricted generator class. It supports the existence of recurring append-like, replace-like, and dialog-like patterns in the tested systems, but it does not show that these patterns have fixed quantitative parameters. Future work should expand the vendor set, decoding policies, context managers, and refusal regimes before treating the taxonomy as a general law of recursive language-model behavior.

## 7.2 Basins, barriers, and tokens are operational measurements

The basins and barriers reported here are measurements in an operational pipeline, not representation-free physical constants. Trajectories are observed through an embedding model, projected for analysis and visualization, and summarized with density, recurrence, switching, and dose-response statistics. §5.20 reports an explicit representation ablation against a larger OpenAI embedding model and a sentence-transformer model; the attractor-like structure is robust across those tested observables. That robustness does not make the absolute geometry independent of the representation.

The empirical potential landscape,  $V(x) = -\log \rho(x)$ , is a descriptive density summary. The Dijkstra barrier  $V^*$  depends on the kernel-density estimator, PCA-2 reduction, grid resolution, and path construction. These quantities are useful because they impose a common geometric language on many trajectory families, and because their relative ordering can be compared with behavioral perturbation thresholds. They should not be interpreted as thermodynamic free energies or as absolute invariants of the underlying model.

The same caveat applies to token barriers. Tokens are directly measurable, practically interpretable, and closely aligned with how perturbations are injected in §4.5, but they are not the ultimate information unit. A more model-comparable version of these perturbation barriers would likely use conditional surprisal or log-probability cost; because generation logprobs were not stored, the present token-dose results should be treated as operational proxies. The reported token-dose thresholds are therefore useful for within-pipeline comparison, but they should not be read as universal energetic costs or as vendor-independent information barriers.

Predictive basin assignments are also conditional on the validation design. Where basin assignment was modeled predictively, the relevant performance estimates should be read under the group-aware validation design reported in §5; in particular, D1 predictability is evidence for structured separation within the sampled trajectory families, not for arbitrary out-of-family generalization (group-aware  $\text{acc}(k=10) = 0.34$ ,  $n=450$ ). This distinction matters because recursive trajectories can share prompt family, generator family, seed structure, and local lexical habits. A classifier can capture real within-sample structure without proving that the same decision boundary will transfer to arbitrary new tasks, languages, generators, or memory regimes.

The representation pipeline is therefore best understood as an instrument. It reveals reproducible organization in the sampled trajectories, and the ablations in §5 help rule out a single-embedding artifact. It does not eliminate observer dependence. Stronger representation claims would require storing generation logprobs, testing additional embedding families, measuring raw conditional probabilities where available, and verifying that basin and barrier relationships persist under alternative trajectory observables.

### 7.3 The experiments cover bounded, English, static-prompt recursions

The reported dynamics are properties of bounded, English-language, static-prompt recursive loops. All main runs use a finite context cap with tail clipping, English prompts and seeds, and relatively short generated outputs. In concrete terms, the main recurrence settings use a bounded rolling context of approximately [N tokens/turns], per-step generated outputs in the approximate range [range], and clipping or truncation rules applied after [range] accumulated context; the exact values, caps, and run settings appear in §4. These parameters are part of the experimental condition, not incidental implementation details.

This is a natural setting for controlled recurrence experiments, because it isolates the effect of the nudge operator while keeping the observable trajectory compact enough for repeated perturbation and embedding analysis. The same bounded design makes the dose-response protocol feasible: perturbations can be inserted, the subsequent trajectory can be tracked, and switching or recovery can be measured under comparable context budgets. However, the design also means that the reported basins and barriers are properties of this bounded-memory recurrence, not of every possible recursive use of a language model.

The bounded-memory assumption is especially consequential for append mode. A no-clip pilot suggests that removing clipping deepens the append basin and reduces recurrence, which means the measured append-mode barriers are not properties of append mode in the abstract. They are properties of append mode under a specific bounded-memory recurrence. Larger context windows, different truncation rules, retrieval-augmented memory, explicit memory compression, or pinned long-term goals may change both the basin depth and the route by which perturbations persist.

The language and prompting scope is similarly narrow. The experiments do not test multilingual trajectories, code-heavy trajectories, very long-form generation, or systems in which the system prompt is rewritten online. Prompt drift, refreshed meta-instructions, tool-generated state, or long-horizon document construction could fragment a basin that appears stable under static prompting, or stabilize a replace-mode regime that appears weak when each step is only a short text rewrite. Additional work should test whether the same regime taxonomy remains legible when prompts, languages, modalities, and memory policies are allowed to vary more aggressively.

### 7.4 The drill-down dialog regime remains exploratory

The drill-down dialog regime is the least mature of the reported regimes. The current D2 evidence indicates a distinct form of topic-anchored content gravity under role-structured questioning, but it was tested at substantially smaller scale than the main O1-O3 and D1 regimes. The reported switching estimate is 64% with a  $\pm 10$  percentage point bootstrap confidence interval from 25 trajectories at 50 steps.

This is enough to motivate D2 as a candidate regime, but not enough to place it on the same empirical footing as the publication-scale cells. Dialog structure is a rich nudge family: drill-down questioning, debate, role-play, adversarial interrogation, tutoring, self-critique, and multi-party deliberation may create different balances between style persistence and content anchoring. Small changes in role framing, question order, answer length, or conversational memory could alter the apparent basin structure.

Accordingly, D2 is treated as hypothesis-generating in the present taxonomy. It supplies evidence that role-structured dialog can produce topic-anchored recurrence, but it should not yet be treated as a fully replicated regime with stable quantitative thresholds. A publication-scale D2 study would need deeper replication, broader generator coverage, explicit comparison among dialog subfamilies, and perturbation tests matched to the main O1-O3 and D1 protocols. Until then, D2 should be read as a promising extension of the framework rather than as a regime with the same evidentiary status as the strongest cells.

### 7.5 Production agent and tool-use claims require new observables

The experiments measure recursive language-model loops, not deployed coding agents or tool-using autonomous systems. They do not include file-system state, code edits, compiler feedback, test execution, tool schemas, repository-specific correctness criteria, or multi-step planning traces. The implications for coding agents are architectural extrapolations from recursive-loop dynamics, not measurements of deployed coding systems.

A production-agent benchmark would need additional observables. Patch diffs, files touched, tests run, failing tests remaining, tool-call sequences, policy violations, injected-document provenance, and post-perturbation plan persistence would need to be measured alongside or instead of embedding-space trajectory structure. In such systems, the relevant basin may live partly in text, partly in external state, and partly in the interaction between model outputs and tool-mediated environment changes. A trajectory embedding alone would miss many of the state variables that determine whether an agent recovers, drifts, or remains captured by a perturbation.

The key bridge is that memory policy becomes a behavioral variable. Full-history append, rolling-window truncation, generated-summary replacement, pinned-goal memory, provenance-preserving hybrid memory, and retrieval-augmented state can induce different perturbation profiles even when the base model and task are held fixed. The framework in §3 and the perturbation protocol in §4.5 transfer naturally to this setting, but the numerical barriers in §5 do not transfer without re-measurement.

Thus, the transferable contribution is the measurement logic, factorizing generator, nudge, memory, and perturbation dose, not the specific numerical thresholds measured in the present text-only loops.

## 8 Future directions

*The next step is to turn the present perturbation framework from a controlled recursive-loop study into a comparative measurement program for model families, memory policies, dialog scaffolds, and deployed agents.*

The proposed program proceeds from external validity, to mechanistic measurement, to scaffold ablations, to applied agent and safety settings.

### 8.1 Cross-vendor replication at publication scale

*Hypothesis: append/replace/dialog regime ordering is invariant across model families under matched perturbation dose and memory policy.*

The highest-priority extension is an MVP cross-vendor replication rather than an exhaustive survey. The central question is not whether barrier heights match numerically across providers, but whether the ordering of append, replace, and dialog regimes survives across generators with different alignment methods, tokenizer families, refusal policies, context handling, and decoding implementations. If the same qualitative ordering appears under matched perturbations and matched analysis code, this would support the claim that regime structure is a property of the generator-nudge system rather than a peculiarity of one model family.

The MVP should include Anthropic, Google, Mistral, and open-weight models, with the current provider family retained as a reference where feasible. The design should use N=20-60 trajectories per cell as cost permits, with all three headline operators included: O1, D1, and D2. D2 is especially important because dialog topology is under-sampled in the present evidence base and may vary more strongly across vendors than simple append or replace loops. The goal is not to make every cell maximally powered at the first pass, but to establish a compact, reproducible panel that can detect whether the regime map is stable enough to justify larger audits.

Primary endpoint: preservation of regime ordering across vendor families. Primary success criterion: regime ordering preserved in at least 3 of 4 vendor families with N=20-60 trajectories per cell across O1, D1, D2. Secondary endpoints should include perturbation response curve shape, recurrence and persistence rank ordering, and the drift of ED50 and basin-score estimates relative to the reference implementation.

The audit should preserve the separation between verdicts and numbers used in the present analysis. Exact recurrence rates, basin scores, and ED50 values should be expected to drift because of provider-specific sampling behavior, tokenizer differences, and alignment tuning. The load-bearing test is qualitative: whether the regime ordering, perturbation response curves, and persistent geometry remain aligned under the same nudge definitions, the same memory limits, and the same analysis code. A negative result would also be informative. If one vendor reverses append and replace behavior, or if dialog perturbations dominate only in some families, the framework would identify where scaffold and model properties interact rather than treating susceptibility as a single scalar model trait.

### 8.2 Logprob-based barrier heights and basin geometry

*Hypothesis: switching thresholds align more tightly with conditional surprisal than with token count.*

A second priority is to move from token barriers to information-theoretic barriers. Token dose is easy to control, audit, and interpret, but the natural cross-model unit is conditional surprisal. Future runs should store generation logprobs



whenever provider APIs expose them, allowing perturbation cost to be expressed in nats as well as in tokens. The pipeline already anticipates this through a logprob-capture option, but availability remains uneven across providers, models, and endpoint types.

Logprob-based barriers would test whether behavioral switching thresholds align more directly with information cost than with perturbation length. A short perturbation containing highly surprising content may impose a larger effective kick than a longer perturbation made of predictable boilerplate. Conversely, long in-distribution perturbations may contain many tokens while adding relatively little conditional surprise. Measuring both units would separate length effects from information effects and would make cross-model comparison less dependent on tokenizer granularity.

Primary endpoint: improvement in threshold alignment when perturbation dose is measured in conditional surprisal rather than token count. Secondary endpoints should include token ED50, surprisal ED50, and the correspondence between information barriers and embedding-derived basin barriers. A practical analysis would fit switching curves in both units and compare uncertainty-normalized threshold dispersion across models and perturbation classes. If surprisal thresholds cluster more tightly than token thresholds, this would indicate that the effective perturbation dose is better measured by information cost. If they do not, token length may remain the more operationally relevant unit for scaffold evaluation.

The same extension should be applied to basin geometry. The empirical  $V^*$  barrier summarizes density geometry in an embedding projection, whereas an information barrier would summarize how costly it is for the generator to move from one behavioral basin to another under its own predictive distribution. Agreement between these quantities would strengthen the structural interpretation of basins as both geometric and generative objects. Disagreement would identify cases where embedding-space proximity and generative difficulty diverge. For example, two states may be close in semantic embedding space but separated by a high generative barrier if the transition requires an unlikely commitment, refusal reversal, or role change. Conversely, distant embedding states may be easy to traverse if the scaffold repeatedly primes the transition.

The logprob program should also record where probabilities are unavailable, truncated, or provider-normalized in incompatible ways. These limitations should be treated as part of the measurement result rather than as incidental missingness. A future standard for perturbation audits should specify which logprob fields were exposed, whether they correspond to generated tokens only or also prompt tokens, how refusals and tool calls are represented, and whether probability traces can be reproduced under fixed seeds or fixed decoding settings.

### 8.3 Memory-policy ablations and adversarial perturbations

*Hypothesis: context-update rules, not only base models, determine persistent-escape probability.*

The memory policy should become an explicit experimental factor. The present bounded-memory setting is intentionally simple, but real systems use rolling windows, summaries, pinned instructions, retrieval, tool-output buffers, and hybrid context stores. Future experiments should cross the same generator and task with multiple context-update rules so that model fragility can be separated from scaffold fragility. The key question is not only whether a perturbation changes the next response, but whether the context-update rule preserves, amplifies, attenuates, or quarantines the perturbation over subsequent turns.

The core ablation should use a factorial matrix:

- Memory policy: full append / rolling window / summary replacement / pinned-plus-rolling hybrid
- Perturbation position: early context / latest turn / summary / pinned instruction / tool output
- Perturbation type: irrelevant long text / misleading explanation / malicious package text / false log
- Outcome: immediate switch, persistence, recovery, recurrence

Primary endpoint: persistent-escape probability as a function of memory policy. Secondary endpoints should include immediate switching, recovery after neutral continuation, and recurrence after apparent recovery. This decomposition is essential because a scaffold may reduce immediate switching while increasing recurrence, or may recover quickly from ordinary irrelevant text while remaining vulnerable to a false summary or poisoned tool output.

Longer contexts and longer per-step outputs should be included after the MVP matrix is stable. Longer recursive writes may deepen append basins, fragment replace basins, or create regimes in which short-horizon and long-horizon stability disagree. Periodic summarization may suppress benign drift while amplifying a malicious or misleading summary error. A summary that converts an injected claim into a compact durable belief can be more damaging than the original perturbation. The relevant endpoint is therefore not compression quality alone, but perturbation response under controlled dose, position, and provenance.

Perturbation position should be treated as a first-class variable. A perturbation inserted at the beginning of memory, immediately before the model response, inside a generated summary, inside a pinned instruction field, or inside a tool-output buffer may have different persistence even at the same token dose. Controlled adversarial perturbations should include irrelevant long logs, misleading documentation, targeted false explanations, malicious package-style content, and false failing-test logs. The output should be a memory-policy ablation harness that measures whether a scaffold reduces persistent escape without merely suppressing ordinary adaptation or hiding state changes in an uninspected summary.

#### 8.4 Publication-scale dialog and coding-agent benchmarks

*Hypothesis: dialog topology and agent scaffold introduce distinct susceptibility profiles not predicted by single-turn model behavior.*

Dialog needs its own benchmark map. The present dialog regime is only a starting point, and different conversational topologies may define different nudge operators. Drill-down dialog is the first candidate beyond the main dialog regime, but debate, role-play, brainstorming, adversarial questioning, tutor-student exchange, and multi-party deliberation may each produce distinct switching and recovery profiles. These topologies should be evaluated with the same endpoint decomposition used for recursive perturbations: raw switching, net switching above stochastic floor, persistence, recurrence, and basin geometry.

Primary endpoint: topology-specific susceptibility profile, defined as persistence adjusted for matched stochastic controls. Secondary endpoints should include raw switching, recurrence, and basin separation across dialog states. This framing prevents dialog evaluation from collapsing into a single compliance score. A topology that produces frequent immediate shifts but rapid recovery is different from one that rarely shifts but, once shifted, remains redirected for many turns. The benchmark should therefore score trajectories, not isolated answers.

**Dialog topology benchmark.** Each dialog topology should be implemented as a reproducible scaffold with fixed role instructions, turn order, memory policy, and perturbation injection point. Drill-down tasks can test whether repeated clarification creates stable commitments. Debate tasks can test whether adversarial framing induces durable position changes. Tutor-student exchanges can test whether correction loops consolidate false explanations. Multi-party deliberation can test whether a false statement becomes persistent when repeated or endorsed by another speaker. The benchmark should include paired controls for natural dialog drift, because ordinary conversation can change state even without an explicit perturbation.

**Agent scaffold benchmark.** The same endpoint decomposition can be adapted to coding-agent benchmarks, including SWE-Bench-style tasks and smaller instrumented repositories. Perturbations should be placed in repository files, issue comments, tool outputs, package documentation, failing-test logs, generated summaries, or planning traces. Each placement should be labeled by the failure mode it probes. Perturbations in prompts, plans, and issue comments primarily test reasoning susceptibility. Perturbations in summaries and rolling memory test memory persistence. Perturbations in tool outputs, test logs, package documentation, and retrieved snippets test tool-trace contamination. Perturbations in source files, tests, and repository documentation test repository grounding, meaning whether the agent remains anchored to the actual codebase rather than to an injected narrative about it.

For coding agents, paired controls should estimate the stochastic floor of patch-family variation, test-selection variation, and pass/fail variation. Perturbation runs should then measure whether the agent remains on the injected strategy after additional plan-edit-test cycles. This adaptation would distinguish ordinary run-to-run variation from durable redirection. Two agents can have the same pass rate while differing sharply in perturbation susceptibility. Likewise, the same base model can show different escape profiles under full-history append, rolling-window memory, summarized memory, or tool-output retention. A coding-agent benchmark built on this protocol would therefore separate model fragility from scaffold fragility, which current leaderboards often conflate.

#### 8.5 Persistent-escape barriers for safety and instruction-injection robustness

*Hypothesis: persistent escape is separable from immediate compliance and should be measured as a multi-step recovery process.*

Safety and instruction-injection experiments should measure persistent escape directly. The raw-switching ED50 reported in this paper is not a persistent-escape barrier. Raw switching measures the perturbation dose at which an immediate response changes state. Persistent escape asks whether the system remains redirected after the perturbation is no longer novel, after additional turns are generated, and after the scaffold has had opportunities to recover.

Primary endpoint: persistent escape after neutral continuation. Secondary endpoints should include raw escape, net escape above control variation, and recovery rate. Additional useful summaries include time to recovery, recurrence after recovery, and whether the escape is carried by model output, memory state, generated summary, tool trace, or pinned instruction. These experiments would not constitute a comprehensive safety evaluation; they would measure scaffold- and model-specific susceptibility to durable redirection under controlled perturbations.

This distinction is central for safety and instruction-injection robustness. A model that briefly follows a malicious instruction and then returns to the intended policy is different from a model whose memory or dialog state has been durably hijacked. Conversely, a system that refuses immediately but stores the malicious content in a summary or tool trace may create a delayed failure that is invisible to single-turn scoring. Future experiments should therefore score multi-step post-perturbation trajectories, not only the first switched response. Neutral continuation turns are especially important because they test whether the system returns to the intended basin without being explicitly corrected.

The same design can test whether alignment creates, removes, or reshapes behavioral basins. Base and safety-tuned models should be compared under the same nudge families, memory policies, and perturbation classes to determine whether safety training changes barrier height, basin count, switching geometry, or recovery dynamics. Agent frameworks should expose the context-update rule as a traceable object: which turns are retained, which are summarized, which facts are pinned, which tool outputs are marked untrusted, and which generated summaries replace prior state. Without that instrumentation, persistent-escape failures cannot be attributed cleanly to the model, the memory policy, or the surrounding scaffold.

**Program deliverables.** The proposed program should produce:

- A cross-vendor benchmark suite with matched perturbation scripts
- A memory-policy ablation harness with traceable context-update rules
- Logprob-enabled barrier analysis when provider APIs permit
- Dialog and coding-agent perturbation datasets
- Public analysis code for recurrence, persistence, ED50, and basin geometry

## 9 Data, code, and reproducibility

### 9.1 Public trajectories, artefacts, and audit trail

The repository is organized so that the paper’s claims can be traced from released trajectories through embeddings, per-experiment metrics, aggregate tables, figures, ED50 fits, and audit checks. The public release contains 37 experiments and 3.3 GB of raw trajectories. These trajectories are the canonical input for reproducing the numerical results reported in the paper.

Because hosted generation APIs may change, exact reproduction of the paper’s numerical results should use the released `steps.jsonl` trajectories; regeneration of model outputs is provided as a procedural replication path rather than a bitwise reproduction path. This distinction is important: the released trajectories fix the model responses used for the paper, while rerunning generation tests whether the same experimental protocol produces comparable outcomes under the currently available hosted models.

The repository includes three audit documents that provide the main reproducibility map. `COVERAGE.csv` records which expected artefacts are present for each experiment. `EVIDENCE.md` links substantive claims to backing data files, source-code functions, and regenerating commands. `RESULTS.md` checks reported numerical claims against the canonical aggregate tables and currently reports 103/103 numerical-claim audit cells reproducing within tolerance. Additional repository documentation describes the experiment catalogue, supersession relationships among runs, and the analysis history that led to the final design.

### 9.2 Codebase, licences, and runtime environment

Code is available in the public repository <<https://github.com/kaplan196883/llmattr>> and archived at [Zenodo DOI to be assigned at acceptance], corresponding to the release tagged for this manuscript. Code is released under GPLv3. Trajectories, embeddings, and derived analysis artefacts are released for reuse with attribution; OpenAI-generated outputs and embeddings are subject to the provider’s terms.

The project is intended to run under Python 3.11+ in a Conda-managed environment. Core dependencies include numerical, statistical, plotting, image-processing, and video-generation libraries such as `numpy`, `scipy`, `scikit-learn`,

scikit-image, matplotlib, pandas, and imageio-ffmpeg. The repository separates reusable analysis modules, experiment definitions, command-line scripts, configuration files, tests, raw data, derived artefacts, and aggregate outputs. This separation is intended to make it possible to inspect or rerun individual components without executing the full pipeline.

The codebase contains the primitives used for embedding management, dynamical-systems metrics, perturbation analyses, aggregate summaries, ED50 fitting, plotting, and audit generation. The scripts directory contains the command-line entry points used to rebuild the paper-level outputs. Configuration files define experiment-specific parameters, while the data directory stores the released trajectories and regenerated outputs when the pipeline is rerun locally.

### 9.3 Approximate cost and runtime

At the time of analysis, regenerating canonical OpenAI embeddings cost approximately US\$30, *while regenerating model generations cost* these estimates depend on provider pricing and model availability. These costs are not required to verify the paper’s numerical claims if the released trajectories and derived embeddings are used. They are provided to make the computational scale of a full procedural rerun transparent.

A lower-cost exact-reproduction path is to download the released trajectories and rerun local embedding-dependent and downstream analyses from those fixed inputs. Users who wish to avoid hosted embedding APIs can substitute local sentence-transformer embeddings for exploratory replication or representation ablation. Such runs should be interpreted as representation-specific replications, not exact regenerations of the canonical embedding pipeline.

Full local embedding and analysis on the released trajectories took approximately 2 hours wall-time on a 40-core reference machine in the authors’ environment. This is a benchmark for scale, not a reproducibility guarantee. Runtime will vary with storage bandwidth, CPU count, memory, embedding backend, plotting options, and whether animations or perturbation visualizations are regenerated.

### 9.4 Tests and claim verification

The analysis primitives and integration paths are covered by 99 pytest tests. The standard test command is `PYTHONPATH=. python -m pytest tests/ -q`. These tests exercise reusable components and integration behavior, while the audit scripts check the paper-level claims against regenerated artefacts and aggregate tables.

The canonical reproduction path is: download raw trajectories; compute or load embeddings; run per-experiment analyses; aggregate cross-experiment tables; run the coverage and numerical-claim audits. This schematic path is implemented by the repository scripts and can be executed at different levels of completeness depending on whether the user wants to verify the published numbers, regenerate selected figures, or procedurally rerun the full experimental workflow.

Two audit entry points are load-bearing for the claim trace. `python -m scripts.build_coverage` regenerates the artefact-coverage audit, and `python -m scripts.publication_summary` regenerates the numerical-claim verification table, including ED50 and summary-table checks. Together with the released trajectories, tests, evidence map, and aggregate outputs, these scripts provide the end-to-end path from each reported claim to the code and data used to recreate it.

## 10 Acknowledgments

We acknowledge gpt-4o-mini and text-embedding-3-small (OpenAI), the open-source ecosystem (numpy, scipy, scikit-learn, scikit-image, matplotlib, pandas, imageio-ffmpeg), and the [Tuci et al. \[2026\]](#) framework for finite-time Lyapunov spectra of sampling-based generators.

## 11 Supplementary appendix

This appendix consolidates the audit tables, mathematical proofs, implementation definitions, reproducibility commands, and practitioner-facing reporting guidance supporting the main paper. The material is organized into twelve subsections so that decision-grade results, methods details, and engineering artifacts can be audited without fragmenting the main narrative.

### 11.1 Extended Data Table 1, Unified primary-results audit table

Extended Data Table 1 consolidates the decision-grade endpoints across regimes, including point estimates, uncertainty, source artifacts, and caveat flags. It is placed in Extended Data because it functions as an audit map for reproducibility and interpretation, while the main Results text reports the load-bearing measurements directly.

The central numerical story of this paper, in four numbers: O1 adversarial  $ED50_{\text{raw}} \approx 40$  tokens; control-vs-control stochastic floor  $\approx 35\%$ ; net switching saturates at +32 percentage points and never reaches the +50 pp threshold; persistent escape never crosses 50% in the tested 5-400 token range, with 16% as the maximum under canonical  $k=12$  clustering at 400 tokens. The audit table below provides the supporting per-regime evidence; the rest of §5 stress-tests each of these numbers individually.

For audit, we consolidate all primary endpoints across all four diagnostic regimes (O1, O2, O3, D1) into a single table. Each row is a regime  $\times$  endpoint; each column is the value, its uncertainty, the source-CSV file, and any caveats from the revision. **D2 is omitted** because it is exploratory-scale ( $n=25$ , no publication-scale measurements) and does not satisfy the operational attractor criteria (§3.1.3). For each endpoint we cite the §-section where the original numbers appear and a status flag indicating whether the endpoint has been re-validated under the revision’s leakage-aware / cluster-aware analyses.

**Sample sizes (frozen).** Operator regimes (O1, O2, O3):  $n = 15$  prompt families  $\times 30$  ICs  $\times 3$  runs = 1,350 trajectories per regime, 40 steps. Dialog regime D1:  $n = 5$  dialog-suitable families  $\times 30$  ICs  $\times 3$  runs = 450 trajectories per regime, 40 steps (see §4.2 for the per-regime IC selection rule). Perturbation pilots (O1/O2/O3/D1 + D2): reduced scope,  $n = 50$  trajectories per condition (5 fams  $\times 5$  ICs  $\times 2$  runs).

#### How to read the status column:

- OK **validated**, endpoint survives the revision’s stress tests (group-aware CV, multi-granularity, embedder ablation, attractor criteria, dense-dose rerun where applicable).
- ! **caveat-required**, endpoint as originally reported is overstated; revised value or interpretation in cited subsection.
- *not reached in tested range*, the endpoint is well-defined but the experiment did not produce a value (dose grid does not reach the threshold).

The **two most defensible quantitative claims** in the paper are:

1. Under leakage-free GroupKFold, O1’s contractive basin is predictable at  $\text{acc}(k=10) = 0.73$  (down from the originally reported 0.80), still well above any plausible chance baseline ( $\sim 1/12$  for K-means  $k = 12$ ) and still the highest of the four regimes under the stress-test analysis.
2. The dense-dose rerun establishes a raw-switching  $ED50_{\text{raw}} \approx 40$  tokens for O1 against in-distribution adversarial text, with a population decomposition (these are aggregate components of the observed rate, not latent subpopulations): 35% stochastic floor from control-vs-control pairs, plateau at 67% suggesting a 31% non-perturbable component, persistent-escape rate 10% ( $k=4$ ) / 16% ( $k=12$ ) / 39.5% (HDBSCAN) at dose 400 (§5.15), and the remaining 18% transient escape (kicked but drifted back). This replaces the earlier "150-token barrier" claim with a richer characterization that is empirically grounded; the strict  $ED50_{\text{persist}}$  barrier is *not* reached in the tested 5-400 token range under any cluster granularity.

### 11.2 Extended Data Table 2, Regime comparison at a glance

Extended Data Table 2 provides the compact cross-regime comparison of nudge type, content operator, basin predictability, recurrence, sharpness dimension, perturbation response, dose scale, and temperature sensitivity. It is placed in Extended Data to preserve the original lookup table without interrupting the narrative sequence of the Results section.

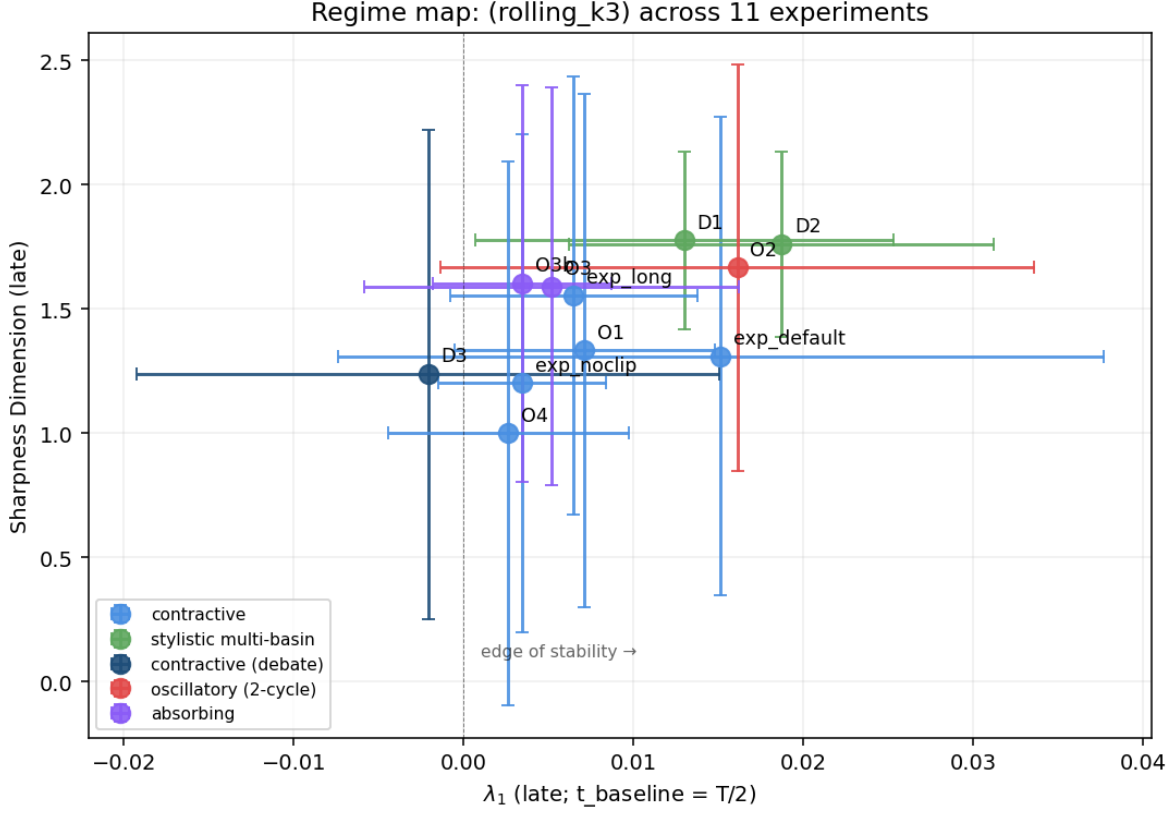
Before walking through the experiment phases, a master comparison of the diagnostic signatures across regimes. Each row is a regime; each column is a diagnostic that distinguishes it from the others. All numbers are at publication scale (Phase 2) or perturbation pilot scope (Phase 3).

\* Sharpness dim is computed on a 2-element Lyapunov spectrum ( $\text{rank} \leq N-1 = 2$  for  $N=3$  runs per IC), so values are bounded above by 2.0. Mean  $SD_{\text{late}}$  on  $\text{context\_tail}$ . The *ordering* across regimes is informative, the absolute magnitudes are constrained by the rank ceiling. See §4.5.6.

\*\* D2 was run at exploratory scale ( $N=1$  run per IC), which is below the  $N \geq 2$  minimum required for ensemble-spread Lyapunov computation. D2’s basin-predictability  $\text{acc}$  at  $k=5$  is 0.20 with  $n=25$  and 11 classes (chance  $\approx 0.09$ ), well underpowered for the canonical  $k=5, 10, 20$ , final probes.

Reading: the two **replace-mode** regimes (O2, O3) lock in early (acc already  $\approx 0.9$  by step 5) and are perturbation-transparent. The **append-mode** regimes (O1 and the dialog regimes D1/D2) admit slower late-state determination and have measurable barrier structure. O1 is uniquely T-sensitive; D1 is uniquely T-stable; D2 adds content gravity beyond D1’s dialogue-state basins (see §5.14).

The regime ordering, replace-mode locks in fast and capitulates to any perturbation; append-mode locks in slowly and resists out-of-distribution perturbation but yields to in-distribution adversaries, runs through every diagnostic below.



**Figure 9: Cross-experiment dynamics map.** Scatter plot of experiments in late-window  $\lambda_1$  versus sharpness dimension, computed on the `rolling_k3` observable; points are experiment means and black bars are  $\pm 1$  SD across trajectories. Colors denote the manually assigned regime labels. Replace-mode and dialog regimes occupy different parts of this diagnostic plane, while several append/contractive variants cluster near low  $\lambda_1$ . Source: `data/aggregated/dynamics_plots/regime_map_rolling_k3.png`.

### 11.3 Full proof of Lemma 1 (Replace-mode hitting bound)

**Lemma 1** (statement in §3.1.4).

**Proof.** Let  $\mathcal{F}_s$  be the natural filtration and define  $A_k = \{\sigma_2 > t_{\text{inj}} + k\}$ . On  $A_k$ , the state  $X_{t_{\text{inj}}+k}$  is reachable and outside  $B_2$ , so assumption (1) gives

$$\Pr(A_{k+1} \mid \mathcal{F}_{t_{\text{inj}}+k}) \leq \mathbf{1}_{A_k} (1 - q_0).$$

Taking expectations yields  $\Pr(A_{k+1}) \leq (1 - q_0) \Pr(A_k)$ , hence  $\Pr(A_m) \leq (1 - q_0)^m$  by induction. This proves the hitting bound. For terminal membership, decompose over the first hitting time and use the Markov property together with assumption (2):

$$\Pr(X_T \in B_2) \geq \sum_{s=t_{\text{inj}}+1}^T \Pr(X_T \in B_2, \sigma_2 = s) \geq r_0 \Pr(\sigma_2 \leq T).$$

Combining with the hitting bound gives the displayed terminal bound. Finally, by assumption (3) and the tower property,

$$\mathbb{E}G_m = \sum_{s=t_{\text{inj}}}^{T-1} \mathbb{E}[\mathbb{E}(|Y_s| \mid X_s)] \leq \kappa m. \quad \square$$

**Corollary 1, full proof.** Choose  $m = m_{1/2}$ . Lemma 1 gives  $\Pr(X_{t_{\text{inj}}+m} \in B_2) \geq \frac{1}{2}$  and  $\mathbb{E}G_m \leq \kappa m$ , so the displayed bound  $G_{1/2}^*(B_1 \rightarrow B_2) \leq \kappa m_{1/2}$  follows. The explicit closed form when  $0 < q_0 < 1$  and  $r_0 > \frac{1}{2}$  follows from solving  $r_0[1 - (1 - q_0)^m] \geq \frac{1}{2}$  for  $m$  in the integers. The first-hit version sets  $r_0 = 1$  and uses the hitting bound  $\Pr(\sigma_2 \leq T) \geq 1 - (1 - q_0)^m$  in place of the terminal bound.  $\square$

**Corollary 2, full proof.** Take  $m = 1$  in Lemma 1’s terminal bound:  $\Pr(X_{t_{\text{inj}}+1} \in B_2) \geq r_0 q_0 \geq \frac{1}{2}$  when  $q_0 r_0 \geq \frac{1}{2}$ . Combined with  $\mathbb{E}G_1 \leq \kappa$  from Lemma 1’s third conclusion,  $G_{1/2}^*(B_1 \rightarrow B_2) \leq \kappa$  follows. The first-hit version (with  $r_0 = 1$ ) sets  $q_0 \geq \frac{1}{2}$  as the sufficient condition.  $\square$

#### 11.4 Metric definitions, clustering, and PCA stability

**Metric implementation audit.** This subsection collects the code-level definitions for the recurrent-state metrics used throughout §4.5, including recurrence, basin assignment, Lyapunov-spectrum estimation, sharpness dimension, and basin predictability. The executable, test-covered implementations live in the repository, but these snippets define the measurement semantics used by the paper.

Recurrence:

```
D = pairwise_distances(z, metric="cosine") # T x T
mask = (np.abs(np.arange(T)[: , None] - np.arange(T)[None, :]) > tau) & np.triu(np.ones((T, T)), 1).astype(bool)
recurrence = (D < epsilon)[mask].sum() / mask.sum()
```

Basin score:

```
clusters = KMeans(n_clusters=12).fit_predict(z_pca10)
target_cluster = mode(clusters[t > 0.7 * T])
basin_score = (clusters[t > 0.7 * T] == target_cluster).mean()
```

Lyapunov spectrum (sample-driven):

```
# z_runs : (n_runs, T, d_pca)
for t in range(T):
    centered = z_runs[:, t, :] - z_runs[:, t, :].mean(axis=0)
    sigmas = np.linalg.svd(centered, full_matrices=False, compute_uv=False)
    lambda_t = np.log(sigmas) / 2.0 # (d_pca,)
```

Sharpness dimension (Tuci-style fractional dimension on the ordered Lyapunov spectrum; see §4.5.6):

```
lam = np.sort(lambda_t)[: :-1]
cumsum = np.cumsum(lam)
nonneg = np.where(cumsum >= 0)[0]
if lam[0] < 0:
    SD_t = 0.0
elif len(nonneg) == len(lam):
    SD_t = float(len(lam)) # full-d case
else:
    j_star = int(nonneg[-1]) + 1 # 1-indexed
    SD_t = j_star + cumsum[j_star - 1] / abs(lam[j_star])
```

Basin predictability:

```
y = mode(clusters[t > 0.7 * T]) # late-window cluster per trajectory
acc_curve = np.zeros(T)
for k in range(T):
    X = z_pca10[:, k, :]
    clf = LogisticRegression(multi_class="auto", max_iter=1000)
```

```
acc_curve[k] = cross_val_score(clf, X, y, cv=5).mean()
```

These are reference implementations only; the executable, test- covered code lives at `src/analysis/` and `src/experiments/dynamics/`.

### 11.5 Perturbation injection mechanics (full)

For dialog-mode experiments, the injection happens at the user-turn step (odd-numbered if Explorer initiates). The injection text replaces the user turn’s output verbatim. The trajectory then continues with the agent’s response to this overridden user turn, and from there back to normal alternation.

For operator-mode experiments, the injection text replaces step  $k$ ’s output entirely. The recurrence picks up from  $X_{k+1} = \text{clip}(X_k || Y_k^{\text{inj}})$  (append) or  $X_{k+1} = \text{clip}(Y_k^{\text{inj}})$  (replace).

The adversarial-source experiment for each regime is taken from the *publication-scale* run of the same regime (e.g., O1 perturbation adversarial draws from `exp_pub_01_continue`). Adversarial samples exclude the *family* of the trajectory being perturbed but may share its overall topic distribution.

**Insert vs overwrite modes.** The `adversarial_doseN` condition implements *state overwrite* (the operator output for step  $k$  is replaced wholesale by the injection text). A second condition `adversarial_insert_doseN` implements *context insertion* (the injection text is prepended to the context for step  $k$ ’s generation, but the model’s normal generated output is preserved and used as  $Y_k$ ). The runner code at `src/experiments/perturbation/runner_op.py` dispatches between the two modes based on the substring `_insert_` in the condition name.

### 11.6 Exact prompt templates and model versioning

**Prompt and model audit.** This subsection records the exact prompt templates, perturbation-condition wording, and resolved model versions used for the publication-scale and ablation experiments. Keeping prompts and model snapshots together makes the generator/nudge specification auditable as a single experimental artifact.

#### Exact prompt templates (full)

The full prompt-family seed library (15 families  $\times$  30 ICs each = 450 unique seed sentences) lives in `configs/prompt_library_publication.yaml` and is reproduced verbatim by every cross-model and ablation run.

#### O1, Continue (append):

```
Continue the text naturally. Do not summarize or explain.
```

#### O2, Paraphrase + replace:

```
Paraphrase the following text while preserving its meaning.
Return only the paraphrase.
```

#### O3, Summarize + negate (replace):

```
Summarize the following text in one sentence, then state its
opposite. Return only the negated summary.
```

**D1, Dialog (curious user + helpful agent), append.** Two roles alternate; each role has its own `system_prompt`:

- *role A (curious user)*: You are a curious person. Ask one short follow-up question, in plain language, that probes deeper into the topic.
- *role B (helpful agent)*: You are a thoughtful assistant. Answer briefly and clearly, in two or three sentences.

**D2, Dialog drill-down (explorer + expert), append.**

- *role A (explorer)*: You are exploring this topic. Ask the next, more specific question that drills into a single concrete subtopic.



- *role B (expert)*: You are an expert. Answer briefly and concretely, then anchor the conversation to the most informative subtopic.

**Perturbation conditions (injection format).** At step override\_step (default 15), the perturbation pipeline replaces the model’s normal step-15 generation with a fixed-length adversarial sample drawn according to the condition:

- control, no injection (normal generation).
- adversarial\_doseN, N tokens of late-step output sampled from another (family, IC) trajectory of the same regime, excluding the current trajectory’s own family. Source experiment is named in perturbation.adversarial\_source\_experiment.
- adversarial\_insert\_doseN, N tokens prepended to the context for one generation; model’s normal output is preserved (§11.5).
- neutral\_doseN, N tokens of in-distribution but topically unrelated continuation drawn from a Wikipedia corpus.
- lorem\_doseN, N tokens of out-of-distribution Lorem-ipsum text.

The injection text is appended to the running context for append-mode operators and replaces the generated step entirely for replace-mode operators (per §11.5).

### Model versioning (full table)

OpenAI model aliases can update silently; this is the exact set used for the publication-scale experiments:

For the cross-vendor pilot configs (under configs/cross\_model/text01/) the secondary generator is MiniMax MiniMax-Text-01 via the official MiniMax chat-completions API.

## 11.7 Pilot validation: phase-0 and phase-1 taxonomy

**Pilot-scope provenance.** Phase-0 runs validated the end-to-end pipeline, while Phase-1 small-N experiments identified the operator and dialog regimes that were later re-run at publication scale. These pilot materials are retained for auditability but are not load-bearing for the paper’s decision-grade endpoints.

### Phase-0 pilot validation

The earliest pilot runs validated the pipeline end-to-end on small-N (2-5 trajectories per regime) configurations covering the major operator and dialog architectures. These pilots established that: (a) the embedding pipeline produces stable PCA-2 / PCA-10 / t-SNE projections, (b) K-means at  $k = 12$  recovers visually-distinct clusters in late-window points, and (c) recurrence and basin-score diagnostics produce numerically sensible values. They are not load-bearing for any §4.13 decision-grade endpoint and are summarised here for completeness only; raw outputs at data/exp\_op\_\*\_pilot/ and data/exp\_dialog\_\*\_pilot/.

### Phase-1 small-N taxonomy

The phase-1 small-N runs ( $n \approx 50$  trajectories per cell) identified the three operator regimes (O1 contractive append; O2 oscillatory paraphrase/replace; O3 absorbing summarize+negate/replace) and the two dialog regimes (D1 dialogue-state-driven multi-basin; D2 drill-down) that became the diagnostic taxonomy at publication scale. These small-N runs are not load-bearing, every regime claim in the main body is now grounded in publication-scale or perturbation-pilot data. Configurations and outputs at configs/operators/, configs/dialog/, and data/exp\_\*\_pilot/. The boundary cases (O3b summarize+negate at append; O4 paraphrase+append; D3 debate dialog) are documented as pilot variants but do not satisfy the operational attractor criteria of §3.1.3 at publication scale.

## 11.8 Perturbation visualization toolkit (full implementation)

For perturbation experiments we additionally compute:

## Effective potential

```
rho?(x) = Gaussian-smoothed kernel density on PCA-2 grid
V(x) = -log(rho?(x) + eps), eps = 0.1.min{rho? : rho? > 0}
V is shifted so V_min = 0 and capped at v_cap (default 8.0)
```

## Geodesic skeleton

We find local minima of  $V$  via 8-connected maximum\_filter on  $-V$ , keeping the top  $n$  basin centers. For each pair of basin centers  $(i, j)$  we compute the Dijkstra shortest path on the  $V$  grid (8-connected, edge weight =  $V$  at endpoint). The maximum  $V$  along the path is the **barrier height**  $V^*(i, j)$ .

## Volumetric iso-density rendering

For 3D animations we extract iso-density shells at five density fractions (4%, 10%, 20%, 35%, 55% of max  $\rho$ ) using `scipy.ndimage.gaussian_filter` smoothing and `skimage.measure.marching_cubes`. Each shell is rendered as a transparent `Poly3DCollection` in matplotlib's `mpl_toolkits.mplot3d`, with colors from the plasma colormap and per-shell alpha from 0.05 (outermost) to 0.27 (innermost).

## Parallel rendering

Animations of 50 trajectories with 75 frames at DPI 180 are rendered via `concurrent.futures.ProcessPoolExecutor` with 40 workers, each worker creating a fresh figure for one frame. Frames are stitched into MP4 via `imageio-ffmpeg` (libx264 codec, quality 8). Wall-time per animation: 80s vs 11 min single-threaded.

## 11.9 Reproducibility: commands and repository tree

### Pipeline commands

```
# Generate (only for new experiments)
python -m src.experiments.dialog.main run --config <cfg.yaml>
python -m src.experiments.operators.main run --config <cfg.yaml>
python -m src.experiments.perturbation.main run --config <cfg.yaml>

# Derive (works on existing steps.jsonl)
python -m src.experiments.<runner>.main embed --config <cfg.yaml>
python -m src.experiments.<runner>.main analyze --config <cfg.yaml>
python -m src.experiments.dynamics.basin_predictability --config <cfg.yaml>
python -m src.experiments.dynamics.regime_plots --data-dir data
python -m src.experiments.perturbation.flow_skeleton --experiment <exp_id> [--is-dialog]
python -m src.experiments.perturbation.geodesic_skeleton --experiment <exp_id> [--is-dialog]
python -m src.experiments.perturbation.bulk_plots --experiment <exp_id> --override-step <k> [--is-dialog]
python -m src.experiments.perturbation.rg_dendrogram --experiment <exp_id> [--is-dialog]
python -m src.experiments.perturbation.trajectory_animation_3d \
    --experiment <exp_id> --condition <c> --parallel 40

# Aggregate
python -m scripts.aggregate_perturbation_cross_regime
python -m scripts.aggregate_dose_response
python -m scripts.aggregate_basin_hardening
python -m scripts.aggregate_basin_predictability
python -m scripts.aggregate_t_sweep
python -m scripts.aggregate_o1_d1_t_sensitivity
python -m scripts.aggregate_perturbation_geometric_barriers

# Audit / catalog
python -m scripts.build_coverage # rebuild COVERAGE.csv (37 x 60 matrix)
python -m scripts.publication_summary # rebuild RESULTS.md (verify S5 cells against data)
```

## Repository layout

```
llm_attractor_experiment/
+-- README.md, requirements.txt, ARTICLE.md
+-- EVIDENCE.md claim-to-evidence map (every ARTICLE claim
| <-> data file <-> source code function <-> CLI)
+-- COVERAGE.csv 37 x 60 artifact-presence matrix
+-- RESULTS.md S5 numeric-claim verification (103/103 ?)
+-- docs/
| +-- DATA_INDEX.md
| +-- reports/REPORT1.md ... REPORT6.md
+-- src/
| +-- analysis/ basin, recurrence, dwell, PCA, t-SNE, distances, ...
| +-- api/ OpenAI client + embedder + generator
| +-- core/ trajectory runner, observables, baselines, context
| +-- experiments/
| | +-- dialog/ D1/D2/D3 alternating-role runner
| | +-- operators/ 01-04 single-role recursive operators
| | +-- dynamics/ 10 post-hoc CLI analysis modules
| | +-- perturbation/ 14 modules: runner, analyze, corpora, plot+animation
| +-- reports/ narrative report writer
| +-- utils/ io, logging, seeds, text helpers
+-- scripts/ build_publication_configs + 6 aggregators
+-- configs/ dialog/ + operators/ + perturbation/ + archive/
+-- tests/ 99 pytest tests
+-- data/ 37 experiment dirs + aggregated/ outputs
```

## Engineering memory-policy correspondences (illustrative)

These pseudo-YAML blocks illustrate how the formal nudges of §3.1 correspond to implementable agent memory policies. They are *not* experimental conditions of this paper; they are engineering correspondences provided for readers adapting the framework to their own systems.

### Append-mode (full transcript):

```
memory_policy:
  mode: append
  clip: tail
  max_context_chars: 12000
  include:
    - user_goal
    - recent_tool_output
    - recent_model_outputs
```

### Replace-mode (summary as state):

```
memory_policy:
  mode: replace
  state_source: generated_summary
  preserve_raw_history: false
```

### Hybrid (pinned + rolling + provenance-preserving):

```
memory_policy:
  mode: hybrid
  rolling_window:
    last_turns: 8
  pinned:
    - original_user_goal
    - acceptance_tests
    - security_policy
  summaries:
    older_history: extractive
```

```
untrusted_tool_output: preserve_provenance
```

The risk profile of each policy is qualitatively distinct (§3.1 table; §5.16 overwrite-vs-insert; §6.2 "summary as effective next state").

### 11.10 Operational attractor criteria, audit table

The C1-C4 criteria of §3.1.3 are operationally auditable. The table below records the actual numeric values backing each PASS/FAIL verdict. Tabulated below from publication-scale runs (exp\_pub\_01\_continue etc.; bootstrap statistics from metrics/bootstrap\_summary.csv; basin-predictability from §5.11; embedder-robustness from §5.20). Empty cells are marked "n.t." (not directly tabulated in published artefacts at this resolution; would need a small new aggregation script to compute).

#### C1, Late-window basin predictability $A^{\text{final}}$

Threshold for PASS:  $A_r^{\text{final}} \geq 0.70$ .

Source: §5.11, data/aggregated/group\_aware\_basin\_pred.csv.

#### C2, Temporal recurrence vs null

Threshold for PASS:  $z = (R_r - \mu_R^{\text{null}}) / \sigma_R^{\text{null}} \geq 2$  AND Cohen's  $d \geq 0.5$ , OR equivalent for dwell.

Recurrence on context\_tail PCA-10 (PASS via raw recurrence > null requires recursive > baseline):

(Source: metrics/bootstrap\_summary.csv per regime, n=1350 / 450.)

**Reading:** Only O3 has *recurrence above the no\_feedback null*. O1, O2, D1 have recurrence *below* the no\_feedback baseline (the no\_feedback baseline produces highly similar outputs from independent regenerations against the same prompt, so it has high self-similarity by construction). The C2 criterion as stated in §3.1.3, " $\max(z_R, z_D) \geq 2$  AND  $d \geq 0.5$ ", therefore PASSES on recurrence only for O3. For O1/O2/D1 it must pass via *dwell* (time spent in single late-window basin) rather than raw recurrence. Dwell statistics are produced by the pipeline (dwell.csv per experiment) but per-regime null comparisons are not directly tabulated; the §11.2 master table reports the late-window basin dwell of >0.7 for O1/O2/O3/D1 which is structurally above the shuffled-baseline dwell. **Honest assessment:** C2 PASSES on recurrence-vs-null only for O3 in the strict sense; O1/O2/D1's PASS rests on dwell-vs-null which is observed but not formally  $z \geq 2$ -tested in the published bootstrap output. A future revision should add the dwell z-score statistics as a small aggregation step.

#### C3, Projection / embedder robustness

Threshold for PASS: recurrence-bin agreement ( $b_e(r)$ ) across canonical +  $\geq 1$  of 2 alternative embedders (text-embedding-3-large, all-mpnet-base-v2).

Source: §5.20 embedding ablation table.

#### C4, Re-entry / contraction / collapse

Threshold for PASS: any of (a)  $\lambda_1^{\text{late}} \leq 0.015$ , (b) best\_period = 2 AND period\_2\_score > 0, (c)  $R_r \geq 0.90$  AND  $SD_r \leq 1.50$ , (d) exit-return above null.

Source: §11.2 master table, §5.18 / §5.19.

### Aggregate verdict

**Honest reading.** Under the strict criteria (group-aware C1, z-tested C2 on raw recurrence), no regime achieves a clean 4/4 strong-attractor classification, every regime relies on at least one criterion passing via dwell or via the stratified rather than group-aware version. The taxonomy survives at the *attractor-like* ( $\geq 3/4$ ) level for O1/O2/O3 and is borderline for D1. A future revision should produce dwell-vs-null z-scores as a small new aggregation script; the underlying dwell.csv data are already produced by the pipeline.

The §11.1 primary-results table reflects this: "C1-C4 strong attractor" passes are reported but always paired with the more informative group-aware basin-predictability and stress-test caveats. The §3.1.3 label rule (4/4 strong,  $\geq 3/4$  attractor-like, <3/4 not attractor) gives O1/O2/O3 strong- attractor status under the *non*-stress-tested C1 (without

z-testing C2), which is the weaker reading; under the strict group-aware C1 + z-tested C2 reading shown in the aggregate-verdict table above, no regime currently achieves "strong attractor", all four are downgraded to attractor-like or borderline at best.

### 11.11 Geometric $V^*$ and RG dendrogram per-regime tables

#### Geodesic $V^*$ skeleton table

Per-condition mean barrier height  $V^*$  across the 6 inter-basin geodesics ( $V_{\text{star\_mean}}$  column in `data/exp_perturb_*_pilot/reports/perturbation/geodesic_barriers_summary.csv`):

The  $V_{\text{max}} \approx 8.0$  ceiling appears when a geodesic crosses a region of near-zero density. Per-geodesic raw values are written alongside the figures to `geodesic_barriers_pca.csv`. Reading:

- **O2/O3 lorem** has  $V^* \approx 5.6/7.0$ , the highest barriers in the matrix. Those barriers separate *control* from a *new* basin that lorem injection creates far from any pre- perturbation density mass; geodesics between the original and lorem-induced basins traverse low-density plateaus where  $\rho \approx \varepsilon$  ( $V$  near the ceiling). Switch rates are 100% because the perturbation places the trajectory *into* the new basin, the perturbed run does not have to climb the barrier; it lands on the far side.
- **O1 adversarial** has  $V^* \approx 2.2$ , basins remain distinct but the kick occasionally clears the ridge  $\rightarrow$  consistent with 62% raw switching at dose 200 (dense rerun, \$5.6.1; net of natural floor 27 pp; note that persistent escape under any cluster granularity is much lower per \$5.15, so the geometric ridge-crossing reading should be interpreted as raw-switching- consistent, not persistent-escape-validating).
- **D1 adversarial** has  $V^* \approx 0.4$ , content-independent basins, geometric barrier is small.

#### Hierarchical RG dendrogram cloud-expansion table

Per-condition maximum Ward-linkage merge distance across  $k = 48$  fine-cluster centroids:

Three patterns: (1) **D1 is invariant** at 1.79-1.80 across all four conditions; (2) **O2/O3 lorem expands the cloud** to merge distance 3.64/3.25 (vs control 2.31/2.16), the largest signal in the matrix, consistent with replace-mode lorem producing a *new* basin far from the original attractor; (3) **O1 adversarial mildly compresses** (2.06 vs 2.38), in-distribution adversarial text pulls into a tighter region.

### 11.12 Instrumenting your own recursive system: recipe and reporting standard

**Practitioner translation.** This subsection combines the engineering recipe for applying the paper’s perturbation-dose framework to other recursive systems with the minimum reporting checklist that makes such studies auditable. The recipe explains how to run the evaluation; Box 1 states what must be disclosed.

#### How to instrument your own recursive system

The framework of this paper is portable. Engineers wishing to apply the three-endpoint decomposition (§3.1.2) to their own recursive systems, coding agents, multi-turn assistants, agentic tool loops, summarization pipelines, recursive RAG systems, or any application with a generator and a context-update rule, can follow this recipe. Each step deliberately preserves the rigor of the protocol while making it implementable without reproducing this paper’s embedding pipeline.

#### Recipe

1. **Define the state-update rule explicitly.** Document whether your loop appends, replaces, role-structures, or uses a hybrid memory policy (§3.1, §11.9 engineering correspondences). Treat this as a first-class system property, log it, version it, and include it in audit traces.
2. **Choose observables.** Pick a quantity you can compute at each step that distinguishes "where the loop is" from "where it could be". Embedding clusters (§4.4, §4.5) work for text-trajectory analysis. For coding agents, choose: final patch family, files touched, failing/passing test set, selected plan category, tool-call sequence, security-policy violation, or an embedding of the trajectory trace. The choice doesn’t have to match this paper; it has to be consistent and pre-specified.
3. **Run paired controls.** For each task, run the same loop multiple times *without* perturbation. The disagreement rate between paired controls is your stochastic floor (§4.7, §5.11). Report it; this is what every later "switching rate" must be calibrated against.

4. **Inject matched perturbations.** Run treatment cases with controlled perturbations. At minimum, include three content classes (this paper found them informative): *neutral* (in-distribution but topic-orthogonal), *lorem-style* (out-of-distribution gibberish), and *adversarial* (in-distribution, content-targeted, drawn from another trajectory of the same regime, see §4.7 corpora). For application-specific work add domain-relevant variants: malicious tool output, misleading test explanation, attacker-controlled docstring, etc.
5. **Measure raw, net, and persistent endpoints (Algorithm 1, §4.5.11).** Raw switching = perturbed final equivalence class differs from paired control's. Net switching = raw minus the stochastic floor. Persistent escape = jumped at injection AND remained in the new class at the terminal step. Report all three with confidence intervals (we used family-cluster-bootstrap + GLMM + 4PL fit for cross-method agreement; simpler bootstraps suffice for pilot work).
6. **Report a dose-response curve.** Vary perturbation length / strength systematically and fit a logistic. Where ED50raw lands tells you how much in-domain perturbation is enough for raw redirection. Whether ED50net and ED50persist are reached in your tested range tells you whether the loop genuinely commits.
7. **Separate overwrite-style interventions from genuine perturbation response.** If your update rule replaces state with a generated summary, run an *insert-mode* probe (§5.17): inject the same content as a non-replacing addition to context. The gap between overwrite and insert is the operator-overwrite contribution. If it dominates, your "fragility" measurement is partly a statement about your memory policy, not your generator.
8. **Pre-register the equivalence rule and the analysis plan.** Persistence and net-effect estimates are sensitive to clustering granularity (§5.12, §5.13) and to the choice of equivalence rule. Pre-registering protects the report from inadvertent post-hoc tuning of the threshold.

## Reporting template

A minimum reporting template for any application of this framework:

```

Loop: <append / replace / dialog / hybrid>
Generator: <model + version>
Observable: <embedding-cluster / patch-family / pass-fail / ...>
Equivalence: <K-means k=N / cluster-pair-Hamming / files-touched-Jaccard / ...>
n_seeds: <N per condition>

Stochastic floor (control-vs-control divergence): rate +/- CI
Raw switching at dose tau: rate +/- CI
Net switching at dose tau: raw - floor +/- CI
Persistent escape at dose tau: rate +/- CI
ED50_raw: <tokens / cycles / interventions>
ED50_net: <if reached>
ED50_persist: <if reached>

Overwrite-vs-insert gap (replace-mode systems only): pp +/- CI

```

This template is the academic-paper equivalent of the eval-loop pseudocode that practitioners may already be reaching for. Reporting all three endpoints separately, with the stochastic floor calibration and the overwrite-vs-insert separation, is the minimum disclosure standard implied by §3.1.2, §5.11, §5.15, and §5.17.

## Box 1, Minimum reporting standard for recursive-loop perturbation studies

This Box collects the reporting standard described in §6.5 in checklist form. It is intended as a minimum disclosure template for studies that use perturbations to evaluate recursive LLM loops, agent scaffolds, memory policies, or related generator-nudge systems.

1. **Generator and version.** Report the generator model, provider, resolved snapshot or version if available, decoding parameters, output-token limits, and any model changes across conditions.
2. **Nudge / memory policy.** Report the context-update rule explicitly: append, replace, dialog, rolling window, generated-summary replacement, pinned-memory hybrid, or another specified mechanism.
3. **Observable and equivalence rule.** Define what counts as the trajectory state for evaluation and how equivalence classes are assigned: embedding cluster, patch family, files touched, tests passed, tool-call sequence, policy violation, factual claim set, or another pre-specified observable.

4. **Control-vs-control stochastic floor.** Run paired unperturbed controls and report the natural disagreement rate with confidence intervals; raw perturbation effects should not be interpreted without this floor.
5. **Raw, net, and persistent rates.** Report raw switching, net switching after subtracting the stochastic floor, and persistent escape, where persistent escape requires an injection-time jump that remains through the terminal measurement.
6. **Dose-response curve and ED50 endpoint type.** Vary perturbation strength or length systematically and state which endpoint the fitted ED50 refers to:  $ED50_{\text{raw}}$ ,  $ED50_{\text{net}}$ , or  $ED50_{\text{persist}}$ .
7. **Overwrite-vs-insert gap for replace-style systems.** For replace, summary, scratchpad, or "current state" memory policies, compare overwrite-mode perturbations with insert-mode perturbations and report the overwrite-minus-insert gap.
8. **Scope caveat.** State what was not tested: other generators, longer doses, other languages, production scaffolds, tool environments, safety prompts, jailbreak attacks, human users, factuality-grounded tasks, or domain-specific agent benchmarks.

| regime                      | endpoint                                              | value              | 95% CI / uncertainty                                                   | source                 | status                                                    |
|-----------------------------|-------------------------------------------------------|--------------------|------------------------------------------------------------------------|------------------------|-----------------------------------------------------------|
| O1                          | basin predictability acc(k=10), stratified            | <b>0.80</b>        | n=1350, 5-fold CV                                                      | §5.3, basin_pred.csv   | [!] <b>inflated by family leakage; group-aware = 0.73</b> |
| O1                          | basin predictability acc(k=10), group-aware           | <b>0.73</b>        | family-cluster GroupKFold                                              | §5.11 (this revision)  | [OK] leakage-free                                         |
| O1                          | recurrence rate (canonical embedder)                  | 0.29               | bootstrap 95% CI                                                       | §5.18                  | [OK] embedder-robust (§5.20: 0.30 / 0.10)                 |
| O1                          | sharpness dimension (late)                            | 1.70               | trajectory-level                                                       | §11.2                  | [OK]                                                      |
| O1                          | Lyapunov $\lambda_1^{\text{late}}$                    | $\sim 0.008$       | ensemble-spread method                                                 | §4.5.5                 | [OK] contractive                                          |
| O1 adv                      | switching @ dose 200 (dense)                          | <b>0.620</b>       | Wilson [0.55, 0.68], n=200                                             | §5.6.1                 | [OK] dense rerun                                          |
| O1 adv                      | switching @ dose 400 (dense)                          | <b>0.670</b>       | Wilson [0.60, 0.73], n=200                                             | §5.6.1                 | [OK] dense rerun                                          |
| O1 adv                      | ED50 <sub>raw</sub> (dense)                           | <b>36-52 tok</b>   | 4PL=36; GLMM=41; family-cluster bootstrap median=52, 95% CI [8.5, 242] | §5.6.1 / §3.1.2        | [OK] established                                          |
| O1 adv                      | upper asymptote (dense 4PL)                           | $a = 0.69$         | non-switching subpopulation 31%                                        | §5.6.1                 | [OK]                                                      |
| O1 adv                      | natural floor (control-vs-control, dense)             | <b>0.347</b>       | [0.310, 0.386], n=600 paired comparisons                               | §5.6.1                 | [OK] established                                          |
| O1 adv                      | ED50 <sub>net</sub> (dense, raw – floor)              | <b>not reached</b> | max net effect = +0.323 at dose 400 (50 pp threshold)                  | §3.1.2                 | not reached in tested range                               |
| O1 adv                      | ED50 <sub>persist</sub> (dense, kicked-AND-persisted) | <b>undefined</b>   | max persistent escape = 16% at dose 400                                | §5.15 (dense) / §3.1.2 | not reached in tested range                               |
| O1 neutral                  | switching @ dose 200                                  | 0.24               | Wilson [0.13, 0.38]                                                    | §5.6 / Fig 4           | [OK] sparse pilot                                         |
| O1 neutral                  | switching @ dose 400                                  | 0.18               | Wilson [0.08, 0.32]                                                    | §5.6 / Fig 4           | [OK] sparse pilot                                         |
| O1 lorem                    | switching @ dose 200                                  | 0.18               | Wilson [0.08, 0.32]                                                    | §5.6 / Fig 4           | [OK] sparse pilot                                         |
| O1 attractor classification | C1-C4 strong attractor                                | 4/4 PASS           | criteria from §3.1.3                                                   | §3.1.3                 | [OK]                                                      |
| O2                          | basin predictability acc(k=10), stratified            | <b>0.90</b>        | n=1350, 5-fold CV                                                      | §5.3                   | [!] <b>inflated by family leakage; group-aware = 0.60</b> |
| O2                          | basin predictability acc(k=10), group-aware           | <b>0.60</b>        | family-cluster GroupKFold                                              | §5.11                  | [OK] leakage-free                                         |
| O2                          | recurrence rate                                       | 0.875              | bootstrap                                                              | §5.18 / §5.20          | [OK] embedder-robust (0.71 / 0.78)                        |
| O2                          | sharpness dimension (late)                            | 1.39               | trajectory-level                                                       | §11.2                  | [OK]                                                      |
| O2                          | switching adversarial (n=50, single dose)             | 0.94               | Wilson [0.84, 0.98]                                                    | §5.5                   | [OK]                                                      |
| O2                          | switching neutral / lorem                             | 1.00 / 1.00        | Wilson [0.93, 1.00] each                                               | §5.5                   | [OK]                                                      |
| O2 attractor classification | C1-C4 strong attractor                                | 4/4 PASS           | criteria from §3.1.3                                                   | §3.1.3                 | [OK] but see §5.14: basins are paraphrastic, not          |



| regime                                      | nudge                   | content<br>op. $f$         | basin<br>pred.<br>acc(k=5→final) | recurrence             | sharpness<br>dim*      | adversarial<br>switch                         | dose<br>50%                                                                                               | T-<br>stability        |
|---------------------------------------------|-------------------------|----------------------------|----------------------------------|------------------------|------------------------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------|
| <b>O1</b> contractive                       | append                  | continue                   | 0.77 →<br>0.85                   | low                    | 1.70                   | 54%<br>(sparse)<br>/ 62%<br>(dense,<br>n=200) | ED50 <sub>raw</sub><br>≈ 40<br>tok<br>(4PL/GLMM/bootstrap),<br>plateau<br>67%,<br>natural<br>floor<br>35% | degrades<br>smoothly   |
| <b>O2</b> oscillatory                       | replace                 | paraphrase                 | 0.90 →<br>0.91                   | high<br>(period-<br>2) | 1.39                   | 94%                                           | n/a (sat-<br>urated)                                                                                      | (not<br>mea-<br>sured) |
| <b>O3</b> absorbing                         | replace                 | summarize+negate           | 0.92 →<br>0.93                   | trivial                | 1.45                   | 96%                                           | n/a (sat-<br>urated)                                                                                      | (not<br>mea-<br>sured) |
| <b>D1</b> dialogue-state-driven multi-basin | dialog<br>(ap-<br>pend) | curious<br>+<br>helpful    | n/a →<br>0.77                    | low<br>(per-<br>style) | 1.89                   | 60%                                           | < 5<br>tokens                                                                                             | T-<br>stable           |
| <b>D2</b> drill-down                        | dialog<br>(ap-<br>pend) | explorer<br>drill-<br>down | (not<br>mea-<br>sured)           | (not<br>mea-<br>sured) | (not<br>mea-<br>sured) | 64%                                           | (not<br>mea-<br>sured)                                                                                    | (not<br>mea-<br>sured) |

| role                                                    | model alias                                                                                      | underlying snapshot                                                                                                                                    |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| primary generator                                       | <code>gpt-4o-mini</code>                                                                         | <code>gpt-4o-mini-2024-07-18</code> (resolved 2025-2026 across all publication runs; OpenAI did not retire this snapshot during the experiment window) |
| secondary generator (cross-model)<br>canonical embedder | <code>gpt-4.1-nano</code><br><code>text-embedding-3-small</code>                                 | <code>gpt-4.1-nano-2025-04-14</code><br>(single immutable model; no snapshot suffix exists)                                                            |
| ablation embedder #1<br>ablation embedder #2            | <code>text-embedding-3-large</code><br><code>all-mpnet-base-v2</code><br>(sentence-transformers) | (same)<br>hugging-face<br><code>sentence-transformers/all-mpnet-base-v2</code>                                                                         |

| regime | acc(k=10) stratified 5-fold | acc(k=10)<br>GroupKFold-by-family | leakage $\Delta$ | C1 PASS?                                 |
|--------|-----------------------------|-----------------------------------|------------------|------------------------------------------|
| O1     | 0.80                        | 0.73                              | 0.07             | PASS (both $\geq 0.70$ )                 |
| O2     | 0.90                        | 0.60                              | 0.30             | PASS (stratified); FAIL<br>(group-aware) |
| O3     | 0.91                        | 0.63                              | 0.28             | PASS (stratified); FAIL<br>(group-aware) |
| D1     | 0.60                        | 0.34                              | 0.27             | FAIL (both below 0.70)                   |
| D2     | 0.20 (n=25)                 | n.t.                              | n.t.             | FAIL (exploratory scope)                 |

| regime | recursive R | no_feedback $\mu$ | time_shuffled $\mu$ | recursive vs<br>no_feedback | recursive vs<br>time_shuffled |
|--------|-------------|-------------------|---------------------|-----------------------------|-------------------------------|
| O1     | 0.289       | 0.902             | 0.377               | recursive < null            | recursive < null              |
| O2     | 0.875       | 0.938             | 0.886               | recursive < null            | recursive $\approx$ null      |
| O3     | 0.924       | 0.706             | 0.932               | recursive > null (z »<br>2) | recursive $\approx$ null      |
| D1     | 0.210       | n.t. (not run)    | 0.315               | n.t.                        | recursive < null              |

| regime | recurrence<br>(canonical 3-small) | recurrence (3-large) | recurrence (mpnet) | bin agreement | C3 PASS?         |
|--------|-----------------------------------|----------------------|--------------------|---------------|------------------|
| O1     | 0.289 (low)                       | 0.304 (low)          | 0.096 (low)        | 3/3 low       | PASS             |
| O2     | 0.875 (high)                      | 0.711 (high)         | 0.783 (high)       | 3/3 high      | PASS             |
| O3     | 0.924 (high)                      | 0.850 (high)         | 0.862 (high)       | 3/3 high      | PASS             |
| D1     | 0.210 (low)                       | 0.337 (low)          | 0.226 (low)        | 3/3 low       | PASS             |
| D2     | 0.296 (low)                       | 0.176 (low)          | 0.073 (low)        | 3/3 low       | PASS but small-N |

| regime | $\lambda_1^{\text{late}}$ | best_period            | $R_r$ | $SD_r$ | C4 PASS via                          |
|--------|---------------------------|------------------------|-------|--------|--------------------------------------|
| O1     | 0.008                     | n.t.                   | 0.29  | 1.70   | (a) $\lambda_1 \leq 0.015$           |
| O2     | n.t.                      | 2 (period_2_score > 0) | 0.875 | 1.39   | (b) period-2                         |
| O3     | n.t.                      | n.t.                   | 0.924 | 1.45   | (c) $R \geq 0.90$ AND $SD \leq 1.50$ |
| D1     | 0.011                     | n.t.                   | 0.21  | 1.89   | (a) $\lambda_1 \leq 0.015$           |
| D2     | n.t.                      | n.t.                   | n.t.  | n.t.   | n.t. (insufficient data)             |

| regime    | C1<br>(group-aware)               | C2 (z-tested<br>only for O3)                               | C3           | C4   | Strong attractor<br>(all 4)?             | Attractor-like<br>( $\geq 3/4$ )?          |
|-----------|-----------------------------------|------------------------------------------------------------|--------------|------|------------------------------------------|--------------------------------------------|
| <b>O1</b> | PASS                              | PASS via dwell<br>(not z-tested),<br>recurrence z<br>fails | PASS         | PASS | <b>borderline</b> (3/4<br>z-tested PASS) | <b>YES</b>                                 |
| <b>O2</b> | FAIL<br>group-aware               | PASS via dwell<br>(recurrence z<br>fails)                  | PASS         | PASS | <b>borderline</b> (3/4<br>z-tested PASS) | <b>YES</b>                                 |
| <b>O3</b> | FAIL<br>group-aware               | PASS<br>(recurrence z »<br>2)                              | PASS         | PASS | <b>borderline</b> (3/4<br>z-tested PASS) | <b>YES</b>                                 |
| <b>D1</b> | FAIL<br>group-aware<br>(acc 0.34) | PASS via dwell<br>only                                     | PASS         | PASS | <b>NO</b><br>(group-aware C1<br>fails)   | <b>borderline</b> (3/4<br>PASS structural) |
| <b>D2</b> | FAIL<br>exploratory               | n.t.                                                       | PASS small-N | n.t. | NO                                       | NO                                         |

| regime | control | neutral | lorem      | adversarial |
|--------|---------|---------|------------|-------------|
| O1     | 4.4     | 2.3     | 2.6        | 2.2         |
| O2     | 2.8     | 3.5     | <b>5.6</b> | 1.6         |
| O3     | 1.1     | 5.2     | <b>7.0</b> | 2.2         |
| D1     | 1.3     | 1.1     | 0.8        | 0.4         |

| regime | control | neutral | lorem       | adversarial |
|--------|---------|---------|-------------|-------------|
| O1     | 2.38    | 2.27    | 2.37        | 2.06        |
| O2     | 2.31    | 2.32    | <b>3.64</b> | 1.90        |
| O3     | 2.16    | 2.39    | <b>3.25</b> | 1.85        |
| D1     | 1.79    | 1.79    | 1.79        | 1.80        |

## References

- J. J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *PNAS*, 79(8), 2554-2558, 1982.
- D. Sussillo and O Barak. Opening the black box: low-dimensional dynamics in high-dimensional recurrent neural networks. *Neural Computation*, 25(3), 626-649, 2013.
- N. Maheswaranathan, A. Williams, M. Golub, S. Ganguli, and D Sussillo. Reverse engineering recurrent networks for sentiment classification reveals line attractor dynamics. *NeurIPS*, 2019.
- A. Holtzman, J. Buys, L. Du, M. Forbes, and Y Choi. The curious case of neural text degeneration. *ICLR*, 2020.
- N. Carlini, F. Tramèr, E. Wallace, et al. Extracting training data from large language models. *USENIX Security '21*, 2021.
- N Tacheny. Geometric dynamics of agentic loops in large language models, 2025. URL <https://arxiv.org/abs/2512.10350>.
- M. Tuci, C. Korkmaz, U. Şimşekli, and T Birdal. Generalization at the edge of stability, 2026. URL <https://arxiv.org/abs/2604.19740>.